UNIT-I

Introduction to AI

In today's world, technology is growing very fast, and we are getting in touch with different new technologies day by day.

Here, one of the booming technologies of computer science is Artificial Intelligence which is ready to create a new revolution in the world by making intelligent machines. Artificial Intelligence is now all around us. It is currently working with a variety of subfields, ranging from general to specific, such as self-driving cars, playing chess, proving theorems, playing music, Painting, etc.

AI is one of the fascinating and universal fields of Computer Science which has a great scope in future. AI holds a tendency to cause a machine to work as a human.

Artificial Intelligence is composed of two words **Artificial** and **Intelligence**, where Artificial defines *"man-made"* i.e., made or produced by human being rather than occurring naturally or a copy of something like naturally. and intelligence defines *"thinking power"* i.e., it is defined as the ability to learn, understand, and apply knowledge and skills. hence AI means *"a man-made thinking power."*

So, we can define AI as:

"It is a branch of computer science by which we can create intelligent machines which can behave like a human, think like humans, and able to make decisions."

Or

Artificial intelligence (AI) is the simulation of human intelligence in machines that are programmed to think and act like humans. Learning,

reasoning, problem-solving, perception, and language comprehension are all examples of cognitive abilities.

Artificial Intelligence is a method of making a computer, a computer-controlled robot, or a software think intelligently like the human mind. AI is accomplished by studying the patterns of the human brain and by analysing the cognitive process. The outcome of these studies develops intelligent software and systems.

Artificial Intelligence exists when a machine can have human based skills such as learning, reasoning, and solving problems.

With Artificial Intelligence you do not need to pre-program a machine to do some work, despite that you can create a machine with programmed algorithms which can work with own intelligence, and that is the awesomeness of AI.

For example, Natural Language Processing (NLP) uses AI to analyze and interpret human language in text or speech. It enables applications like chatbots and virtual assistants to understand user queries, extract meaningful information, and deliver accurate, context-aware responses, transforming unstructured communication into actionable insights.

Core Concepts in AI

Artificial Intelligence (AI) operates on a core set of concepts and technologies that enable machines to perform tasks that typically require human intelligence. Here are some foundational concepts:

1. **Machine Learning (ML)**: This is the backbone of AI, where algorithms learn from data without being explicitly programmed. It involves training an algorithm on a data set, allowing it to improve over time and make predictions or decisions based on new data.

2. **Neural Networks**: Inspired by the human brain, these are networks of algorithms that mimic the way neurons interact, allowing computers to recognize patterns and solve common problems in the fields of AI, machine learning, and deep learning.

3. **Deep Learning**: A subset of ML, deep learning uses complex neural networks with many layers (hence "deep") to analyze various factors of data. This is instrumental in tasks like image and speech recognition.

4. **Natural Language Processing (NLP)**: NLP involves programming computers to process and analyze large amounts of natural language data, enabling interactions between computers and humans using natural language.

5. **Robotics**: While often associated with AI, robotics merges AI concepts with physical components to create machines capable of performing a variety of tasks, from assembly lines to complex surgeries.

6. **Cognitive Computing**: This AI approach mimics human brain processes to solve complex problems, often using pattern recognition, NLP, and data mining.

7. **Expert Systems**: These are AI systems that emulate the decision-making ability of a human expert, applying reasoning capabilities to reach conclusions.

Each of these concepts helps to build systems that can automate, enhance, and sometimes outperform human capabilities in specific tasks.

How Does AI Work?

Artificial intelligence (AI) enables machines to learn from data and recognize patterns in it, to perform tasks more efficiently and effectively. AI works in five steps:

- **Input:** Data is collected from various sources. This data is then sorted into categories.
- **Processing:** The AI sorts and decodes the data using patterns it has been programmed to learn until it recognizes similar patterns in the data.
- **Outcomes:** The AI can then use those patterns to predict outcomes.
- **Adjustments:** If the data sets are considered a "fail," AI learns from that mistake, and the process is repeated under different conditions.
- **Assessments:** In this way, AI is constantly learning and improving.

Why Artificial Intelligence?

Before Learning about Artificial Intelligence, we should know that what is the importance of AI and why should we learn it. Following are some main reasons to learn about AI:

- With the help of AI, you can create such software or devices which can solve real-world problems very easily and with accuracy such as health issues, marketing, traffic issues, etc.
- With the help of AI, you can create your personal virtual Assistant, such as Cortana, Google Assistant, Siri, etc.
- With the help of AI, you can build such Robots which can work in an environment where survival of humans can be at risk.
- AI opens a path for other new technologies, new devices, and new Opportunities.

Goals of Artificial Intelligence:

Following are the main goals of Artificial Intelligence:

1. Replicate human intelligence
2. Solve Knowledge-intensive tasks
3. An intelligent connection of perception and action
4. Building a machine which can perform tasks that requires human intelligence such as:
    - Proving a theorem
    - Playing chess
    - Plan some surgical operation
    - Driving a car in traffic
5. Creating some system which can exhibit intelligent behavior, learn new things by itself, demonstrate, explain, and can advise to its user.

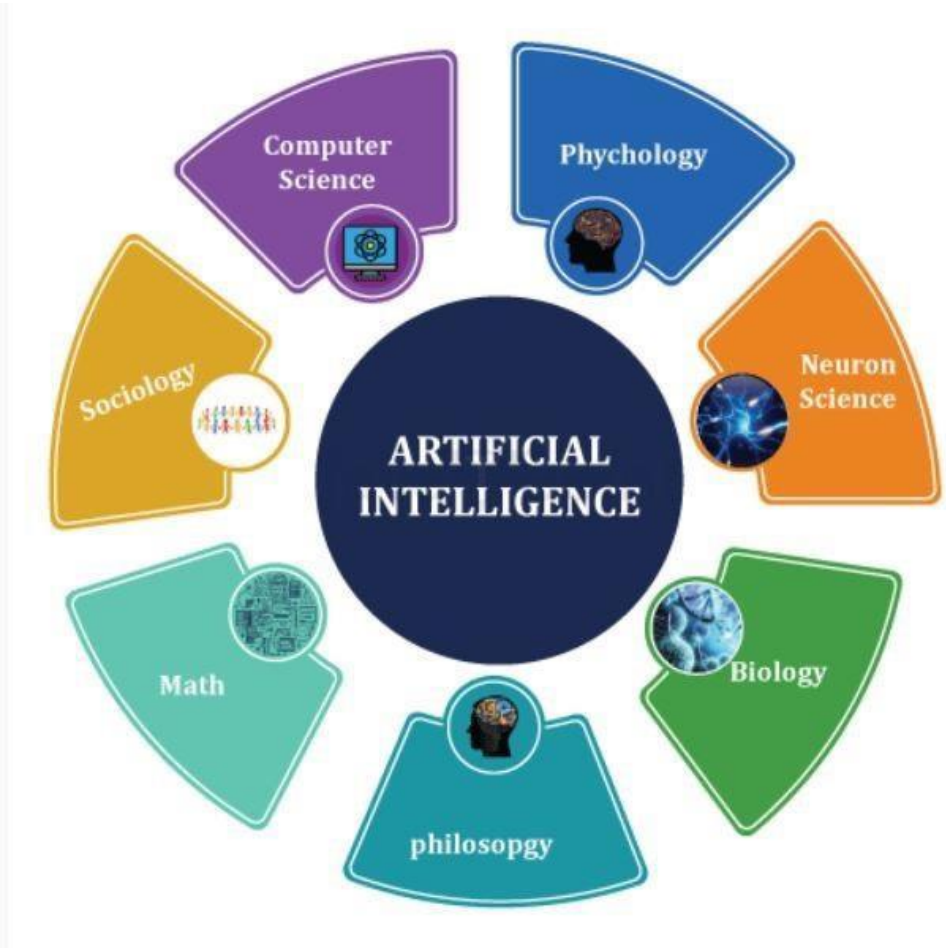
What Comprises to Artificial Intelligence?

Artificial Intelligence is not just a part of computer science even it's so vast and requires lots of other factors which can contribute to it. To create the AI first we should know that how intelligence is composed, so the Intelligence is an intangible part of our brain which is a combination of **Reasoning, learning, problem-solving perception, language understanding, etc**.

To achieve the above factors for a machine or software Artificial Intelligence requires the following discipline:

- Mathematics
- Biology

- Psychology
- Sociology
- Computer Science
- Neurons Study
- Statistics



Types of AI (Artificial Intelligence):

When discussing artificial intelligence (AI), it is common to distinguish between three broad categories: weak AI, strong AI and super AI. Let's explore the characteristics of each type:

- **Weak AI (Narrow AI):**

Weak AI refers to AI systems that are designed to perform specific tasks and are limited to those tasks only. These AI systems excel at their designated functions but lack general intelligence.

Examples of weak AI include voice assistants like Siri or Alexa, recommendation algorithms, and image recognition systems. Weak AI operates within predefined boundaries and cannot generalize beyond their specialized domain.

- **Strong AI (General AI):**

Strong AI, also known as general AI, refers to AI systems that possess human-level intelligence or even surpass human intelligence across a wide range of tasks. Strong AI would be capable of understanding, reasoning, learning, and applying knowledge to solve complex problems in a manner like human cognition. However, the development of strong AI is still largely theoretical and has not been achieved to date.

- **Super AI (S AI)**:

Super intelligent AI, or Artificial Superintelligence, represents an AI that not only mimics but significantly surpasses human intelligence across all fields — science, general wisdom, social skills, and more. ASI would be capable of extraordinary problem-solving and creative abilities, far beyond what current human minds can achieve.

Advantages of Artificial Intelligence:

The widespread adoption of Artificial Intelligence (AI) has brought about numerous benefits and advantages across various industries and aspects of our lives. Here are some of the key benefits of AI:

1. **Improved Efficiency and Productivity:** AI-powered systems can perform tasks with greater speed, accuracy, and consistency than humans, leading to improved efficiency and productivity in various industries. This can result in cost savings, reduced errors, and increased output.

2. **Enhanced Decision-Making:** AI algorithms can analyze large amounts of data, identify patterns, and make informed decisions faster than humans. This can be particularly useful in fields such as finance, healthcare, and logistics, where timely and accurate decision-making is critical.

3. **Personalization and Customization:** AI-powered systems can learn from user behavior and preferences to provide personalized recommendations, content, and experiences. This can lead to increased customer satisfaction and loyalty, as well as improved targeting and marketing strategies.

4. **Automation of Repetitive Tasks:** AI can be used to automate repetitive, time-consuming tasks, freeing up human resources to focus on more strategic and creative work. This can lead to cost savings, reduced errors, and improved work-life balance for employees.

5. **Improved Safety and Risk Mitigation:** AI-powered systems can be used to enhance safety in various applications, such as autonomous vehicles, industrial automation, and medical diagnostics. AI algorithms

can also be used to detect and mitigate risks, such as fraud, cybersecurity threats, and environmental hazards.

6. **Advancements in Scientific Research:** AI can assist in scientific research by analysing large datasets, generating hypotheses, and accelerating the discovery of new insights and breakthroughs. This can lead to advancements in fields such as medicine, climate science, and materials science.

7. **Enhanced Human Capabilities:** AI can be used to augment and enhance human capabilities, such as improving memory, cognitive abilities, and decision-making. This can lead to improved productivity, creativity, and problem-solving skills.

Disadvantages of Artificial Intelligence:

Every technology has some disadvantages, and the same goes for Artificial intelligence. Being so advantageous technology still, it has some disadvantages which we need to keep in our mind while creating an AI system. Following are the disadvantages of AI:

- **High Cost:** The hardware and software requirement of AI is very costly as it requires lots of maintenance to meet current world requirements.
- **Can't think out of the box:** Even we are making smarter machines with AI, but still they cannot work out of the box, as the robot will only do that work for which they are trained or programmed.
- **No feelings and emotions:** AI machines can be an outstanding performer, but still it does not have the feeling so it cannot make any kind of emotional attachment with human and may sometime be harmful for users if the proper care is not taken.

- **Increase dependency on machines:** With the increment of technology, people are getting more dependent on devices and hence they are losing their mental capabilities.
- **No Original Creativity:** As humans are so creative and can imagine some new ideas but still AI machines cannot beat this power of human intelligence and cannot be creative and imaginative.
- **Complexity:** Making and keeping AI systems can be very complicated and need a lot of knowledge. This can make it hard for some groups or people to use them.
- **Job Concerns:** As AI gets better, it might take away not just basic jobs but also some skilled ones. This worries people about losing jobs in different fields.

Challenges of AI:

Artificial Intelligence offers incredible advantages, but it also presents some challenges that need to be addressed:

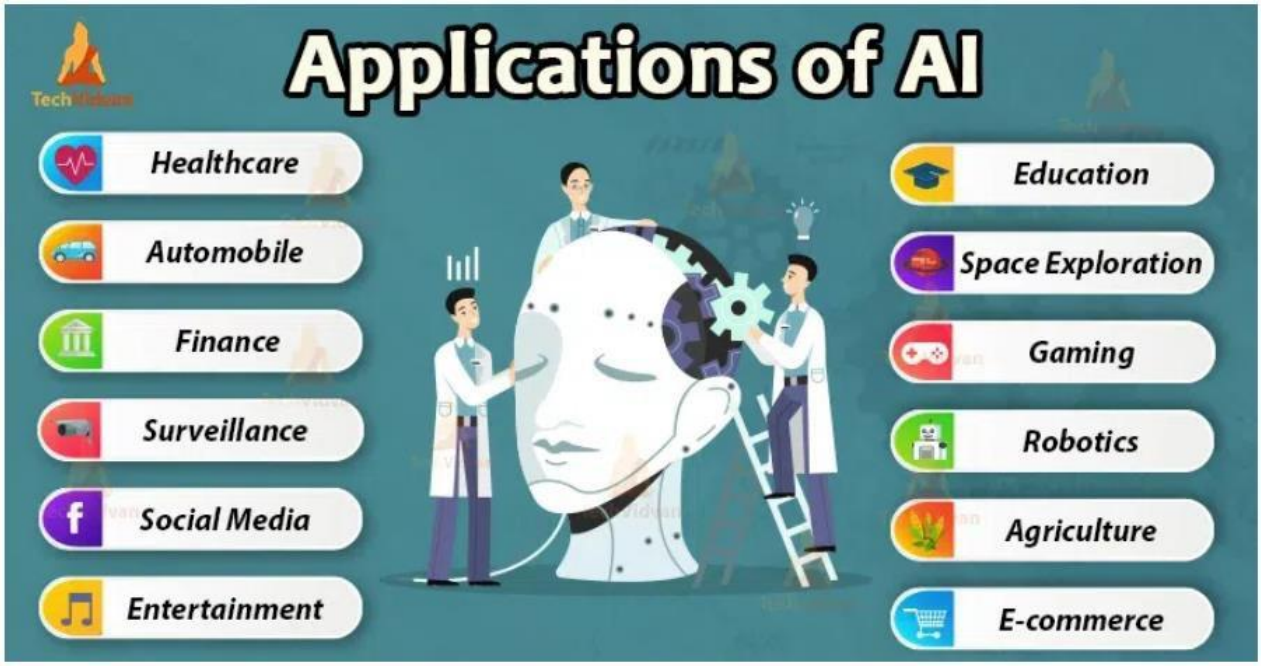
- **Doing the Right Thing:** AI should make the right choices, but sometimes it doesn't. It can make mistakes or do things that aren't fair. We need to teach AI to be better at making good choices.
- **Government and AI:** Sometimes, governments use AI to keep an eye on people. This can be a problem for our freedom. We need to make sure they use AI in a good way.
- **Bias in AI:** AI can sometimes be a bit unfair, especially when it comes to recognizing people's faces. This can cause problems, especially for people who aren't like the majority.

- **AI and social media:** What you see on social media is often decided by AI. But sometimes, AI shows things that aren't true or are kind of mean. We need to make sure AI shows the right stuff.

- **Legal and Regulatory Challenges:** The rapid evolution of AI has outpaced the development of comprehensive laws and regulations, leading to uncertainty about issues like liability and responsibility.

Application of Artificial Intelligence :

Artificial Intelligence has many practical applications across various industries and domains, including:



1. **Healthcare –** AI is used for medical diagnosis by analyzing medical images like X-rays and MRIs to identify diseases. For instance, AI systems are being developed to detect skin cancer from images with high accuracy.

2. **Finance** – AI helps in credit scoring by analysing a borrower's financial history and other data to predict their creditworthiness. This helps banks decide whether to approve a loan and at what interest rate.

3. **Retail** – AI is used for product recommendations by analysing your past purchases and browsing behavior to suggest products you might be interested in. For example, Amazon uses AI to recommend products to customers on their website.

4. **Manufacturing –** AI helps in quality control by inspecting products for defects. AI systems can be trained to identify even very small defects that human inspectors might miss.

5. **Transportation –** AI is used for autonomous vehicles by developing self-driving cars that can navigate roads without human input. Companies like Waymo and Tesla are developing self-driving car technology.

6. **Customer service –** AI-powered chatbots are used to answer customer questions and provide support. For instance, many banks use chatbots to answer customer questions about their accounts and transactions.

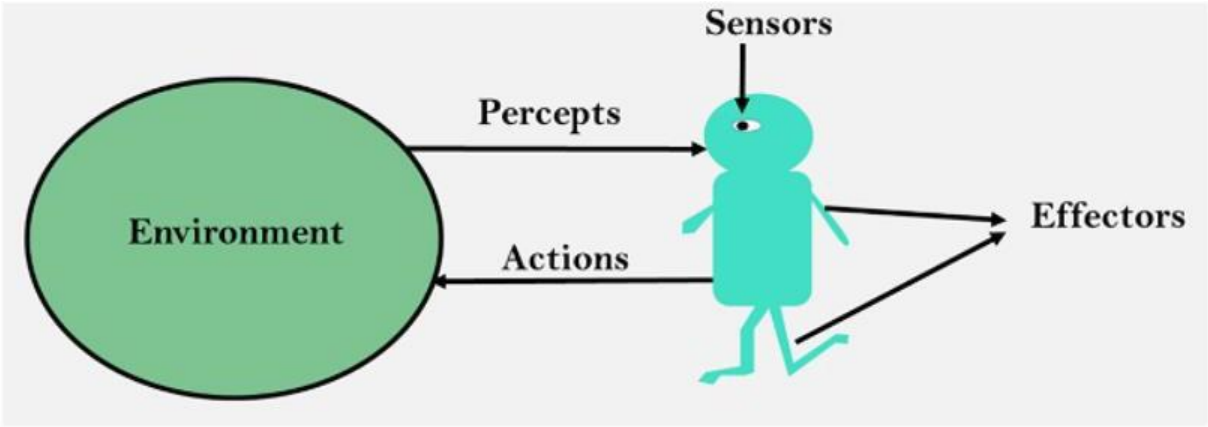
7. **Security –** AI is used for facial recognition by identifying people from images or videos. This technology is used for security purposes, such as identifying criminals or unauthorized individuals.

8. **Marketing –** AI is used for targeted advertising by showing ads to people who are most likely to be interested in the product or service being advertised. For example, social media companies use AI to target ads to users based on their interests and demographics.

9. **Education –** AI is used for personalized learning by tailoring educational content to the individual needs of each student. For example, AI-powered tutoring systems can provide students with personalized instruction and feedback.

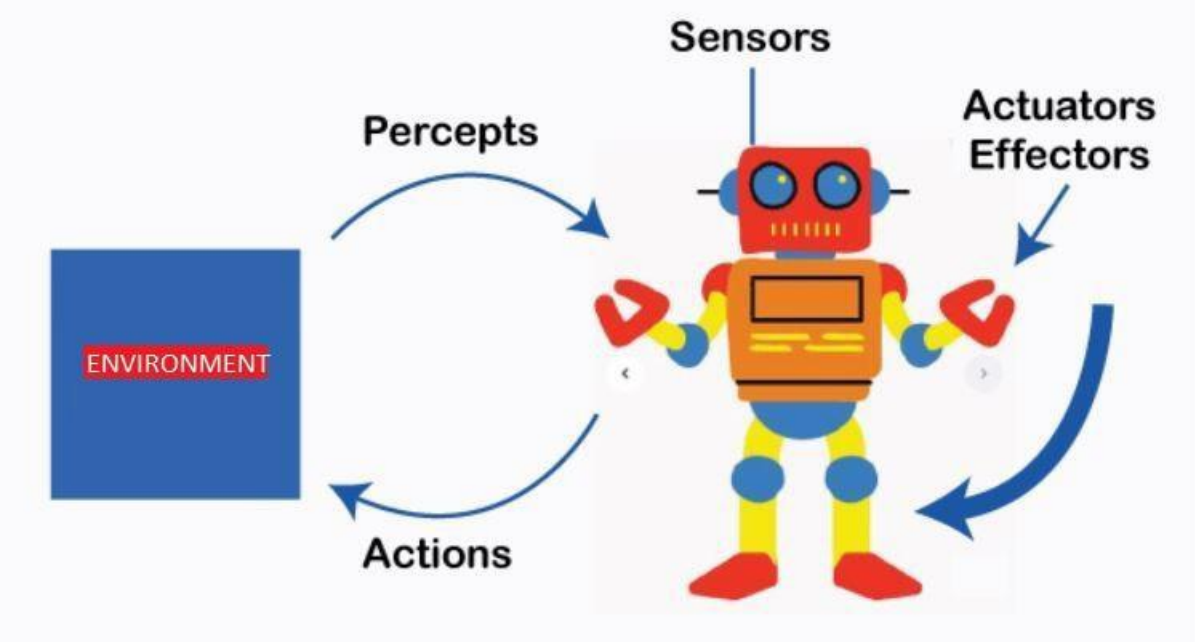
Agents in Artificial Intelligence

An AI system can be defined as the study of the rational agent and its environment. The agents sense the environment through sensors and act on their environment through actuators. An AI agent can have mental properties such as knowledge, belief, intention, etc.



What is an Agent?

An agent can be anything that perceive its environment through sensors and act upon that environment through actuators. An Agent runs in the cycle of perceiving, thinking, and acting.



An agent can be:

- **Human-Agent:** A human agent has eyes, ears, and other organs which work for sensors and hand, legs, vocal tract work for actuators.
- **Robotic Agent:** A robotic agent can have cameras, infrared range finder, NLP for sensors and various motors for actuators.
- **Software Agent:** Software agent can have keystrokes, file contents as sensory input and act on those inputs and display output on the screen.

Hence the world around us is full of agents such as thermostat, cell phone, camera, and even we are also agents.

Before moving forward, we should first know about sensors, effectors, and actuators.

Sensor: Sensor is a device which detects the change in the environment and sends the information to other electronic devices. An agent observes its environment through sensors.

Actuators: Actuators are the component of machines that converts energy into motion. The actuators are only responsible for moving and controlling a system. An actuator can be an electric motor, gears, rails, etc.

Effectors: Effectors are the devices which affect the environment. Effectors can be legs, wheels, arms, fingers, wings, fins, and display screen.

Intelligent Agents

An intelligent agent is an autonomous entity which act upon an environment using sensors and actuators for achieving goals. An intelligent agent may learn from the environment to achieve their goals. A thermostat is an example of an intelligent agent.

Following are the main four rules for an AI agent:

- **Rule 1:** An AI agent must have the ability to perceive the environment.
- **Rule 2:** The observation must be used to make decisions.
- **Rule 3:** Decision should result in an action.
- **Rule 4:** The action taken by an AI agent must be a rational action.

Rational Agent

A rational agent is an agent which has clear preference, models uncertainty, and acts in a way to maximize its performance measure with all possible actions.

A rational agent is said to perform the right things. AI is about creating rational agents to use for game theory and decision theory for various real-world scenarios.

For an AI agent, the rational action is most important because in AI reinforcement learning algorithm, for each best possible action, agent gets the positive reward and for each wrong action, an agent gets a negative reward.

Note: Rational agents in AI are very similar to intelligent agents.

Rationality:

The rationality of an agent is measured by its performance measure. Rationality can be judged based on following points:

- Performance measure which defines the success criterion.
- Agent prior knowledge of its environment.
- Best possible actions that an agent can perform.
- The sequence of percept's.

Note: Rationality differs from Omniscience because an Omniscient agent knows the actual outcome of its action and act accordingly, which is not possible.

Structure of an AI Agent:

The task of AI is to design an agent program which implements the agent function. The structure of an intelligent agent is a combination of architecture and agent program. It can be viewed as:

Agent = Architecture + Agent program

Following are the main three terms involved in the structure of an AI agent:

Architecture: Architecture is machinery that an AI agent executes on.

Agent Function: Agent function is used to map a percept to an action.

$$f:P^* \rightarrow A$$

Agent program: Agent program is an implementation of agent function. An agent program executes on the physical architecture to produce function f.

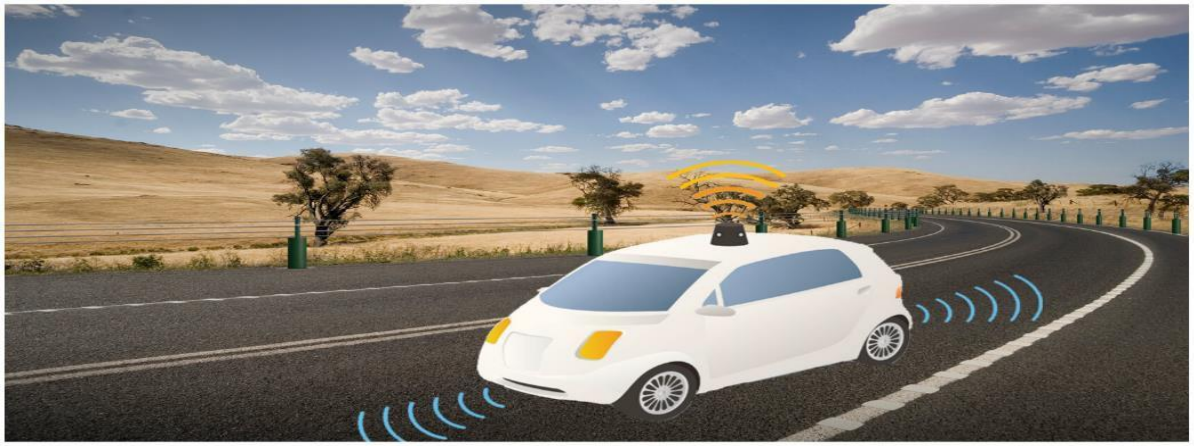
PEAS Representation:

PEAS is a type of model on which an AI agent works upon. When we define an AI agent or rational agent, then we can group its properties under PEAS representation model. It is made up of four words:

- **P:** Performance measure
- **E:** Environment
- **A:** Actuators
- **S:** Sensors

Here performance measure is the objective for the success of an agent's behavior.

PEAS for self-driving cars:



Let's suppose a self-driving car then PEAS representation will be:

Performance: Safety, time, legal drive, comfort

Environment: Roads, other vehicles, road signs, pedestrian

Actuators: Steering, accelerator, brake, signal, horn

Sensors: Camera, GPS, speedometer, odometer, accelerometer, sonar.

Example of Agents with their PEAS representation

Agent	Performance measure	Environment	Actuators	Sensors
1. Medical Diagnose	◦ Healthy patient ◦ Minimized cost	◦ Patient ◦ Hospital ◦ Staff	◦ Tests ◦ Treatments	Keyboard (Entry of symptoms)
2. Vacuum Cleaner	◦ Cleanness ◦ Efficiency ◦ Battery life ◦ Security	◦ Room ◦ Table ◦ Wood floor ◦ Carpet ◦ Various obstacles	◦ Wheels ◦ Brushes ◦ Vacuum Extractor	◦ Camera ◦ Dirt detection sensor ◦ Cliff sensor ◦ Bump Sensor ◦ Infrared Wall Sensor
3. Part -picking Robot	◦ Percentage of parts in correct bins.	◦ Conveyor belt with parts, ◦ Bins	◦ Jointed Arms ◦ Hand	◦ Camera ◦ Joint angle sensors.

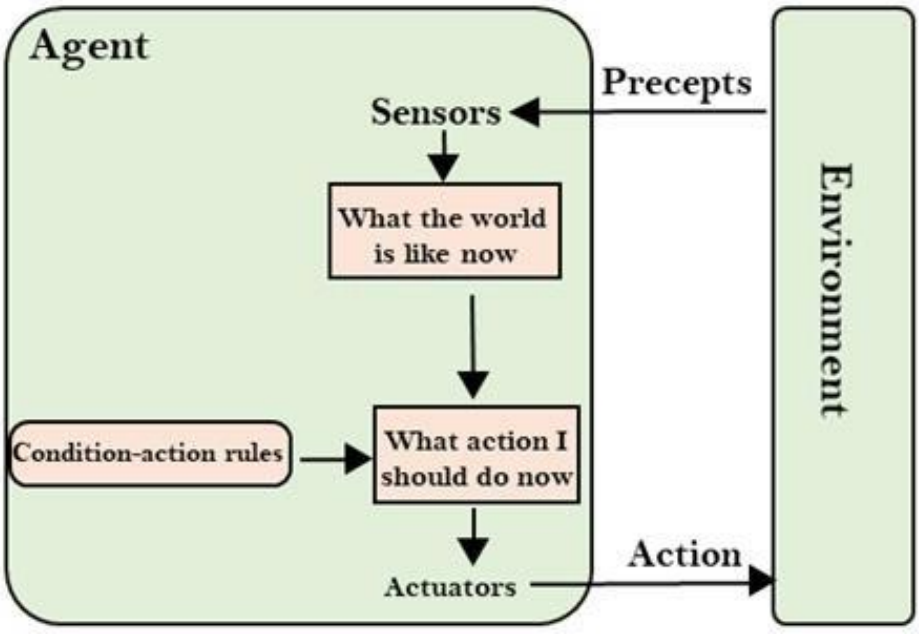
Types of Agents in AI

Agents are often grouped into five classes based on their degree of perceived intelligence and capability. All these agents can improve their performance and generate better action over time. These are given below:

- Simple Reflex Agent
- Model-Based Reflex Agent
- Goal-Based Agents
- Utility-Based Agent
- Learning Agent

Simple Reflex Agent:

- The Simple reflex agents are the simplest agents. These agents take decisions based on the current percepts and ignore the rest of the percept history.
- These agents only succeed in a fully observable environment.
- The Simple reflex agent does not consider any part of percepts history during their decision and action process.
- The Simple reflex agent works on Condition-action rule, which means it maps the current state to action. Such as a Room Cleaner agent, it works only if there is dirt in the room.

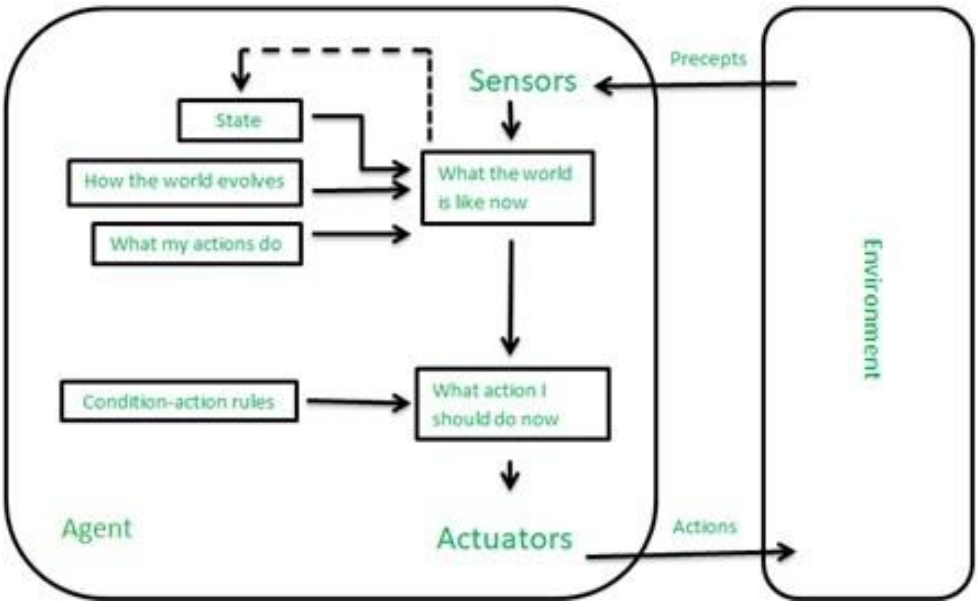


Problems with the straightforward reflex agent design approach:

- They have very limited intelligence.
- They do not have knowledge of non-perceptual parts of the current state.
- Mostly too big to get and to store.
- Not adaptive to changes within the environment.

Model-Based Reflex Agent:

- The Model-based agent can work in a partially observable environment and track the situation.
- A model-based agent has two important factors:
  - **Model:** It is knowledge about "how things happen in the world," so it is called a Model-based agent.
  - **Internal State:** It is a representation of the current state based on percept history.
- These agents have the model, "which is knowledge of the world" and based on the model they perform actions.
- Updating the agent state requires information about:
  a. How the world evolves
  b. How the agent's action affects the world.

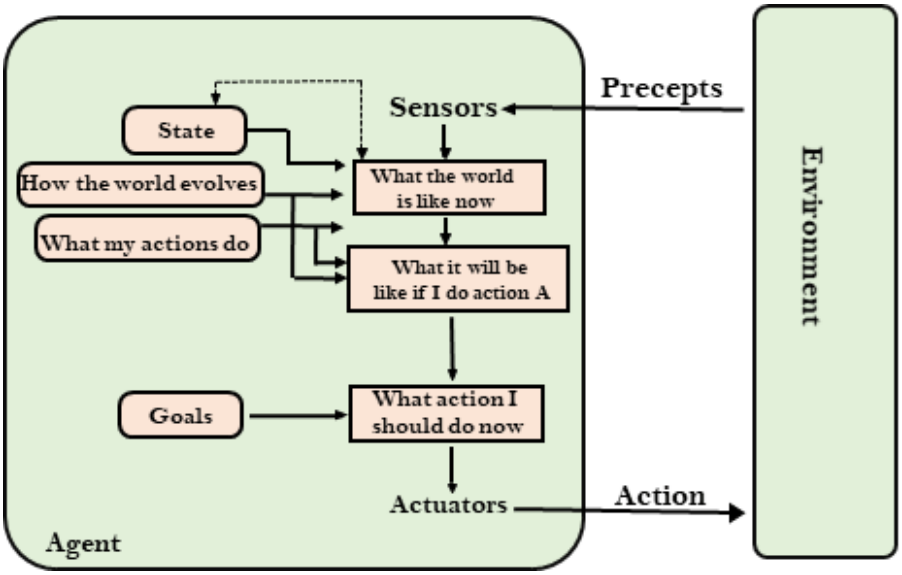


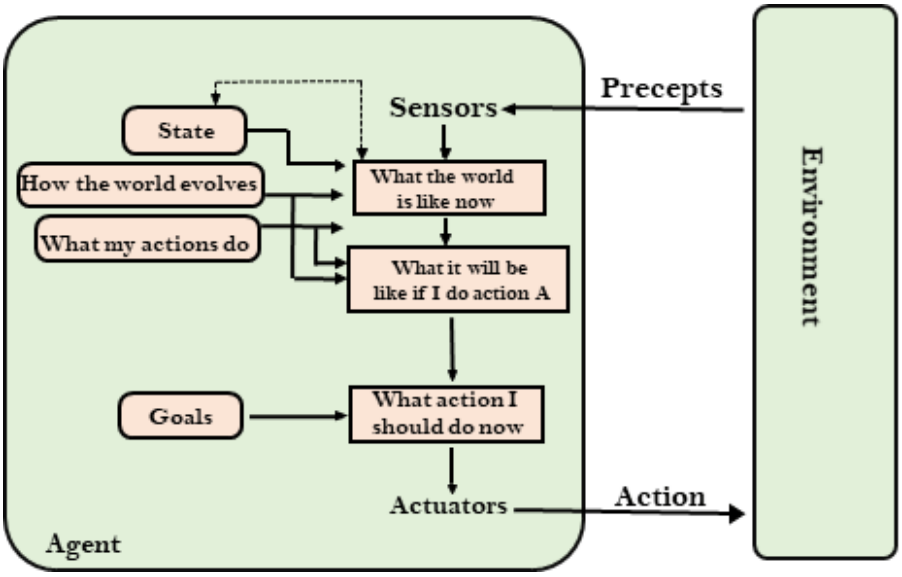
There are some important points that we need to remember about model-based reflex agents:

- Unlike simple reflex agents that rely solely on the current percept, model-based reflex agents consider a broader context.
- It can adapt its behavior based on changes in the environment or new information.
- The ability to reason, plan, and consider a wider context generally leads to improved performance compared to simple reflex agents.

Goal-Based Agent:

- The knowledge of the current state environment is not always sufficient to decide for an agent to what to do.
- The agent needs to know its goal which describes desirable situations.
- Goal-based agents expand the capabilities of the model-based agent by having the "goal" information.
- They choose an action, so that they can achieve the goal.
- These agents may have to consider a long sequence of possible actions before deciding whether the goal is achieved or not. Such considerations of different scenarios are called searching and planning, which makes an agent proactive.



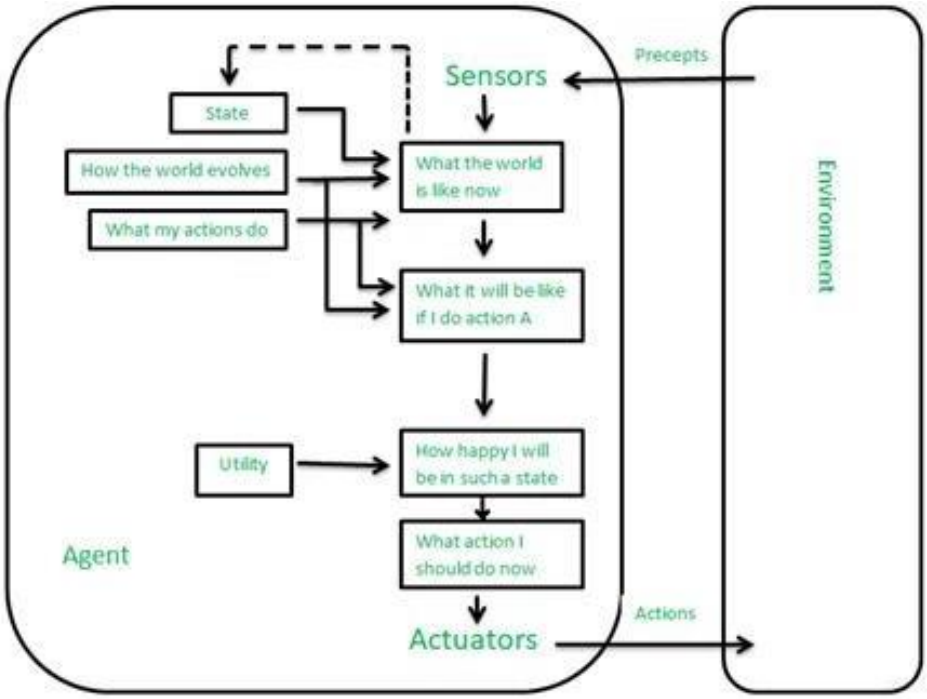

Precepts
Sensors
State
How the world evolves
What my actions do
What the world is like now
What it will be like if I do action A
Goals
What action I should do now
Actuators
Action
Agent
Environment

There are some points about goal-based agent:

- They choose action, in order that they will achieve the goal.
- These agents may need to consider an extended sequence of possible actions before deciding whether the goal is achieved or not.
- Considerations of various scenarios are called searching and planning, which makes an agent proactive.
- Once a plan of action is selected, the goal-based agent executes the actions in the environment.

Utility-Based Agents:

- These agents are like the goal-based agent but provide an extra component of utility measurement which makes them different by providing a measure of success at a given state.
- Utility-based agents act based not only goals but also the best way to achieve the goal.
- The Utility-based agent is useful when there are multiple possible alternatives, and an agent must choose to perform the best action.
- The utility function maps each state to a real number to check how efficiently each action achieves the goals.



There are some points about utility-based agents:

- These agents act based not only on goals but also on the simplest thanks to achieving the goal.

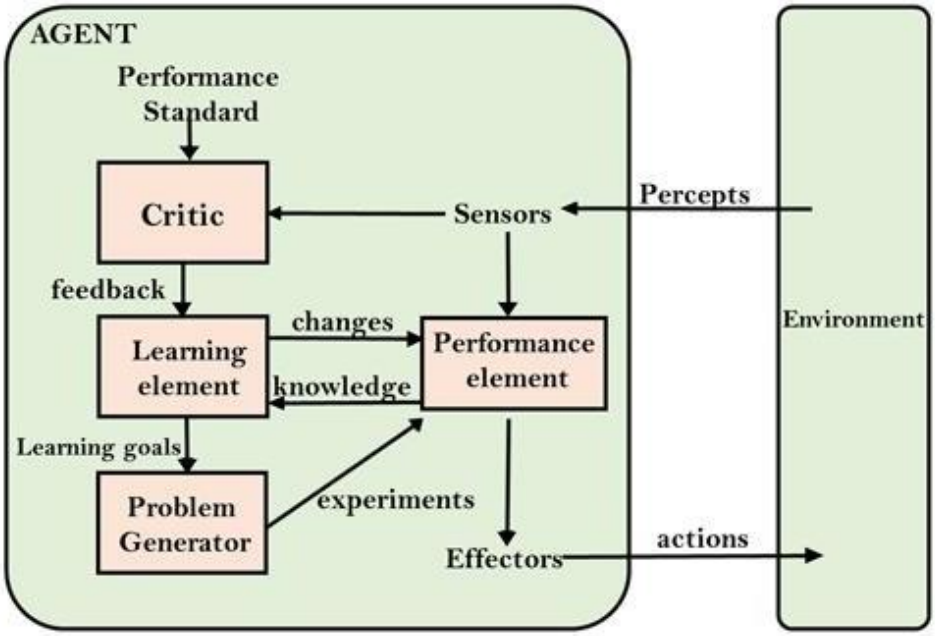
- These agents are beneficial when there are multiple possible alternatives, and an agent has got to prefer to perform the simplest action.

- The utility function maps each state to a true number to see how efficiently each action achieves the goals.

- These agents aim to make rational decisions that maximize their expected utility.

Learning Agents:

- A learning agent in AI is the type of agent which can learn from its past experiences, or it has learning capabilities.
- It starts to act with basic knowledge and then able to act and adapt automatically through learning.
- A learning agent has mainly four conceptual components, which are:
    a. **Learning element:** It is responsible for making improvements by learning from the environment.
    b. **Critic:** Learning element takes feedback from critics which describes that how well the agent is doing with respect to a fixed performance standard.
    c. **Performance element:** It is responsible for selecting external action.
    d. **Problem generator:** This component is responsible for suggesting actions that will lead to new and informative experiences.
- Hence, learning agents can learn, analyze performance, and look for new ways to improve their performance.



The Functions of an Artificial Intelligence Agent:

The functions of an agent in artificial intelligence are as follows:

- To resolve complex issues using intelligent machines.
- To decide what to do in a specific situation.
- To make conclusions and take decisions.
- The perception of dynamic environmental circumstances.
- Using logic to interpret perceptions.
- To try to change environmental conditions.

Uses of Agents:

Artificial Intelligence agents are used in various real-life applications.

Medical Evaluation:

- The surroundings are regarded as patient.
- The sensor that collects information on the patient's complaints is a computer keyboard.

- The intelligent agent uses this data to determine the best plan of action.
- Actuators used in healthcare include tests and therapies.

Automatic Vehicles:

- Various sensors are used in automatic vehicles to gather data from the surroundings.
- These consist of radar, GPS, and cameras.
- The environment in these agents could consist of people, other cars, roads, or road signs. Actions are started using a variety of devices. For instance, the car's brakes are used to stop them. Self-driving vehicles operate better with the assistance of intelligent agents.

Office Works Automation:

- The monotonous workplace tasks can be resolved using AI agents.
- Customer service and sales are two functional areas that have been automated.
- To cut operating expenses, some businesses have automated a few administrative processes.
- Office efficiency has also increased by using intelligent agents.

Agent Environment in AI

The environment is everything in the world which surrounds the agent, but it is not a part of an agent itself. An environment can be described as a situation in which an agent is present.

The environment is where the agent lives, operates, and provides the agent with something to sense and act upon it. The environment is mostly said to be non-feministic.

Features of Environment:

As for Russell and Norvig, an environment can have various features from the point of view of an agent:

1. Fully observable vs Partially Observable
2. Static vs Dynamic
3. Discrete vs Continuous
4. Deterministic vs Stochastic
5. Single-agent vs multi-agent
6. Episodic vs sequential
7. Known vs Unknown.
8. Accessible vs Inaccessible

1. Fully observable vs Partially Observable:

- For an agent sensor can sense or access the complete state of an environment at each point of time then it is **a fully observable** environment, else it is **partially observable**.
- A fully observable environment is easy as there is no need to maintain the internal state to keep track of the history of the world.
- An agent with no sensors in all environments than such an environment is called **unobservable**.

2. Deterministic vs Stochastic:

- If an agent's current state and selected action can completely determine the next state of the environment, then such an environment is called a deterministic environment.
- A stochastic environment is random in nature and cannot be determined completely by an agent.
- In a deterministic, fully observable environment, agents do not need to worry about uncertainty.

3. Episodic vs Sequential:

- In an episodic environment, there is a series of one-shot actions, and only the current percept is required for the action.
- However, in Sequential environment, an agent requires memory of past actions to determine the next best actions.

4. Single-agent vs multi-agent

- If only one agent is involved in an environment and operating by itself then such an environment is called single agent environment.
- However, if multiple agents are operating in an environment, then such an environment is called a multi-agent environment.
- The agent design problems in the multi-agent environment are different from single agent environment.

5. Static vs Dynamic:

- If the environment can change itself while an agent is deliberating, then such an environment is called a dynamic environment else it is called a static environment.
- Static environments are easy to deal with because an agent does not need to continue looking at the world while deciding on an action.

- However, for a dynamic environment, agents need to keep looking at the world at each action.
- Taxi driving is an example of a dynamic environment whereas Crossword puzzles are an example of a static environment.

6. Discrete vs Continuous:

- If in an environment there are a finite number of percepts and actions that can be performed within it, then such an environment is called a discrete environment else it is called continuous environment.
- A chess game comes under a discrete environment as there is a finite number of moves that can be performed.
- A self-driving car is an example of a continuous environment.

7. Known vs Unknown

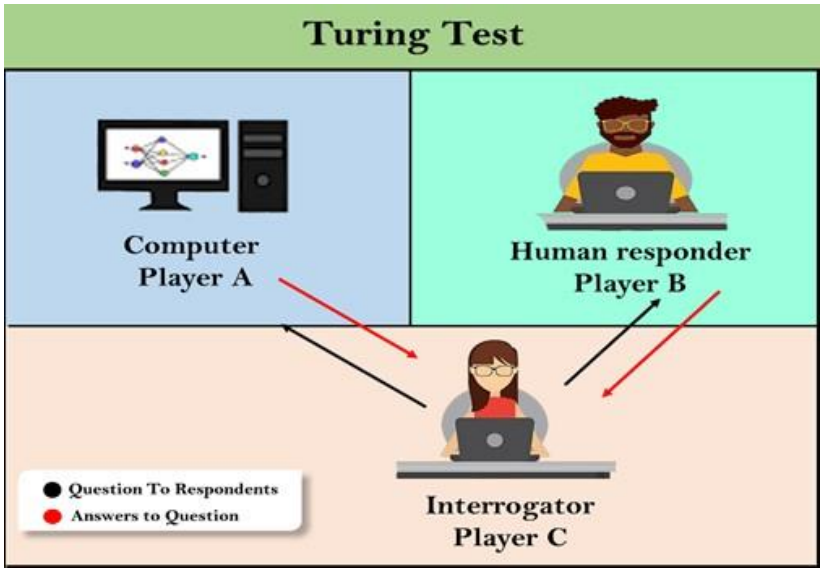
- Known and unknown are not actually a feature of an environment, but it is an agent's state of knowledge to perform an action.
- In a known environment, the results for all actions are known to the agent. While in unknown environment, agent needs to learn how it works to perform an action.
- It is quite possible that a known environment to be partially observable and an Unknown environment to be fully observable.

8. Accessible vs Inaccessible

- If an agent can obtain complete and accurate information about the state's environment, then such an environment is called an Accessible environment else it is called inaccessible.
- An empty room whose state can be defined by its temperature is an example of an accessible environment.
- Information about an event on earth is an example of Inaccessible environment.

Turing Test in AI

- In 1950, Alan Turing introduced a test to check whether a machine can think like a human or not. This test is known as the Turing Test. In this test, Turing proposed that the computer can be said to be intelligent if it can mimic human response under specific conditions.
- Turing Test was introduced by Turing in his 1950 paper, "Computing Machinery and Intelligence," which considered the question, "Can Machine think?"



The Turing test is based on a party game "Imitation game," with some modifications. This game involves three players in which one player is Computer, another player is human responder, and the third player is a human Interrogator, who is isolated from other two players and his job are to find that which player is machine among two of them.

Consider, Player A is a computer, Player B is human, and Player C is an interrogator. The interrogator is aware that one of them is a machine, but he needs to identify this based on questions and their responses.

The conversation between all players is via keyboard and screen so the result would not depend on the machine's ability to convert words as speech.

The test result does not depend on each correct answer, but only how closely its responses are like a human answer. The computer is permitted to do everything possible to force a wrong identification by the interrogator.

The questions and answers can be like:

Interrogator: Are you a computer?

Player A (Computer): No

Interrogator: Multiply two large numbers such as (256896489*456725896)

Player A: Long pause and give the wrong answer.

In this game, if an interrogator is not able to identify which is a machine and which is human, then the computer passes the test successfully, and the machine is said to be intelligent and can think like a human.

"In 1991, the New York businessman Hugh Loebner announces the prize competition, offering a $100,000 prize for the first computer to pass the Turing test. However, no AI program to till date, come close to passing an undiluted Turing test".

Chatbots to attempt the Turing test:

ELIZA: ELIZA was a Natural language processing computer program created by Joseph Weizenbaum. It was created to demonstrate the ability of communication between machines and humans. It was one of the first chatterbots, which attempted the Turing Test.

Parry: Parry was a chatterbot created by Kenneth Colby in 1972. Parry was designed to simulate a person with **Paranoid schizophrenia** (most common chronic mental disorder). Parry was described as "ELIZA with attitude." Parry was tested using a variation of the Turing Test in the early 1970s.

Eugene Goostman: Eugene Goostman was a chatbot developed in Saint Petersburg in 2001. This bot has competed in the various number of Turing

Test. In June 2012, at an event, Goostman won the competition promoted as largest-ever Turing test content, in which it convinced 29% of judges that it was a human. Goostman resembled as a 13-year old virtual boy.

The Chinese Room Argument:

There were many philosophers who really disagreed with the complete concept of Artificial Intelligence. The most famous argument in this list was "**Chinese Room**."

In the year **1980, John Searle** presented "**Chinese Room**" thought experiment, in his paper "**Mind, Brains, and Program**," which was against the validity of Turing's Test. According to his argument, "**Programming a computer may make it understand a language, but it will not produce a real understanding of language or consciousness in a computer**."

He argued that Machine such as ELIZA and Parry could easily pass the Turing test by manipulating keywords and symbol, but they had no real understanding of language. So, it cannot be described as "thinking" capability of a machine such as a human.

Features required for a machine to pass the Turing test:

- **Natural language processing:** NLP is required to communicate with Interrogator in general human language like English.
- **Knowledge representation:** To store and retrieve information during the test.
- **Automated reasoning:** To use the previously stored information for answering the questions.
- **Machine learning:** To adapt new changes and can detect generalized patterns.
- **Vision (For total Turing test):** To recognize the interrogator actions and other objects during a test.
- **Motor Control (For total Turing test):** To act upon objects if requested.

PROBLEM SOLVING AGENTS

PROBLEM-SOLVING APPROACH IN ARTIFICIAL INTELLIGENCE PROBLEMS

The **reflex agents** are known as the simplest agents because they directly map states into actions. Unfortunately, these agents fail to operate in an environment where the mapping is too large to store and learn. **Goal-based agents,** on the other hand, consider future actions and the desired outcomes.

Here, we will discuss one type of goal-based agent known as a **problem-solving agent**, which uses atomic representation with no internal states visible to the *problem-solving algorithms*.

Problem-solving agent:

The problem-solving agent performs precisely by defining problems and its several solutions.

- According to psychology, ***"a problem-solving refers to a state where we wish to reach to a definite goal from a present state or condition."***
- According to computer science, ***problem-solving is a part of artificial intelligence which encompasses several techniques such as algorithms and heuristics to solve a problem***.

Therefore, a problem-solving agent is a **goal-driven agent** and focuses on satisfying the goal.

PROBLEM DEFINITION:

To build a system to solve a particular problem, we need to do four things:

(i) **Define the problem precisely:** This definition must include specification of the initial situations and final situations which constitute (i.e.) an acceptable solution to the problem.

(ii) **Analyzing the problem**: important features have an immense (i.e) huge impact on the appropriateness of various techniques for solving the problems.

(iii) **Isolate and represent the knowledge** to solve the problem.

(iv) **Choose the best problem – solving techniques** and apply it to the problem.

<u>Steps performed by Problem-solving agent:</u>

- **Goal Formulation:** It is the first and simplest step in problem-solving. It organizes the steps/sequence required to formulate one goal out of multiple goals as well as actions to achieve that goal. Goal formulation is based on the current situation and the agent's performance measure (discussed below).
- **Problem Formulation:** It is the most important step of problem-solving which decides what actions should be taken to achieve the formulated goal.

There are the following five components involved in problem formulation:

- **Initial State:** It is the starting state or initial step of the agent towards its goal.
- **Actions:** It is the description of the possible actions available to the agent.

- **Transition Model:** It describes what each action does.
- **Goal Test:** It determines if the given state is a goal state.

  Path cost: It assigns a numeric cost to each path that follows the goal.

The problem- solving agent selects a cost function, which reflects its performance measure. Remember, **an optimal solution has the lowest path cost among all the solutions.**

Note: Initial state, actions, and **transition model** together define the **state-space** of the problem implicitly. State space of a problem is a set of all states which can be reached from the initial state followed by any sequence of actions. The state-space forms a directed map or graph where nodes are the states, links between the nodes are actions, and the path is a sequence of states connected by the sequence of actions.

- **Search:** It identifies all the best possible sequence of actions to reach the goal state from the current state. It takes a problem as an input and returns solution as its output.
- **Solution:** It finds the best algorithm out of various algorithms, which may be proven as thebest optimal solution.
- **Execution:** It executes the best optimal solution from the searching algorithms to reach thegoal state from the current state.

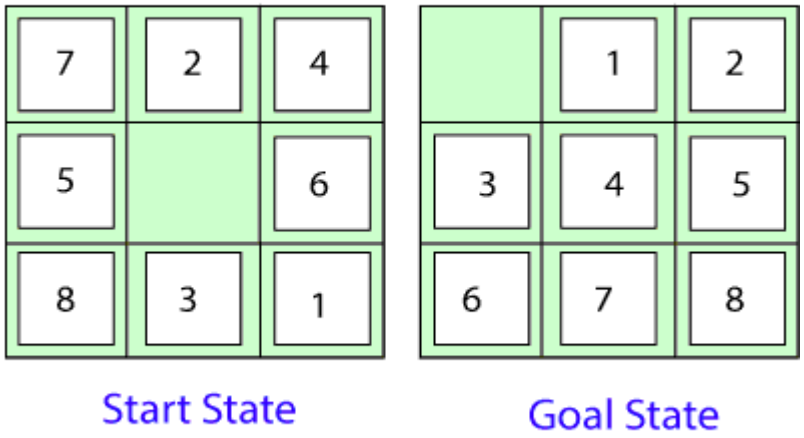
Example Problems:

Basically, there are two types of problem approaches:

- **Toy Problem:** It is a concise and exact description of the problem which is used by the researchers to compare the performance of algorithms.
- **Real-world Problem:** It is real-world based problems which require solutions. Unlike a toy problem, it does not depend on descriptions, but we can have a general formulation of the problem.

Some Toy Problems

- **8 Puzzle Problem:** Here, we have a 3×3 matrix with movable tiles numbered from 1 to 8 with a blank space. The tile adjacent to the blank space can slide into that space. The objective is to reach a specified goal state like the goal state, as shown in the below figure.
- In the figure, our task is to convert the current state into goal state by sliding digits into theblank space.

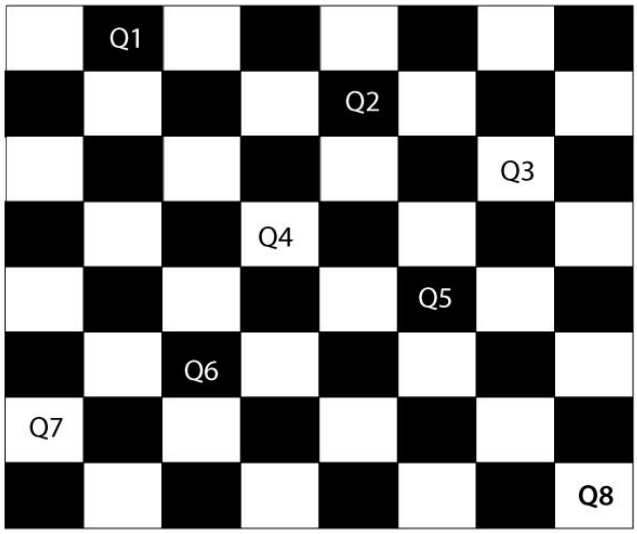


In the above figure, our task is to convert the current (Start) state into goal state by sliding digits into the blank space.

The problem formulation is as follows:

- **States:** It describes the location of each numbered tile and the blank tile.
- **Initial State:** We can start from any state as the initial state.
- **Actions:** Here, actions of the blank space are defined, i.e., either **left, right, up or down**
- **Transition Model:** It returns the resulting state as per the given state and actions.
- **Goal test:** It identifies whether we have reached the correct goal-state.
- **Path cost:** The path cost is the number of steps in the path where the cost of each step is 1. **Note:** The 8-puzzle problem is a type of **sliding-block problem** which is used for testing new search algorithms in artificial intelligence.
- **8-queens problem:** The aim of this problem is to place eight queens on a chessboard in an order where no queen may attack another. A queen can attack other queens either **diagonally or in the same row and column.**

From the following figure, we can understand the problem as well as its correct solution.



It is noticed from the above figure that each queen is set onto the chessboard in a position where no other queen is placed diagonally, in the same row or column. Therefore, it is one right approach to the 8-queens problem.

For this problem, there are two main kinds of formulation:

1. **Incremental formulation:** It starts from an empty state where the operator extends a queen at each step.

The following steps are involved in this formulation:

- **States:** Arrangement of any 0 to 7 queens on the chessboard.
- **Initial State:** An empty chessboard
- **Actions:** Add a queen to any empty box.
- **Transition model:** Returns the chessboard with the queen added in a box.
- **Goal test:** Checks whether 8-queens are placed on the chessboard without any attack.

- **Path cost:** There is no need for path cost because only final states are counted. In this formulation, there is approximately **1.8 x 10^{14}** possible sequence to investigate.

2. **Complete-state formulation:** It starts with all the 8-queens on the chessboard and moves themaround, saving from the attacks.

The following steps are involved in this formulation

- **States:** Arrangement of all the 8 queens one per column with no queen attacking the otherqueen.
- **Actions:** Move the queen at the location where it is safe from the attacks.

This formulation is better than the incremental formulation as it reduces the state space from **1.8 x10^{14}** to **2057**, and it is easy to find the solutions.

Some Real-world problems:

- **Traveling salesperson problem (TSP):** It is a **touring problem** where the salesman can visit each city only once. The objective is to find the shortest tour and sell out the stuff in each city.
- **VLSI Layout problem:** In this problem, millions of components and connections are positioned on a chip in order to minimize the area, circuit-delays, stray-capacitances, and maximizing the manufacturing yield.

  The layout problem is split into two parts:

  - **Cell layout:** Here, the primitive components of the circuit are grouped into cells, each performing its specific function. Each cell has a fixed shape and size. The task is to place the cells on the chip without overlapping each other.

- **Channel routing:** It finds a specific route for each wire through the gaps betweenthe cells.

Protein Design: The objective is to find a sequence of amino acids which will fold into 3D protein having a property to cure some disease.

Searching for solutions:

We have seen many problems. Now, there is a need to search for solutions to solve them.

In this section, we will understand how searching can be used by the agent to solve a problem.

For solving different kinds of problems, an agent makes use of different strategies to reach the goal by searching the best possible algorithms. This process of searching is known as **a search strategy.**

Search Algorithms in Artificial Intelligence

Search algorithms are one of the most important areas of Artificial Intelligence. This topic will explain all about the search algorithms in AI.

Problem-solving agents:

In Artificial Intelligence, Search techniques are universal problem-solving methods. **Rational agents** or **Problem-solving agents** in AI mostly used these search strategies or algorithms to solve a specific problem and provide the best result. Problem-solving agents are goal-based agents and use atomic representation. In this topic, we will learn various problem-solving search algorithms.

Search Algorithm Terminologies:

- **Search:** Searching is a step-by-step procedure to solve a search-problem in each search space. A search problem can have three main factors:
  a. **Search Space:** Search space represents a set of possible solutions, which system may have.
  b. **Start State:** It is a state from where the agent begins **the search**.
  c. **Goal test:** It is a function which observes the current state and returns whether the goal state is achieved or not.
- **Search tree:** A tree representation of search problem is called Search tree. The root of the search tree is the root node which corresponds to the initial state.
- **Actions:** It gives a description of all the available actions to the agent.
- **Transition model:** A description of what each action does can be represented as a transition model.
- **Path Cost:** It is a function which assigns a numeric cost to each path.
- **Solution:** It is an action sequence which leads from the start node to the goal node.

- **Optimal Solution:** If a solution has the lowest cost among all solutions.

Properties of Search Algorithms:

The following are the four essential properties of search algorithms to compare the efficiency of these algorithms:

Completeness: A search algorithm is said to be complete if it guarantees to return a solution if at least any solution exists for any random input.

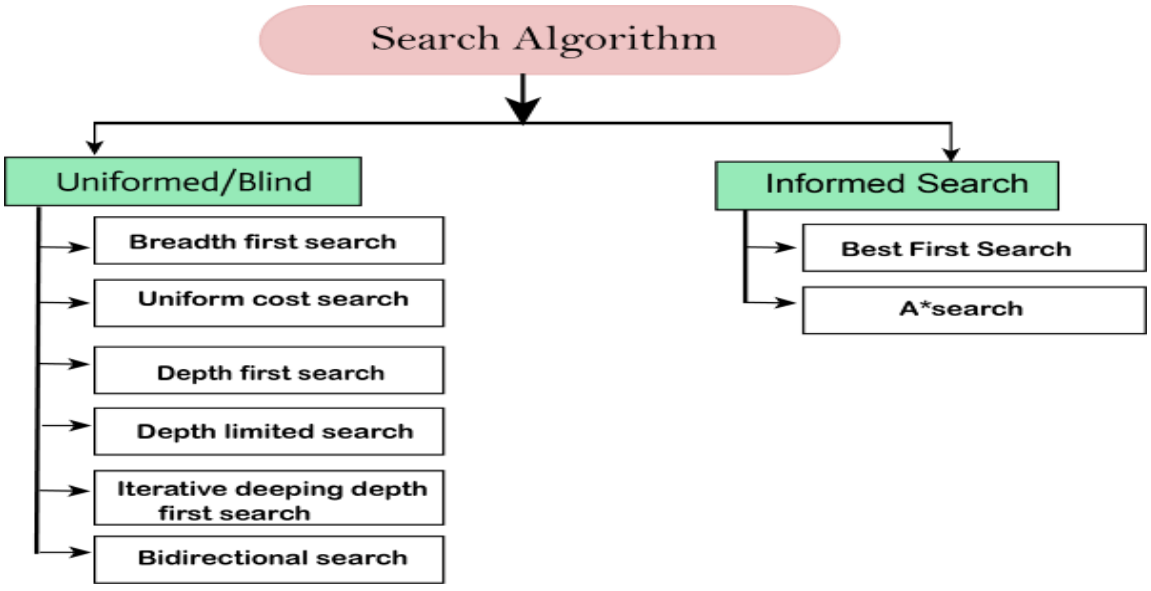
Optimality: If a solution found for an algorithm is guaranteed to be the best solution (lowest path cost) among all other solutions, then such a solution for is said to be an optimal solution.

Time Complexity: Time complexity is a measure of time for an algorithm to complete its task.

Space Complexity: It is the maximum storage space required at any point during the search, as the complexity of the problem.

Types of search algorithms

Based on the search problems we can classify the search algorithms into uninformed (Blind search) search and informed search (Heuristic search) algorithms.



Uninformed/Blind Search:

The uninformed search does not contain any domain knowledge such as closeness, the location of the goal. It operates in a brute-force way as it only includes information about how to traverse the tree and how to identify leaf and goal nodes. Uninformed search applies a way in which search tree is searched without any information about the search space like initial state operators and test for the goal, so it is also called blind search. It examines each node of the tree until it achieves the goal node.

It can be divided into five main types:

- Breadth-first search
- Uniform cost search
- Depth-first search
- Iterative deepening depth-first search
- Bidirectional Search

Informed Search:

Informed search algorithms use domain knowledge. In an informed search, problem information is available which can guide the search. Informed search strategies can find a solution more efficiently than an uninformed search strategy. Informed search is also called a Heuristic search.

A heuristic is a way which might not always be guaranteed for best solutions but guaranteed to find a good solution in reasonable time.

Informed search can solve many complex problem which cannot be solved in another way.

An example of informed search algorithms is a traveling salesman problem.

1. Greedy Search
2. A* Search

Uninformed Search Algorithms:

Uninformed search is a class of general-purpose search algorithms which operate in brute force-way. Uninformed search algorithms do not have additional information about state or search space other than how to traverse the tree, so it is also called blind search.

The following are the various types of uninformed search algorithms:

1. Breadth-first Search
2. Depth-first Search
3. Depth-limited Search
4. Iterative deepening depth-first search
5. Uniform cost search
6. Bidirectional Search

1. Breadth-first Search:

- Breadth-first search is the most common search strategy for traversing a tree or graph. This algorithm searches breadthwise in a tree or graph, so it is called breadth-first search.
- BFS algorithm starts searching from the root node of the tree and expands all successor node at the current level before moving to nodes of next level.
- The breadth-first search algorithm is an example of a general-graph search algorithm.
- Breadth-first search implemented using FIFO queue data structure.

Advantages:

- BFS will provide a solution if any solution exists.
- If there are more than one solution for a given problem, then BFS will provide the minimal solution which requires the least number of steps.

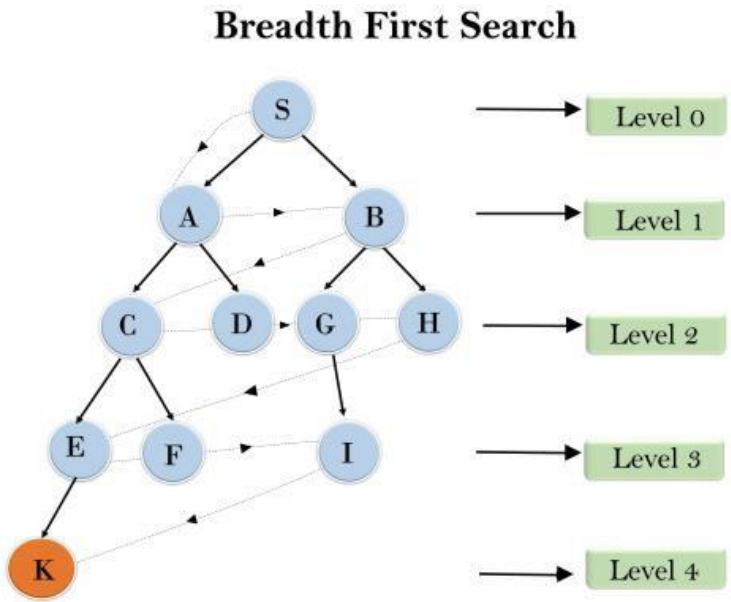
Disadvantages:

- It requires lots of memory since each level of the tree must be saved into memory to expand the next level.
- BFS needs lots of time if the solution is far away from the root node.

Example:

In the below tree structure, we have shown the traversing of the tree using BFS algorithm from the root node S to goal node K. BFS search algorithm traverse in layers, so it will follow the path which is shown by the dotted arrow, and the traversed path will be:

1. S---> A--->B---->C--->D---->G--->H--->E---->F---->I---- >K



Time Complexity: Time Complexity of BFS algorithm can be obtained by the number of nodes traversed in BFS until the shallowest Node. Where the d= depth of shallowest solution and b is a node at every state.

$T(b) = 1+b^2+b^3+\ldots\ldots+ b^d = O(b^d)$

Space Complexity: Space complexity of BFS algorithm is given by the Memory size of frontier which is $O(b^d)$.

Completeness: BFS is complete, which means if the shallowest goal node is at some finite depth, then BFS will find a solution.

Optimality: BFS is optimal if path cost is a non-decreasing function of the depth of the node.

2. Depth-first Search

- Depth-first search isa recursive algorithm for traversing a tree or graph data structure.
- It is called the depth-first search because it starts from the root node and follows each path to its greatest depth node before moving to the next path.
- DFS uses a stack data structure for its implementation.
- The process of the DFS algorithm is like the BFS algorithm.

Advantage:

- DFS requires very little memory as it only needs to store a stack of the nodes on the path from root node to the current node.
- It takes less time to reach the goal node than BFS algorithm (if it traverses in the right path).

Disadvantages:

- There is the possibility that many states keep re-occurring, and there is no guarantee of finding the solution.
- DFS algorithm goes for deep searching and sometimes it may go to the infinite loop.

Example:

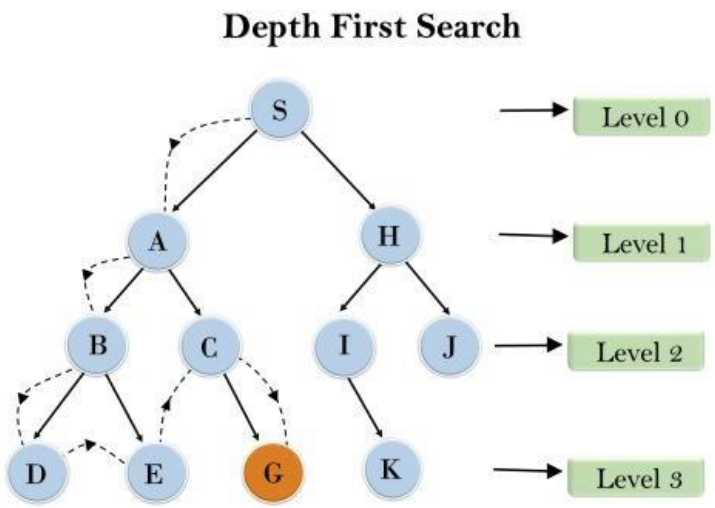
In the search below, we have shown the flow of depth-first search, and it will follow the order as:

Root node--->Left node ----> right node.

It will start searching from root node S, and traverse A, then B, then D and E. After traversing E, it will backtrack the tree as E has no other successor and still goal node is not found. After backtracking it will traverse node C and then G, and here it will terminate as it found goal node.

Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:



Completeness: DFS search algorithm is complete within finite state space as it will expand every node within a limited search tree.

Time Complexity: Time complexity of DFS will be equivalent to the node traversed by the algorithm. It is given by:

$T(n)= 1+ n^2+ n^3 +\ldots\ldots\ldots+ n^m=O(n^m)$

Where, m= maximum depth of any node and this can be much larger than d (Shallowest solution depth)

Space Complexity: DFS algorithm needs to store only single path from the root node, hence space complexity of DFS is equivalent to the size of the fringe set, which is **O(bm)**.

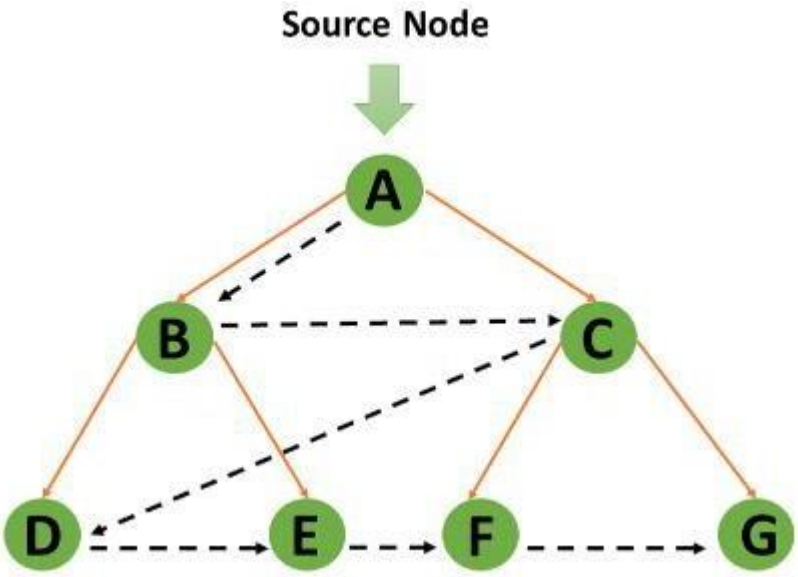
Optimal: DFS search algorithm is non-optimal, as it may generate many steps or high cost to reach the goal node.

Breadth First Search (BFS):

BFS is a fundamental graph traversal algorithm that explores all the vertices in a graph in a level-by-level manner. It starts at a specified vertex and visits all its neighbors before moving on to the next level of neighbors. BFS is commonly used in various applications, including pathfinding, connected components, and shortest path problems in graphs.

According to the BFS, you must traverse the graph in a breadth wise direction:

- To begin, move horizontally and visit all the current layer's nodes.
- Continue to the next layer.



Breadth-First Search uses a queue data structure to store the node and mark it as "visited" until it marks all the neighboring vertices directly related to it. The queue operates on the First in First out (FIFO) principle, so the node's neighbors will be viewed in the order in which it inserts them in the node, starting with the node that was inserted first.

Algorithm for BFS:

1. Initialization:

Enqueue the starting node into a queue and mark it as visited.

2. Exploration:

While the queue is not empty:

Dequeue a node from the queue and visit it (e.g., print its value).

For each unvisited neighbor of the dequeued node:

Enqueue the neighbor into the queue.

Mark the neighbor as visited.

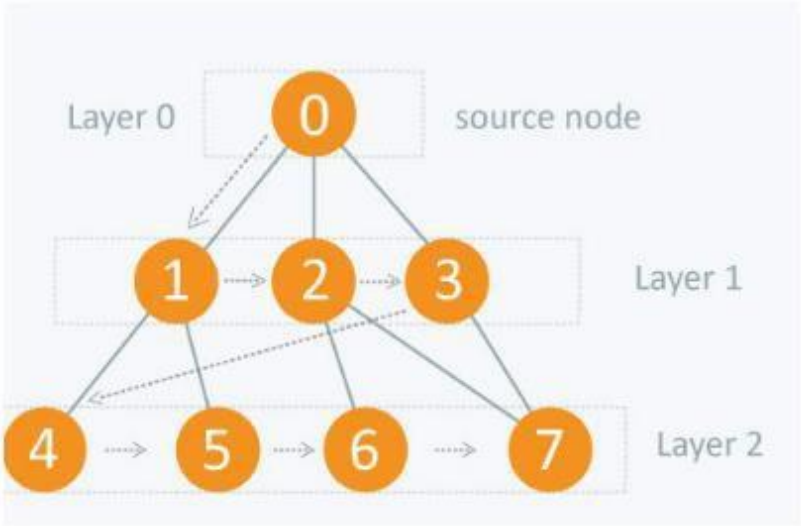
3. Termination:

Repeat step 2 until the queue is empty.

This algorithm ensures that all nodes in the graph are visited in a breadth-first manner, starting from the initial node.

How Does BFS Work?

Starting from the root, all the nodes at a particular level are visited first, and then the nodes of the next level are traversed until all nodes are visited. A queue is used to keep track of adjacent unvisited nodes at each level.



The above tree diagram contains three layers, which are numbered from 0 to 2.

- We are allowed to use any node as our source node as per the law. However, in this case, we can use 0 as our source node.
- Then we explore breadth wise and find the nodes which are adjacently connected to our source node.
- Then we must come down to layer two and find the relative nodes that are adjacent to the layer 1 nodes.
- If you select the alternative source node, this order will change. And that's how the straightforward BFS algorithm operates.

Example Illustration

Let’s understand the working of BFS with an example. Consider the following directed graph:

```
  A
 / \
B   C
|   |
D   G
|   |
E   F
```

- Initially, the queue and visited arrays are empty.
- Push node A into the queue and mark it visited.
- Dequeue A, visit it, and enqueue its neighbors B and C.
- Dequeue B, visit it, and enqueue its children D and E.
- Dequeue C, visit it, and enqueue its children D and G.
- Continue this process until all nodes are visited.

Advantages of BFS:

1. **Completeness:** If there is a solution, BFS is guaranteed to find it.
2. **Optimality:** Finds the shortest path (minimal number of edges) to the goal.
3. **Simplicity:** One of the simplest search strategies.

Disadvantages of BFS:

1. **Memory Requirement:** BFS consumes large memory space due to the queue.
2. **Time Complexity:** In the worst case, BFS explores all vertices and edges, resulting in a time complexity of O(V + E).

Remember, BFS is a powerful tool for exploring graphs, but its memory usage can be a concern in large graphs.

Time Complexity of BFS:

b -> b is a node at every state

d -> depth of shallowest node (goal node)

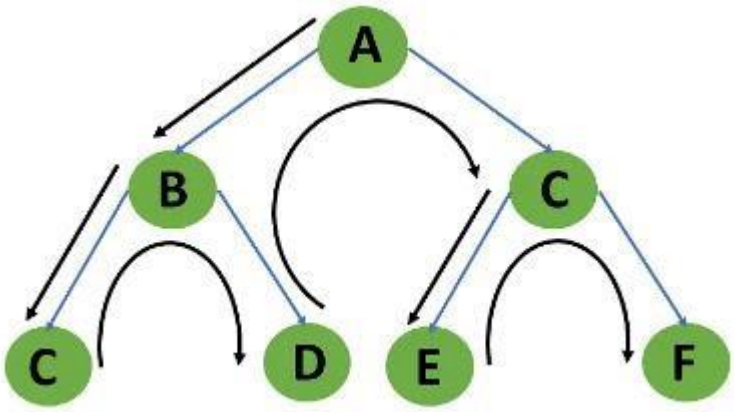
$T(b)= 1+b^2+b^3+\dots\dots\dots\dots+b^d$

$=O(b^d)$

$s(b) = O(b^d)$

Depth First Search (DFS)

Depth First Search (DFS) is a fundamental algorithm used in graph theory to traverse or search through the nodes of a graph. DFS is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at a selected node (root if it's a tree) and explores as far as possible along each branch before backtracking. When a dead-end occurs in any iteration, the Depth First Search (DFS) method traverses a network in a deathward motion and uses a stack data structure to remember to acquire the next vertex to start a search. To visit the next node, pop the top node from the stack and push all its nearby nodes into a stack.



Topological sorting, scheduling problems, graph cycle detection, and solving puzzles with just one solution, such as a maze or a sudoku puzzle, all employ depth-first search algorithms. Other applications include network analysis, such as determining if a graph is bipartite.

Algorithm:

1. Create a stack with the total number of vertices in the graph as the size.

2. Choose any vertex as the traversal's beginning point. Start at the root node
3. Mark the node as visited and push it onto the stack.
4. While the stack is not empty:
   - Pop a node from the stack.
   - If the node has unvisited adjacent nodes, mark the adjacent node as visited and push it onto the stack.
   - If the node has no unvisited adjacent nodes, backtrack.

Example Graph:

Imagine we have the following graph:



DFS Algorithm Steps:

1. Start at the root node (1).
2. Visit node 1 and push it onto the stack.
3. Go to an unvisited adjacent node (2), visit it, and push it onto the stack.
4. Continue to node 4, then to node 6, visiting each and pushing them onto the stack.
5. Since node 6 has no unvisited adjacent nodes, backtrack to node 4.
6. Backtrack to node 2, and since it has no unvisited adjacent nodes, backtrack to node 1.

7. Visit node 3 from node 1, then node 5, and finally node 7, visiting each one.

DFS Visiting Order:

The nodes are visited in the following order: 1, 2, 4, 6, 3, 5, 7.

This is a typical depth-first traversal, where we go as deep as possible along each branch before backtracking to explore new branches. The DFS algorithm is particularly useful in scenarios where we want to explore all possible paths or configurations, such as in puzzle solving, game state exploration, and more.

Advantages:

1. **Memory Efficiency:** DFS requires less memory as it stores only the nodes on the current path and their siblings3.
2. **Solution Depth:** DFS can find a solution without exploring much of the search space if the solution is deep and not far from the root node4.

Disadvantages:

1. **Non-Optimal Solution:** DFS may not find the shortest path to a solution5.
2. **Infinite Loops:** It can get stuck in infinite loops if the graph has cycles and precautions are not taken5.

Time Complexity: If you can access each node in **O(1)** time, then with branching factor of b and max depth of m, the total number of nodes in this tree would be worst case **= $1 + b + b^2 + \ldots + b^{m-1}$**. Using the formula for summing a geometric sequence (or even solving it ourselves) tells that this sums to **= $(b^m - 1)/(b - 1)$,** resulting in total time to visit each node proportional to b^m. Hence the complexity **= $O(b^m)$.**

Space Complexity: The length of longest path = m. For each node, you must store its siblings so that when you have visited all the children, and you come back to a parent node, you can know which sibling to explore

next. For m nodes down the path, you will have to store b nodes extra for each of the m nodes. That's how you get an **$O(b^m)$** space complexity. DFS is a powerful algorithm that forms the basis of many other complex graph algorithms and is widely used in applications such as solving puzzles, scheduling problems, and network analysis.

Applications of DFS:

Depth First Search (DFS) is a versatile algorithm with a wide range of applications in computer science and beyond. Here are some notable applications:

1. **Cycle Detection:** DFS can be used to detect cycles in a graph, which is crucial in many applications, such as deadlock detection in operating systems.
2. **Path Finding:** DFS can find a path between two given vertices, which is useful in routing and navigation systems.
3. **Topological Sorting:** This is used for scheduling jobs from the given dependencies among jobs, such as instruction scheduling, logic synthesis, and resolving symbol dependencies in linkers.
4. **Testing Bipartiteness:** DFS can test whether a graph is bipartite, which is useful in modeling problems that require partitioning, such as scheduling and assigning tasks.
5. **Finding Strongly Connected Components:** In directed graphs, DFS helps identify strongly connected components, which can be applied in understanding the structure of networks.
6. **Solving Puzzles:** DFS is suitable for puzzles that have only one solution, like mazes, as it explores exhaustively until a solution is found.
7. **Web Crawlers:** DFS can be used to navigate the web systematically for web indexing by search engines.
8. **Maze Generation:** DFS can generate random mazes, which is used in game development and recreational mathematics.

9. **Model Checking:** It’s used in verifying that a model of a system meets a certain set of properties.
10. **Backtracking Algorithms:** DFS is the foundation of backtracking algorithms, which are used in constraint satisfaction problems like solving Sudoku.

These applications leverage the exhaustive searching capability of DFS, making it a fundamental tool in algorithms and data structure manipulation.

Hill Climbing Search

- Hill climbing algorithm is a local search algorithm which continuously moves in the direction of increasing elevation/value to find the peak of the mountain or best solution to the problem. It terminates when it reaches a peak where no neighbor has a higher value.
- Hill climbing algorithm is a technique which is used for optimizing mathematical problems. One of the widely discussed examples of Hill climbing algorithm is Traveling-salesman Problem in which we need to minimize the distance traveled by the salesman.
- It is also called greedy local search as it only looks to its good immediate neighbor state and not beyond that.
- A node of hill climbing algorithm has two components which are state and value.
- Hill Climbing is mostly used when a good heuristic is available.
- In this algorithm, we don't need to maintain and handle the search tree or graph as it only keeps a single current state.

Features of Hill Climbing:

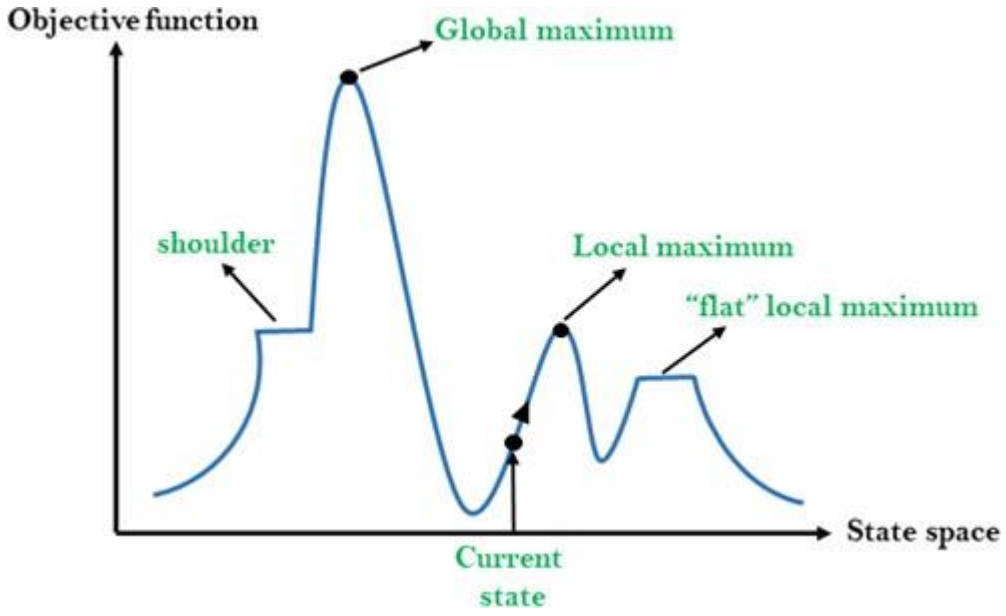
The following are some main features of Hill Climbing Algorithm:

- **Generate and Test variant:** Hill Climbing is the variant of Generate and Test method. The Generate and Test method produce feedback which helps to decide which direction to move in the search space.
- **Greedy approach:** Hill-climbing algorithm search moves in the direction which will give the best outcome, which optimizes the cost.
- **No backtracking:** It does not backtrack the search space, as it does not remember the previous states. i.e., the algorithm cannot go back to its old path.

'

State-space Diagram for Hill Climbing:

- The state-space landscape / diagram is a graphical representation of the hill-climbing algorithm which shows a graph between various states of algorithm and Objective function/Cost.
- On Y-axis we have taken the function which can be an objective function or cost function, and state-space on the x-axis. If the function on Y-axis is cost then, the goal of search is to find the global minimum and local minimum. If the function of Y-axis is Objective function, then the goal of the search is to find the global maximum and local maximum.



Different regions in the state space landscape:

Local Maximum: Local maximum is a state which is better than its neighbor states, but there is also another state which is higher than it.

Global Maximum: Global maximum represents the highest peak in the entire state space diagram, that is the best possible state of state space landscape. It has the highest value of objective function.

Current state: It is a state in a landscape diagram where an agent is currently present during the search.

Flat local maximum/plateau: It is a flat space in the landscape where all the neighbor states of current states have the same cost function.

Shoulder: It is a plateau region which has an uphill edge.

Types of Hills Climbing Algorithm:

- Simple hill Climbing:
- Steepest-Ascent hill-climbing:
- Stochastic hill Climbing:

1. Simple Hill Climbing:

Simple hill climbing is the simplest way to implement a hill climbing algorithm. **It only evaluates the neighbor node state at a time and selects the first one which optimizes current cost and sets it as a current state**. It only checks its successor state, and if it finds better than the current state, then move else be in the same state.

This algorithm has the following features:

- Less time consuming
- Less optimal solution and the solution is not guaranteed

2. Steepest-Ascent hill climbing:

- The steepest Ascent algorithm is a variation of simple hill climbing algorithms. This algorithm examines all the neighboring nodes of the current state and selects one neighbor node which is closest to the goal state. This algorithm consumes more time as it searches for multiple neighbors.

3. Stochastic hill climbing:

- Stochastic hill climbing does not examine all its neighbors before moving. Rather, this search algorithm selects one neighbor node at random and decides whether to choose it as a current state or examine another state.

Algorithm for Simple Hill Climbing:

- **Step 1:** Evaluate the initial state, if it is goal state then return success and stop.
- **Step 2:** Loop Until a solution is found or there is no new operator left to apply.
- **Step 3:** Select and apply an operator to the current state. Then we get the new state.
- **Step 4:** Check new state:
    a. If it is a goal state, then return success and quit.
    b. Else if it is better than the current state then assigns new state as a current state.
    c. Else if not better than the current state, then return to step2.
    d. Until the process of getting the goal state.
- **Step 5:** Exit.

Or

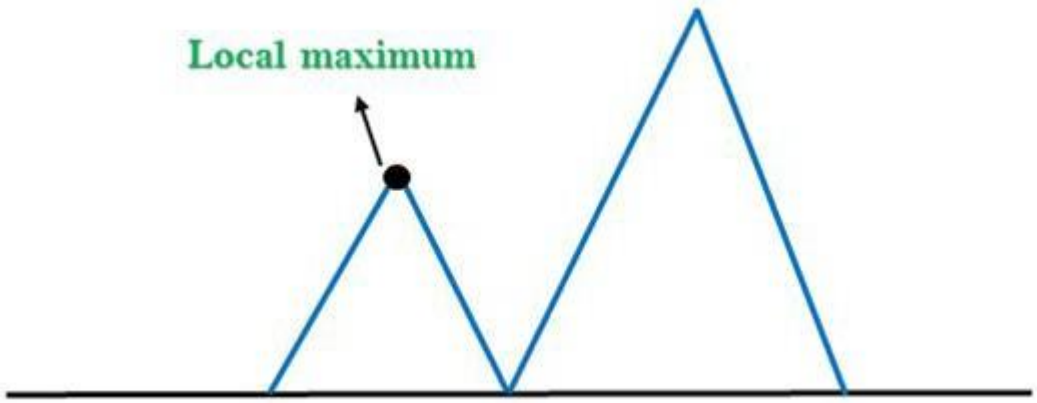
Function HILL-CLIMBING (problem) returns to a state that is a local maximum.

```
Current <- problem.INITIAL
  While true do
      Neighbor<- a highest -valued successor state of current
      If VALUE(neighbor)<= VALUE(current) then return current
      Current<-neighbor
```

Problems in Hill Climbing Algorithm:

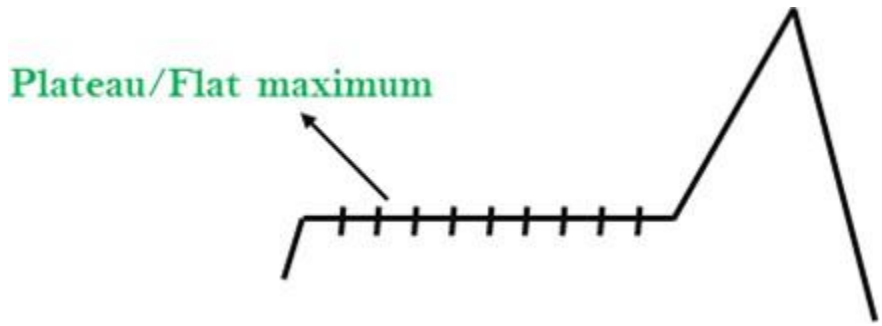
1. Local Maximum: A local maximum is a peak state in the landscape, which is better than each of its neighboring states, but there is another state also present which is higher than the local maximum.

Solution: Backtracking technique can be a solution of the local maximum in state space landscape. Create a list of the promising path so that the algorithm can backtrack the search space and explore other paths as well.



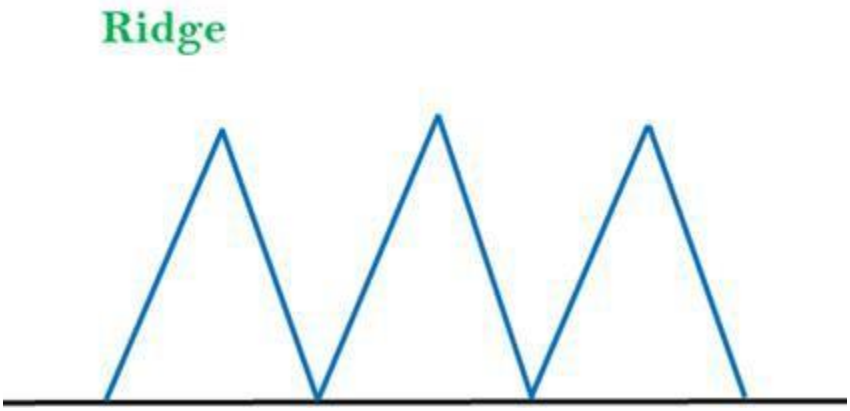
2. Plateau: A plateau is the flat area of the search space in which all the neighbor states of the current state contain the same value, because this algorithm does not find any best direction to move. A hill-climbing search might be lost in the plateau area.

Solution: The solution for the plateau is to take big steps or very little steps while searching, to solve the problem. Randomly select a state which is far away from the current state so it is possible that the algorithm could find a non-plateau region.



3. Ridges: A ridge is a special form of the local maximum. It has an area which is higher than its surrounding areas, but itself has a slope, and cannot be reached in a single move.

Solution: With the use of bidirectional search, or by moving in different directions, we can improve this problem.



Simulated Annealing Search

- The simulated annealing algorithm is based upon physical annealing in real life.
- Physical annealing is the process of heating up a material until it reaches an annealing temperature and then.
- It will be cooled down slowly to change the material to a desired structure.
- When the material is hot, the molecular structure is weaker and is more susceptible to change.
- When the material cools down, the molecular structure is harder and less susceptible to change.
- Thermal Dynamics Equation calculates the probability that the Energy Magnitude will increase.

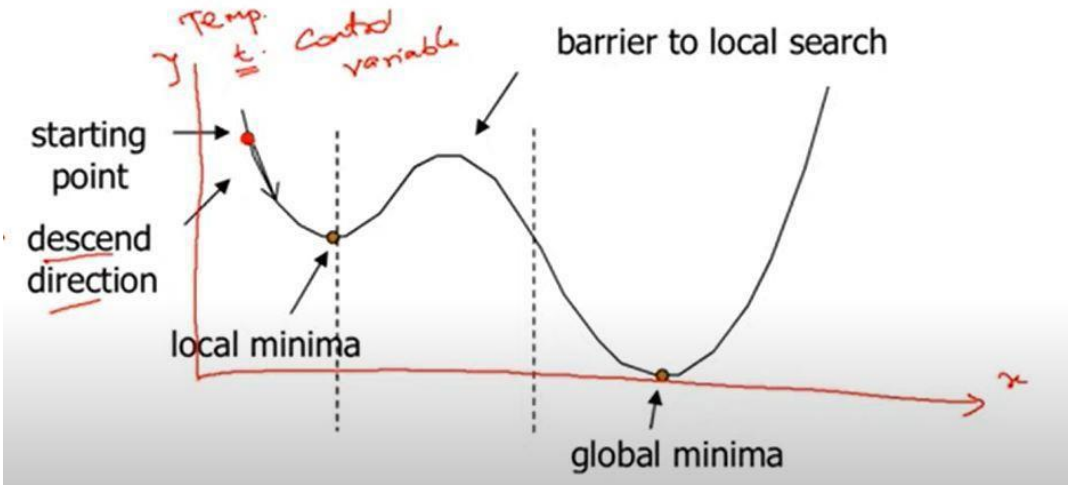
$$P(\Delta E) = e^{-\frac{\Delta E}{k*t}}$$

- Where ΔE - Energy Magnitude
- t - temperature
- k - Boltzmann constant.

Simulated Annealing:

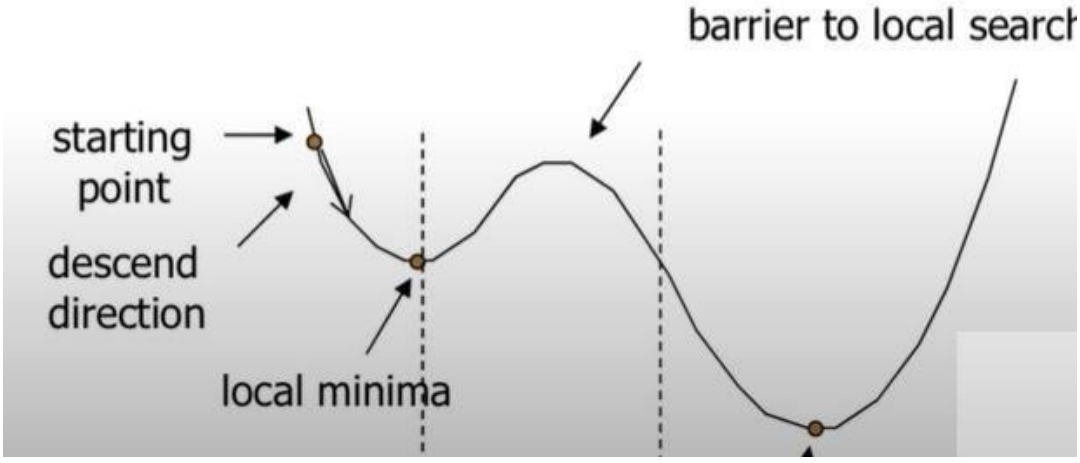
- Simulated Annealing is a stochastic global search optimization algorithm, and it is a modified version of stochastic hill climbing.
- the algorithm appropriate for nonlinear objective functions where other local search algorithms do not operate well.
- This simulated -annealing solution is to start by shaking hard (i.e., at a high temperature) and

- then gradually reduce the intensity of the shaking (i.e., lower the temperature).
- simulated Annealing (SA) is very useful for situations where there are a lot of local minima.



simulated Annealing – Example ping-pong ball:

- imagine the task of getting a ping-pong ball into the deepest crevice in a very bumpy surface.
- If we just let the ball roll, it will come to rest at a local minimum.
- The trick is to shake just hard enough, to bounce the ball out of local minima then the ball will reach the global minimum.



- simulated annealing improves this strategy through the introduction of two tricks.
- This algorithm picks a random move instead of picking the best move.
- If the move improves the result, then it accepts this random move, otherwise it accepts the move with some probability less than 1.
- Based on annealing, the algorithm uses the equation

- Based on annealing, the algorithm uses the equation

$$e^{-\Delta D/T} > R(0, 1),$$

- Where ΔD is Change of distance,
- T is synthetic temperature (control Variable) and
- R(0,1) is a random number in the interval [0,1].

Simulated Annealing Algorithm

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state
  current ← *problem*.INITIAL
  for $t = 1$ **to** ∞ **do**
    $T \leftarrow schedule(t)$
    if $T = 0$ **then return** *current*
    next ← a randomly selected successor of *current*
    $\Delta E \leftarrow$ VALUE(*current*) – VALUE(*next*)
    if $\Delta E > 0$ **then** *current* ← *next*
    else *current* ← *next* only with probability $e^{-\Delta E/T}$

- Simulated annealing was used to solve VLSI layout problems
- It has been applied widely to factory scheduling and
- Other large-scale optimization tasks.

Local Search in Continuous Space

Local search algorithms play a crucial role in optimization problems, especially when finding an exact solution is computationally expensive or impossible. In continuous spaces, where the solution space is not discrete but rather a continuum, local search techniques are particularly valuable.

Local search is an optimization technique that iteratively explores the solution space by making small adjustments to an initial solution.

Unlike global search methods that explore multiple paths simultaneously, local search focuses on a single current node and its neighbors.

The goal is to find an optimal solution by iteratively improving the current solution.

A local search is first conducted in the continuous space until a local optimum is reached. It then switches to a discrete space that represents a discretization of the continuous model to find an improved solution from there.

The process continues, switching between the two problem formulations until no further improvement can be found in either.

To perform local search in continuous state space we need techniques from calculus.

The main technique to find a minimum is called gradient descent.

Gradient descent:

A gradient measures how much the output of a function changes if you change the inputs a little bit.

In machine learning, a gradient is a derivative of a function that has more than one input variable. known as the slope of a function in mathematical terms, the gradient simply measures the change in all weights about the change in error.

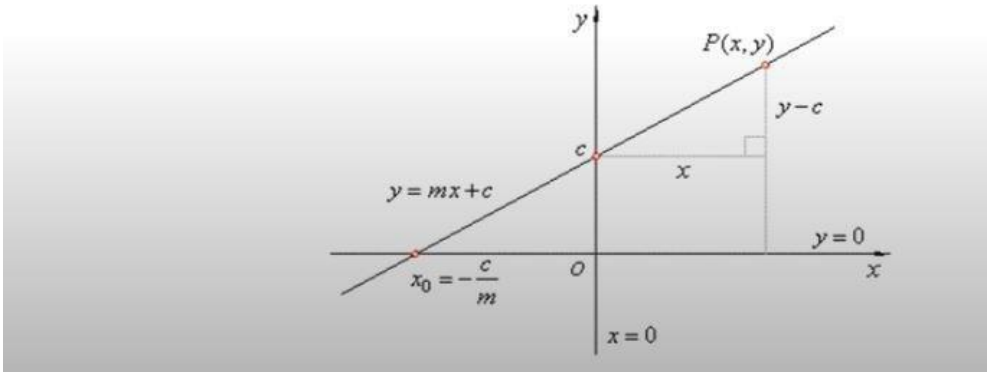
Gradient descent is an iterative optimization algorithm to find the minimum of a function.

Gradient descent is an optimization algorithm for finding a local minimum of a differentiable function. gradient descent is simply used in machine learning to find the values of function's parameters (coefficients) that minimize a cost function as far as possible.

Linear Regression:

In statistics, linear regression is a linear approach to modelling the relationship between a dependent variable and one or more independent variables.

Y = m X + c



Loss function:

The loss is the error in our predicted value of m and c. our goal is to minimize this error to obtain the most accurate value of m and c.

Mean Squared Error function to calculate the loss:

Step 1: find the difference between the actual y and predicted y value(y=mx+c), for a given x.

Step 2: square this difference.

Step 3: find the meaning of the squares for every value in X.

- Mean Squared Error Equation

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - \bar{y}_i)^2$$

- Substituting the value of $\bar{y}_i$

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))^2$$

Gradient Descent Algorithm:

Applying gradient descent to m and c and approach it step by step:

Step 1: initially let m=0 and c=0. Let L be our learning rate. This controls how much the value of m changes with each step. L could be a small value like 0.0001 for good accuracy.

Step 2: calculate the partial derivative of the loss function with respect to m, and plug in the current values of x,y , m and c in it to obtain the derivative value D.

$$D_m = \frac{1}{n}\sum_{i=0}^{n} 2(y_i - (mx_i + c))(-x_i)$$

$$D_m = \frac{-2}{n}\sum_{i=0}^{n} x_i(y_i - \bar{y}_i)$$

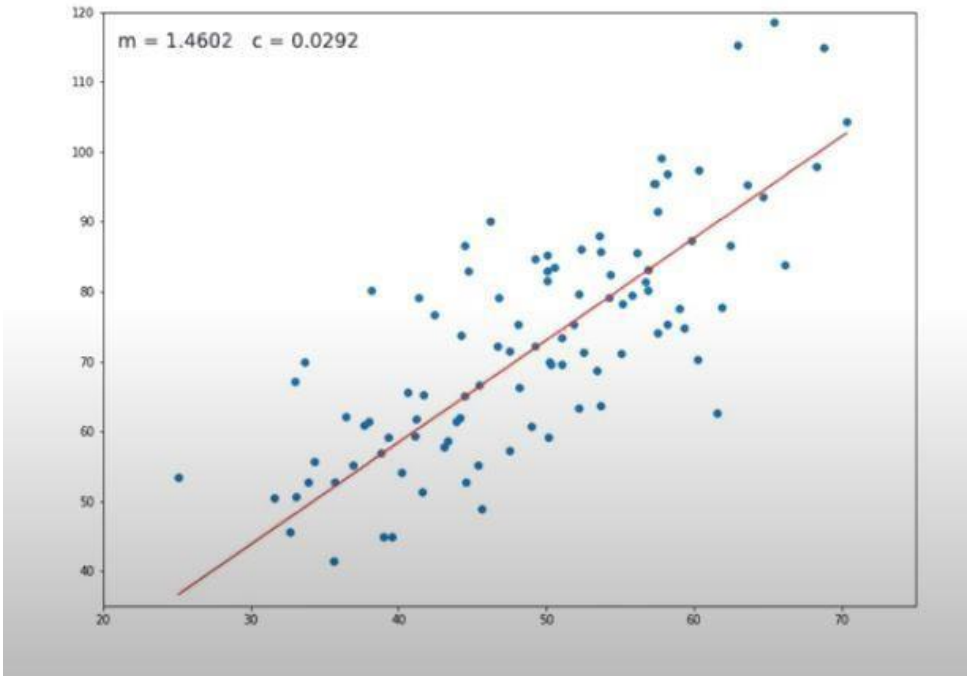
Step 3: Now we update the current value of m and c using the following equation:

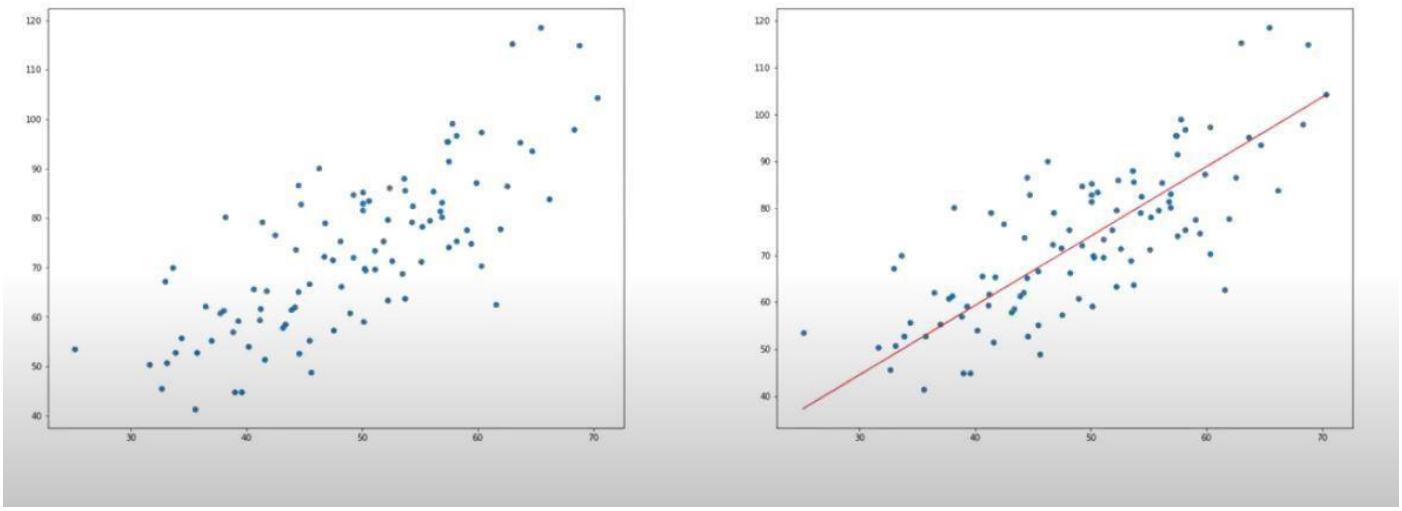
$$m = m - L \times D_m$$

$$c = c - L \times D_c$$

Step 4: we repeat this process until our loss function is a very small value or ideally 0(which means 0 error or 100% accuracy). The value of m and c that we are left with now will be the optimum values.

The value of m and c are updated at each iteration to get the optimal solution.





Characteristics of Local Search in Continuous Spaces:

Iterative Process: Local search algorithms repeatedly modify the current solution to explore nearby solutions.

Single Current Node: Instead of branching into multiple paths, local search operates on a single node and its neighbors.

Infinite Branching Factors: Continuous spaces have infinite possibilities, making local search adaptable to various problem domains.

Local Search Algorithms for Continuous Spaces:

Hill Climbing:

Start with an initial solution and iteratively move to a neighboring solution with a higher objective value. Works well for continuous state and action spaces.

Example: Climbing a hill by always moving uphill.

Simulated Annealing:

Inspired by annealing in metallurgy, it probabilistically explores the solution space.

Allows occasional downhill moves to escape local optima.

Widely used in optimization problems.

Tabu Search, Genetic Algorithms, and other variants can also be adapted for continuous spaces.

Challenges in Local Search:

Local Maxima: Getting stuck in suboptimal solutions.

Plateaus: Flat regions where small adjustments don't lead to improvement.

Applications for Local Search in Continuous Spaces:

Engineering: Optimal design parameters, circuit layout, and resource allocation.

Economics: Portfolio optimization, pricing strategies, and supply chain management.

AI: Neural network training, hyperparameter tuning, and reinforcement learning.

Local search in continuous spaces is a powerful tool for solving complex optimization problems. Its adaptability, efficiency, and ability to handle infinite solution spaces make it indispensable in AI and beyond.

UNIT-II
Games

Adversarial Search

Adversarial search is a search, where we examine the problem which arises when we try to plan the world and other agents are planning against us.

- In previous topics, we have studied search strategies which are only associated with a single agent that aims to find the solution which is often expressed in the form of a sequence of actions.
- But there might be some situations where more than one agent is searching for the solution in the same search space, and this situation usually occurs in game playing.
- The environment with more than one agent is termed as **multi-agent environment**, in which each agent is an opponent of other agent and playing against each other. Each agent needs to consider the action of other agents and the effect of that action on their performance.
- So, Searches in which two or more players with conflicting goals are trying to explore the same search space for the solution, are called **adversarial searches**, often known as Games.
- Games are modeled as a Search problem and heuristic evaluation function, and these are the two main factors which help to model and solve games in AI.

Types of Games in AI:

	Deterministic	Chance Moves
Perfect information	Chess, Checkers, go, Othello	Backgammon, monopoly
Imperfect information	Battleships, blind, tic-tac-toe	Bridge, poker, scrabble, nuclear war

- **Perfect information:** A game with perfect information is in which agents can investigate the complete board. Agents have all the information about the game, and they can see each other's moves also. Examples are Chess, Checkers, Go, etc.

- **Imperfect information:** If in a game agent do not have all the information about the game and are not aware with what's going on, such type of games is called the game with imperfect information, such as tic-tac-toe, Battleship, blind, Bridge, etc.
- **Deterministic games:** Deterministic games are those games which follow a strict pattern and set of rules for the games, and there is no randomness associated with them. Examples are chess, Checkers, Go, tic-tac-toe, etc.
- **Non-deterministic games:** Non-deterministic are those games which have various unpredictable events and have a factor of chance or luck. This factor of chance or luck is introduced by either dice or cards. These are random, and each action response is not fixed. Such games are also called stochastic games.
- Example: Backgammon, Monopoly, Poker, etc.

Zero-Sum Game

- Zero-sum games are adversarial search which involves pure competition.
- In Zero-sum game each agent's gain or loss of utility is exactly balanced by the losses or gains of utility of another agent.
- One player of the game tries to maximize one single value, while another player tries to minimize it.
- Each move by one player in the game is called a play.
- Chess and tic-tac-toe are examples of a Zero-sum game.

Zero-sum game: Embedded thinking

The Zero-sum game involved embedded thinking in which one agent or player is trying to figure out:

- What to do.
- How to decide the move

- Needs to think about his opponent as well.
- The opponent also thinks about what to do.

Each of the players is trying to find out the response of his opponent to their actions. This requires embedded thinking or backward reasoning to solve the game problems in AI.

Formalization of the problem:

A game can be defined as a type of search in AI which can be formalized of the following elements:

- **Initial state:** It specifies how the game is set up at the start.
- **Player(s):** It specifies which player has moved in the state space.
- **Action(a):** It returns the set of legal moves in state space.
- **Result(s, a):** It is the transition model, which specifies the result of moves in the state space.
- **Terminal-Test(s):** Terminal test is true if the game is over, else it is false at any case. The state where the game ends is called terminal states.
- **Utility(s, p):** A utility function gives the final numeric value for a game that ends in terminal states s for player p. It is also called payoff function. For Chess, the outcomes are a win, loss, or draw and its payoff values are +1, 0, ½. And for tic-tac-toe, utility values are +1, -1, and 0.

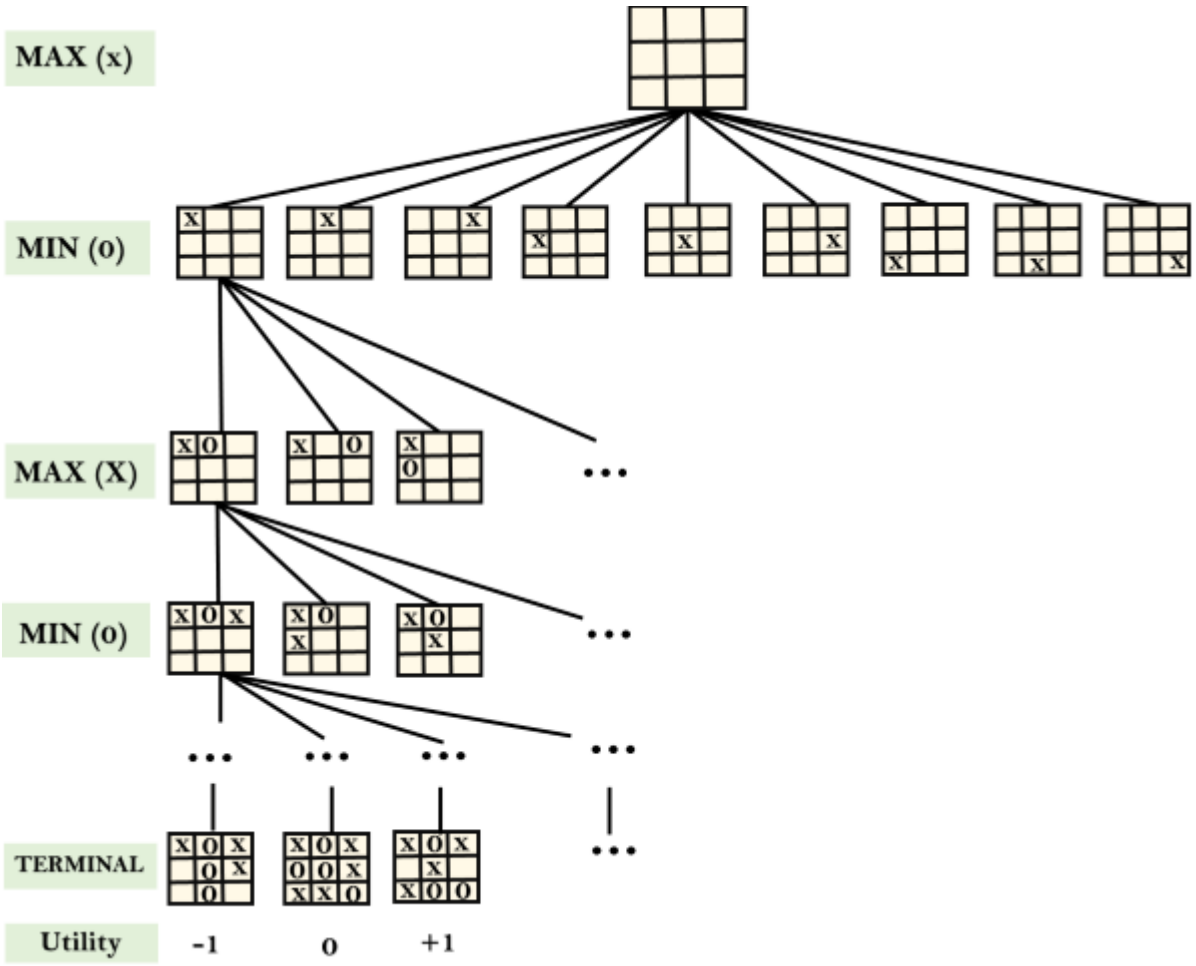
Game tree:

A game tree is a tree where nodes of the tree are the game states and Edges of the tree are the moves by players. Game tree involves initial state, action's function, and result Function.

Example: Tic-Tac-Toe game tree:

The following figure is showing part of the game-tree for tic-tac-toe game. Following are some key points of the game:

- There are two players MAX and MIN.
- Players have an alternate turn and start with MAX.
- MAX maximizes the result of the game tree.
- MIN minimizes the result.



Example Explanation:

- From the initial state, MAX has 9 possible moves as he starts first. MAX place x and MIN place o, and both players play alternatively until we reach a leaf node where one player has three in a row or all squares are filled.
- Both players will compute each node, minimax, the minimax value which is the best achievable utility against an optimal adversary.

- Suppose both the players are aware of the tic-tac-toe and playing the best play. Each player is doing his best to prevent the other one from winning. MIN is acting against Max in the game.
- So, in the game tree, we have a layer of Max, a layer of MIN, and each layer is called as **Ply**. Max place x, then MIN puts o to prevent Max from winning, and this game continues until the terminal node.
- In this either MIN wins, MAX wins, or it's a draw. This game tree is the whole search space of possibilities that MIN and MAX are playing tic-tac-toe and taking turns alternately.

Hence adversarial Search for the minimax procedure works as follows:

- It aims to find the optimal strategy for MAX to win the game.
- It follows the approach of Depth-first search.
- In the game tree, optimal leaf node could appear at any depth of the tree.
- Propagate the minimax values up to the tree until the terminal node is discovered.

In each game tree, the optimal strategy can be determined from the minimax value of each node, which can be written as MINIMAX(n). MAX prefer to move to a state of maximum value and MIN prefer to move to a state of minimum value then:

For a state S MINIMAX(s) =

$$\begin{cases} \text{UTILITY}(s) & \text{If TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{If PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{If PLAYER}(s) = \text{MIN}. \end{cases}$$

Mini-Max Algorithm in Artificial Intelligence:

```
function MINIMAX-DECISION(state) returns an action
  return arg max_{a ∈ ACTIONS(s)} MIN-VALUE(RESULT(state, a))

function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s, a)))
  return v

function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← ∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s, a)))
  return v
```

Figure 5.3 An algorithm for calculating minimax decisions. It returns the action corresponding to the best possible move, that is, the move that leads to the outcome with the best utility, under the assumption that the opponent plays to minimize utility. The functions MAX-VALUE and MIN-VALUE go through the whole game tree, all the way to the leaves, to determine the backed-up value of a state. The notation $\text{argmax}_{a \in S} f(a)$ computes the element a of set S that has the maximum value of $f(a)$.

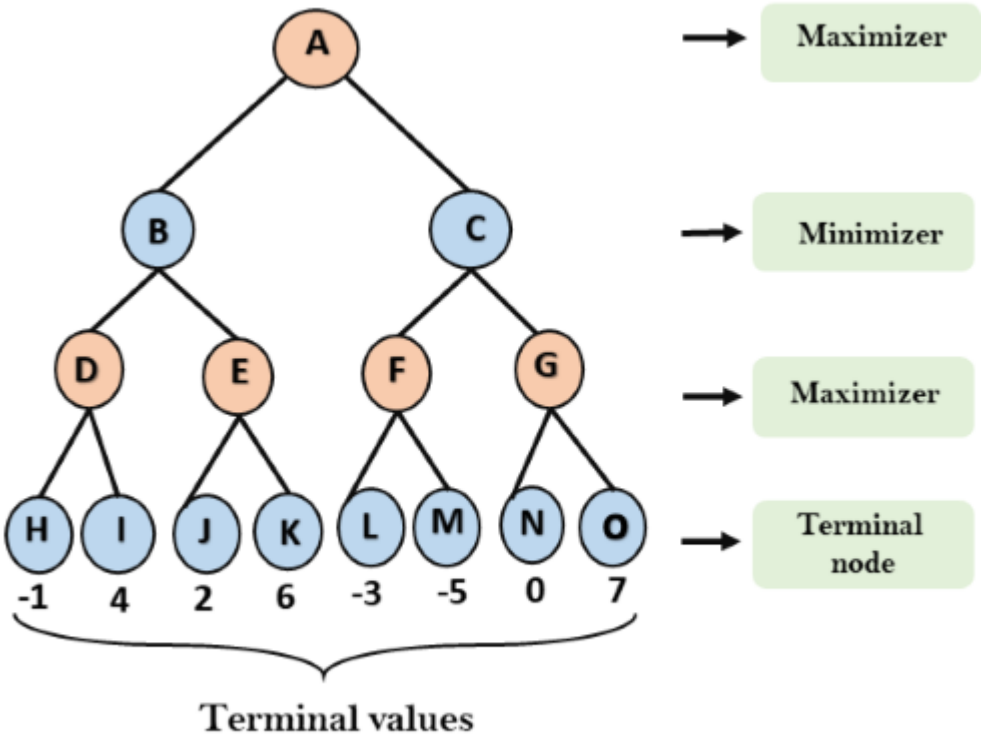
- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory. It provides an optimal move for the player assuming that the opponent is also playing optimally.
- The Mini-Max algorithm uses recursion to search through the game tree.
- The Mini-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, go, and various two-players game. This Algorithm computes the minimax decision for the current state.
- In this algorithm two players play the game; one is called MAX and the other is called MIN.
- Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.

- Both Players of the game are opponents of each other, where MAX will select the maximized value and MIN will select the minimized value.
- The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree.
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

Working of Min-Max Algorithm:

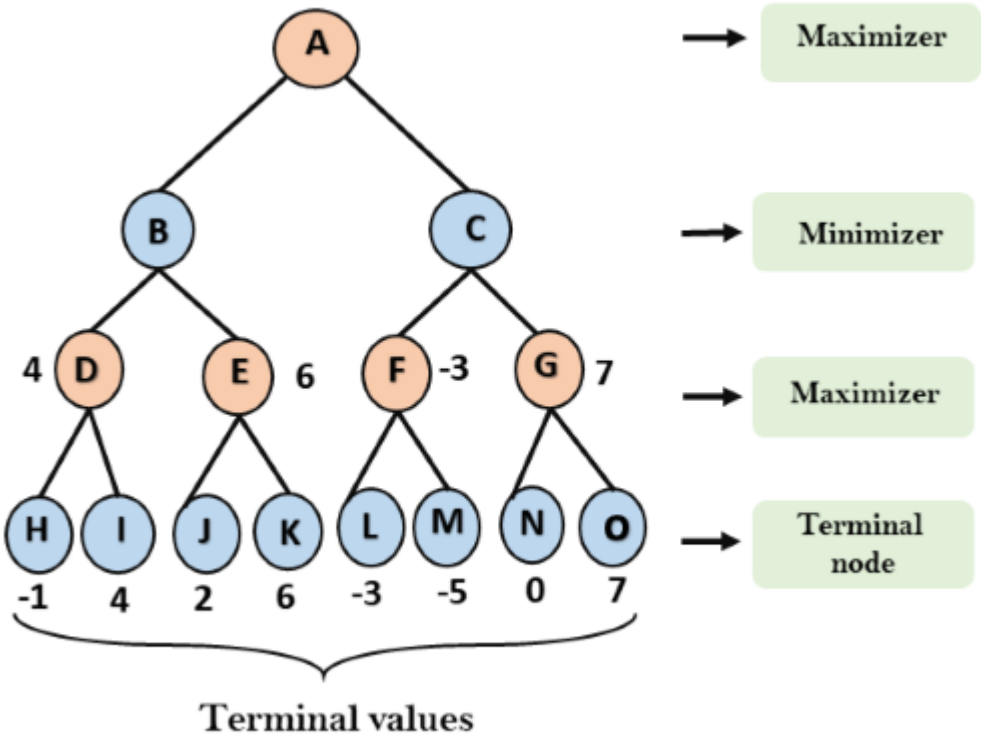
- The working of the minimax algorithm can be easily described using an example. Below we have taken an example of a game tree which represents the two-player game.
- In this example, there are two players one is called Maximizer, and the other is called Minimizer.
- Maximizer will try to get the Maximum possible score, and Minimizer will try to get the minimum possible score.
- This algorithm applies DFS, so in this game-tree, we must go all the way through the leaves to reach the terminal nodes.
- At the terminal node, the terminal values are given so we will compare those values and backtrack the tree until the initial state occurs. Following are the main steps involved in solving the two-player game tree:

Step-1: In the first step, the algorithm generates the entire game-tree and applies the utility function to get the utility values for the terminal states. In the below tree diagram, let's take A is the initial state of the tree. Suppose maximizer takes first turn which has worst-case initial value =-infinity, and minimizer will take next turn which has worst-case initial value = +infinity.



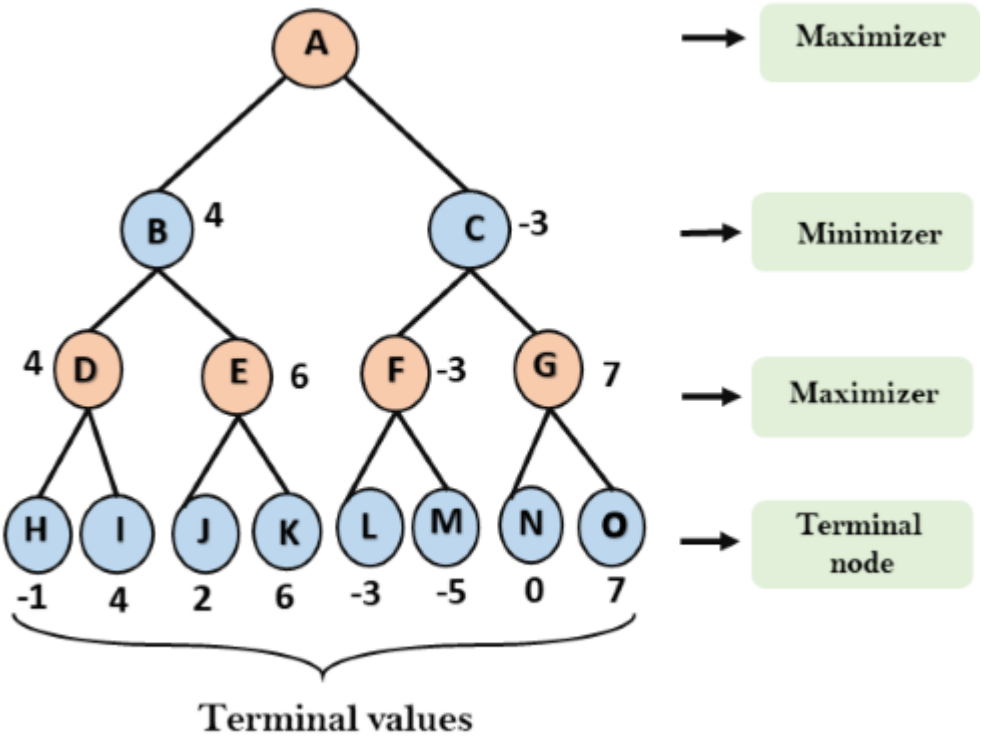
Step 2: Now, first we find the utilities value for the Maximizer, its initial value is -∞, so we will compare each value in terminal state with initial value of Maximizer and determines the higher nodes values. It will find the maximum among all.

- For node D max(-1,- -∞) => max(-1,4)= 4
- For Node E max(2, -∞) => max(2, 6)= 6
- For Node F max(-3, -∞) => max(-3,-5) = -3
- For node G max(0, -∞) = max(0, 7) = 7



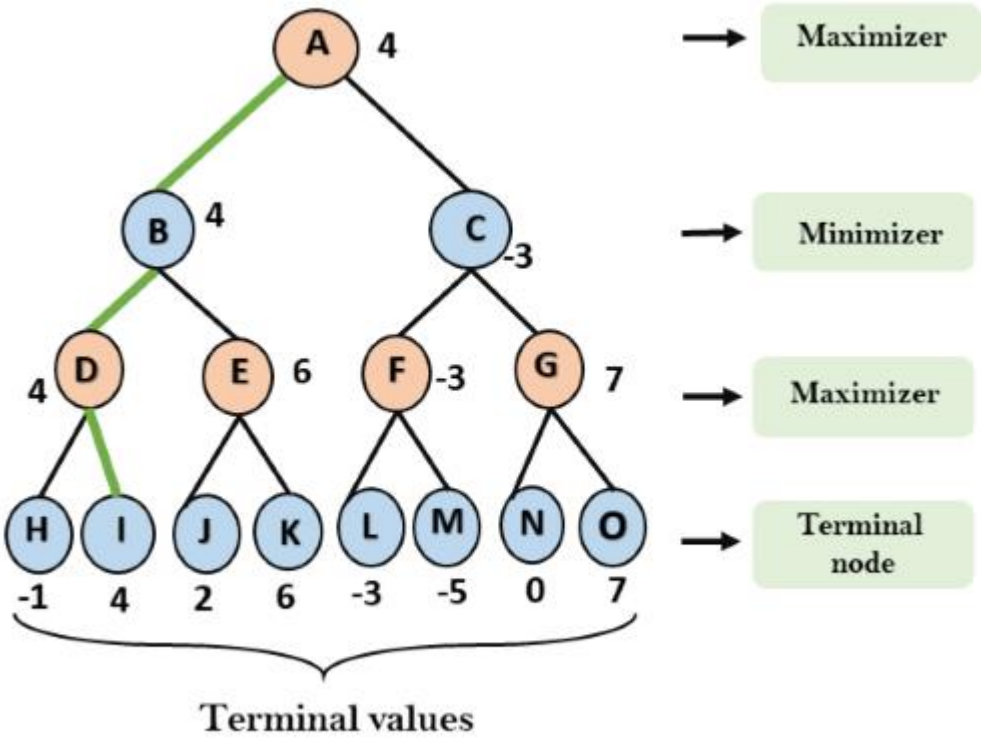
Step 3: In the next step, it's a turn for minimizer, so it will compare all nodes value with +∞ and will find the 3rd layer node values.

- For node B= min(4,6) = 4
- For node C= min (-3, 7) = -3



Step 4: Now it's a turn for Maximizer, and it will again choose the maximum of all nodes value and find the maximum value for the root node. In this game tree, there are only 4 layers, hence we reach immediately to the root node, but in real games, there will be more than 4 layers.

- For node A max(4, -3)= 4



That was the complete workflow of the minimax two player game.

Properties of Mini-Max algorithm:

- **Complete-** Min-Max algorithm is Complete. It will find a solution (if exists), in the finite search tree.
- **Optimal-** Min-Max algorithm is optimal if both opponents are playing optimally.
- **Time complexity-** As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is **$O(b^m)$**, where b is branching factor of the game-tree, and m is the maximum depth of the tree.
- **Space Complexity-** Space complexity of Mini-max algorithm is also like DFS which is **O(bm)**.

Limitation of the minimax Algorithm:

The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess, go, etc. This type of game has a huge branching factor, and the player has lots of choices to decide. This limitation of the minimax algorithm can be improved from **alpha-beta pruning.**

Alpha-Beta Pruning

- Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimization technique for the minimax algorithm.
- As we have seen in the minimax search algorithm that the number of game states it has to examine are exponential in depth of the tree. Since we cannot eliminate the exponent, but we can cut it to half. Hence there is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called **pruning**. This involves two threshold parameter Alpha and beta for future expansion, so it is called **alpha-beta pruning**. It is also called as **Alpha-Beta Algorithm**.
- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree.
- The two-parameter can be defined as:
    a. **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is **-∞**.
    b. **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is **+∞**.

The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes which are not really affecting the final decision but making algorithm slow. Hence by pruning these nodes, it makes the algorithm fast.

Condition for Alpha-beta pruning:

The main condition which required for alpha-beta pruning is:

α>=β

Key points about alpha-beta pruning:

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes.

The Alpha-Beta Search Algorithm:

```
function ALPHA-BETA-SEARCH(state) returns an action
  v ← MAX-VALUE(state, −∞, +∞)
  return the action in ACTIONS(state) with value v

function MAX-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← −∞
  for each a in ACTIONS(state) do
    v ← MAX(v, MIN-VALUE(RESULT(s,a), α, β))
    if v ≥ β then return v
    α ← MAX(α, v)
  return v

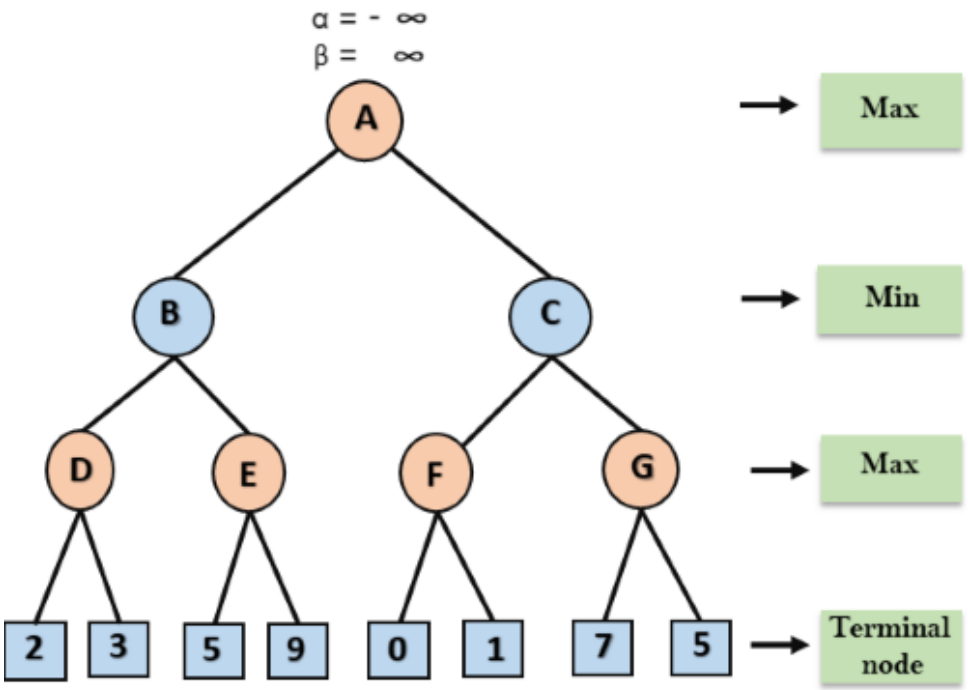
function MIN-VALUE(state, α, β) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
  v ← +∞
  for each a in ACTIONS(state) do
    v ← MIN(v, MAX-VALUE(RESULT(s,a) , α, β))
    if v ≤ α then return v
    β ← MIN(β, v)
  return v
```

Figure 5.7 The alpha–beta search algorithm. Notice that these routines are the same as the MINIMAX functions in Figure 5.3, except for the two lines in each of MIN-VALUE and MAX-VALUE that maintain α and β (and the bookkeeping to pass these parameters along).

Working of Alpha-Beta Pruning:

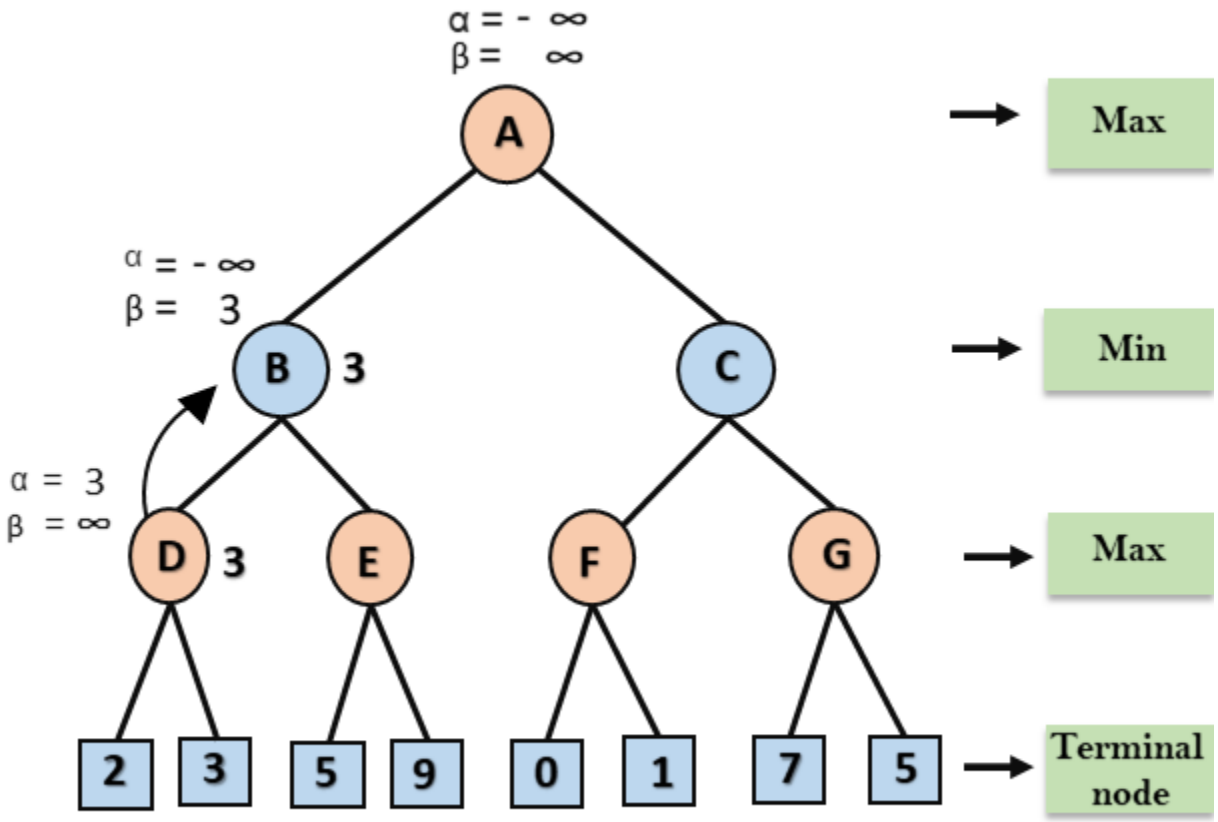
Let's take an example of two-player search tree to understand the working of Alpha-beta pruning

Step 1: At the first step the, Max player will start first move from node A where α= -∞ and β= +∞, these value of alpha and beta passed down to node B where again α= -∞ and β= +∞, and Node B passes the same value to its child D.



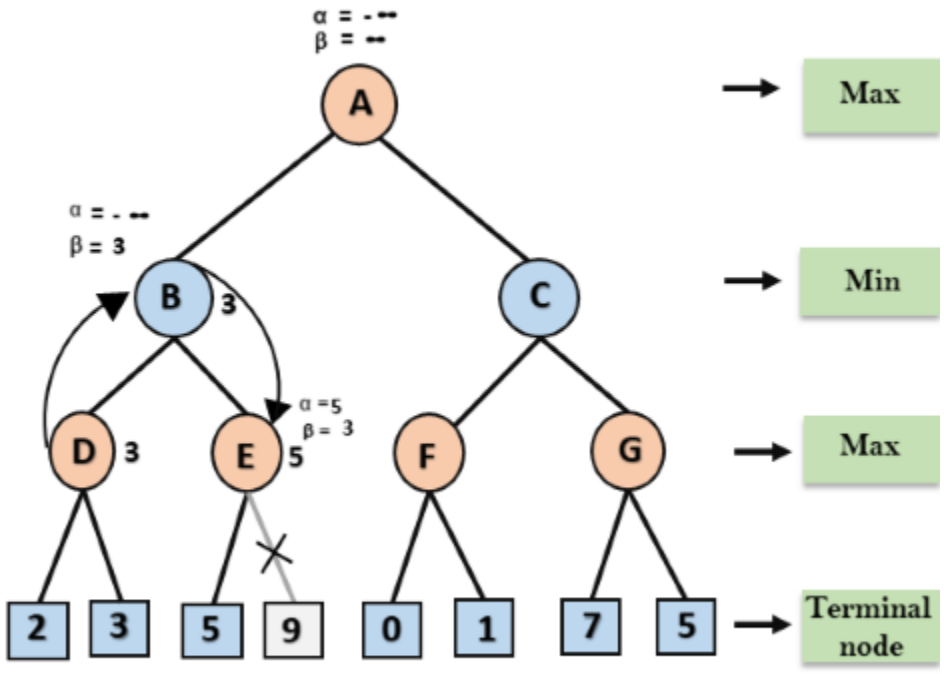
Step 2: At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the max (2, 3) = 3 will be the value of α at node D and node value will also 3.

Step 3: Now algorithm backtrack to node B, where the value of β will change as this is a turn of Min, Now β= +∞, will compare with the available subsequent nodes value, i.e. min (∞, 3) = 3, hence at node B now α= -∞, and β= 3.



In the next step, algorithm traverse the next successor of Node B which is node E, and the values of α= -∞, and β= 3 will also be passed.

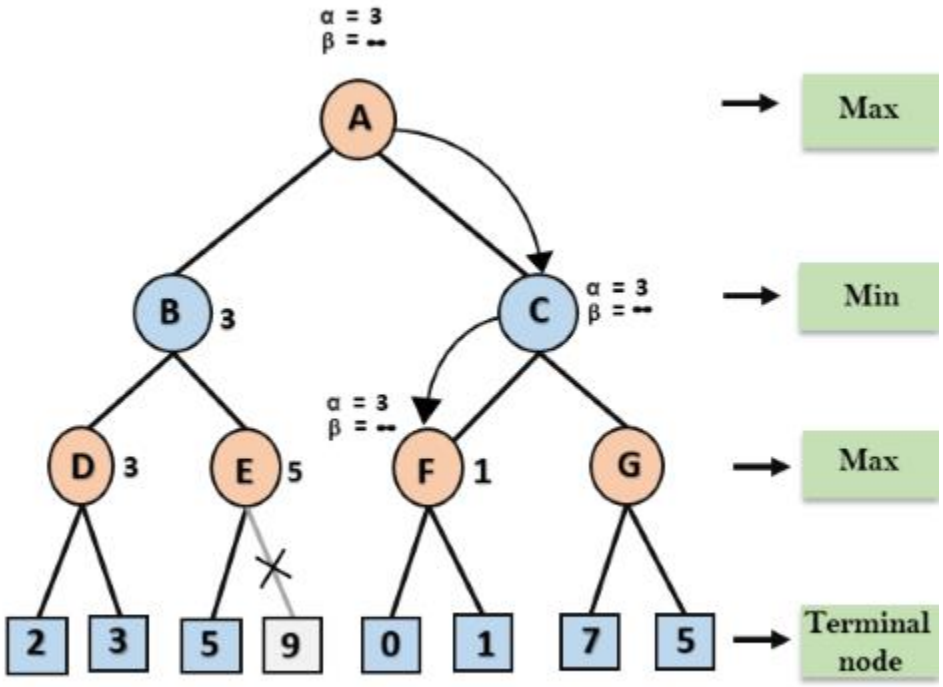
Step 4: At node E, Max will take its turn, and the value of alpha will change. The current value of alpha will be compared with 5, so max (-∞, 5) = 5, hence at node E α= 5 and β= 3, where α>=β, so the right successor of E will be pruned, and algorithm will not traverse it, and the value at node E will be 5.



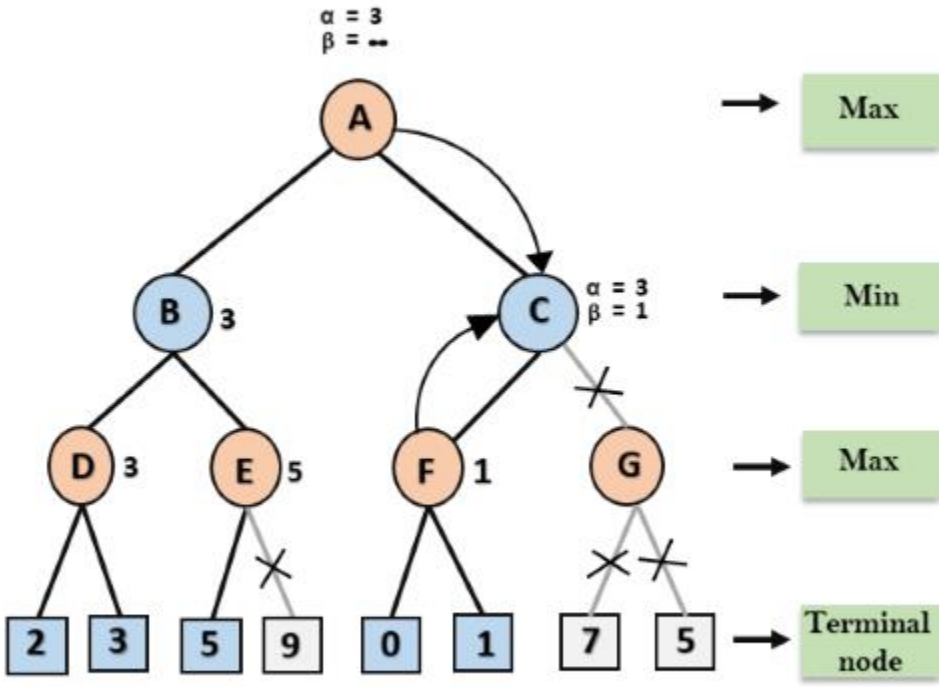
Step 5: At next step, algorithm again backtrack the tree, from node B to node A. At node A, the value of alpha will be changed the maximum available value is 3 as max (-∞, 3)= 3, and β= +∞, these two values now passes to right successor of A which is Node C.

At node C, α=3 and β= +∞, and the same values will be passed on to node F.

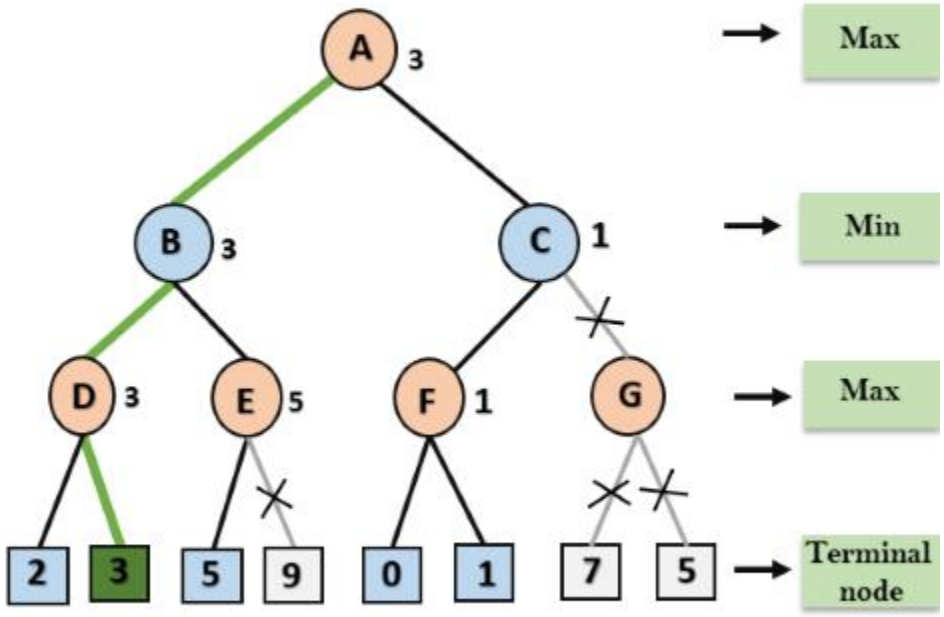
Step 6: At node F, again the value of α will be compared with left child which is 0, and max(3,0)= 3, and then compared with right child which is 1, and max(3,1)= 3 still α remains 3, but the node value of F will become 1.



Step 7: Node F returns the node value 1 to node C, at C α= 3 and β= +∞, here the value of beta will be changed, it will compare with 1 so min (∞, 1) = 1. Now at C, α=3 and β= 1, and again it satisfies the condition α>=β, so the next child of C which is G will be pruned, and the algorithm will not compute the entire sub-tree G.



Step 8: C now returns the value of 1 to A here the best value for A is max (3, 1) = 3. Following is the final game tree which is the showing the nodes which are computed and nodes which has never computed. Hence the optimal value for the maximizer is 3 for this example.



Move Ordering in Alpha-Beta pruning:

The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined. Move order is an important aspect of alpha-beta pruning.

It can be of two types:

- **Worst ordering:** In some cases, alpha-beta pruning algorithm does not prune any of the leaves of the tree, and works exactly as minimax algorithm. In this case, it also consumes more time because of alpha-beta factors, such a move of pruning is called worst ordering. In this case, the best move occurs on the right side of the tree. The time complexity for such an order is **$O(b^m)$.**
- **Ideal ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and best moves occur at the left side of the tree. We apply DFS

hence it first search left of the tree and go deep twice as minimax algorithm in the same amount of time. Complexity in ideal ordering is **$O(b^{m/2})$.**

Rules to find good ordering:

Following are some rules to find good ordering in alpha-beta pruning:

- Occur the best move from the shallowest node.
- Order the nodes in the tree such that the best nodes are checked first.
- Use domain knowledge while finding the best move. Ex: for Chess, try order: captures first, then threats, then forward moves, backward moves.
- We can bookkeep the states, as there is a possibility that states may repeat.

Constraint Satisfaction Problems

Constraint satisfaction problems (CSPs) are mathematical questions defined as a set of objects whose state must satisfy several constraints or limitations.

CSPs represents the objects in problem as a collection of finite constraints over variables, which is solved by constraint satisfaction methods.

Assignment and solution of CSP:

- An assignment is complete when every variable is assigned a value.
- A solution to a CSP is a complete assignment that satisfies all constraints.

Applications:

- Map coloring
- Line drawing interpretation
- Scheduling problems
- Floor planning for VLSI

Formal Definition of CSP:

A CSP consists of

- A finite set of variables x1,x2,……xn
- Non empty domain of possible values for each variable D1,D2,……..,Dn
  Where Di = {v1,v2,……,vk)
- Finite set of constraints C1,C2,C3,…….Cn
- Each constraint Ci limits the values that variables can take,
  e.g, X1≠ X2
- A state is defined as an assignment of values to some or all variables
- A consistent assignment does not violate the constraints, is called as consistent or legal assignment.

- A complete assignment is one in which every variable is maintained and a solution to CSP. i.e., a complete assignment that satisfies all the constraints.
- Some csp also required a solution that maximizes an objective function.

Example of CSP – Map Coloring problem

- The problem is to color the regions of a a given map such that no two adjacent regions have the same colors
- The regions in the map are the variables and the set of possible colors for region is the domain.



- Representation of Problem
- Variables: Vi = {WA, NT, Q, NSW, V, SA, T}
- Domains: Di = {red, green, blue}
- Constraints: adjacent regions must have different colors
- e.g., WA ≠ NT

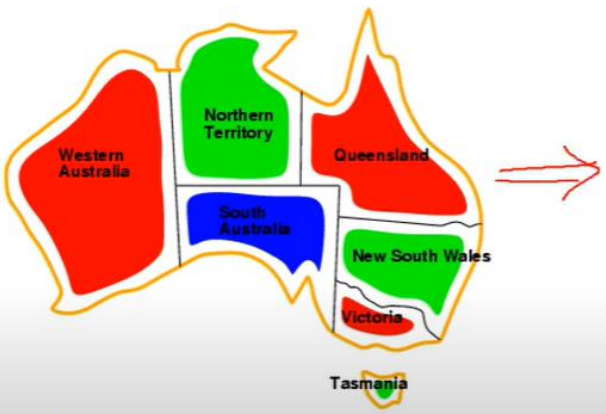
- So (WA,NT) must be in {(red, green),(red, bl}ue),(green,red),……..}

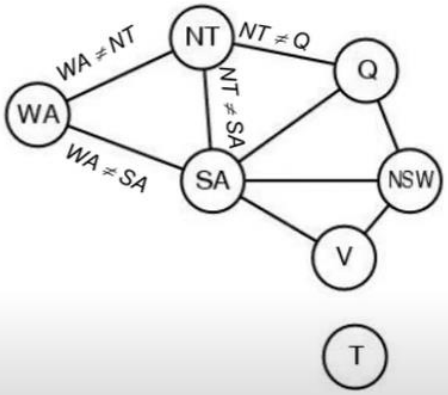
- Initial state:
  - The empty assignment {}, in which all variables are unassigned
- Successor Functions
  - A value can be assigned to any unassigned variable, provided that it does not conflict with previously assigned value
- Goal test
  - Current assignment is complete and test whether the goal reached
- Path cost
  - Compute cost for every step

- Solutions are complete and consistent assignments,
- e.g., WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green



CSP – Map Coloring Vs Graph Coloring





- Most of the times, these problems has many solution
- The topology of constraint graph can be used to identify solutions easily

- Map coloring can be sometimes represented as a graph coloring problem.

Searching for Goal State in CSP

- Every solution must be a complete assignment and therefore appears at depth 'n', if there are 'n' variables
- The search tree extends only to depth 'n'
- Depth-first search algorithms are popular for CSP
- The path by which a solution to be reached is irrelevant
- Local search methods work well for this formulation

Types of constraints in CSP

- Unary constraint (related to single variable in a problem)
- Binary constraint (relates to two variables - Graph Coloring)
- Higher-order constraint (related to many variables - Cryptrithetic puzzle)

Constraint Propagation

Artificial Intelligence (AI) encompasses a variety of methods and techniques to solve complex problems efficiently. One such technique is constraint propagation, which plays a crucial role in areas like scheduling, planning, and resource allocation.

Constraint propagation is a fundamental concept in constraint satisfaction problems (CSPs). A CSP involves variables that must be assigned values from a given domain while satisfying a set of constraints. Constraint propagation aims to simplify these problems by reducing the domains of variables, thereby making the search for solutions more efficient.

Key Concepts

Variables: Elements that need to be assigned values.

Domains: Possible values that can be assigned to the variables.

Constraints: Rules that define permissible combinations of values for the variables.

Constraint Graph:

A constraint graph visually represents CSPs. Variables are nodes, and constraints are edges connecting the nodes.

How Constraint Propagation Works

Constraint propagation works by iteratively narrowing down the domains of variables based on the constraints. This process continues until no more values can be eliminated from any domain. The primary goal is to reduce the search space and make it easier to find a solution.

Example

Consider a simple CSP with two variables, X and Y, each with domains {1, 2, 3}, and a constraint $X \neq Y$. Constraint propagation will iteratively reduce the domains as follows:

If X is assigned 1, then Y cannot be 1, so Y's domain becomes {2, 3}.

If Y is then assigned 2, X cannot be 2, so X's domain is reduced to {1, 3}.

This process continues until a stable state is reached.

Constraint Propagation Process

Constraint propagation involves iteratively applying constraints to reduce the domains of variables, thereby narrowing down the possible values and simplifying the problem. The process aims to make implicit constraints explicit, thus reducing the search space and making it easier to find a solution.

Steps in Constraint Propagation:

Initialization:

Start with all variables having their full domains.

Propagation:

Apply constraints to reduce the domains of the variables. When a domain of a variable is reduced, this new information can further reduce the domains of other connected variables.

Arc Consistency:

A common form of constraint propagation. A variable is arc-consistent if every value in its domain satisfies the binary constraints with some value in the domain of every connected variable.

Example: For variables X and Y with domains {1, 2, 3} and {2, 3} respectively, and a constraint X < Y, the domain of X can be reduced to {1, 2}.

Iteration:

The propagation continues iteratively until no further domain reductions can be made. This state is known as the fixed point.

Techniques in Constraint Propagation

Arc Consistency (AC) Algorithms:

- A variable in a CSP is **arc-consistent** if every value in its domain satisfies the variable's binary constraints.
- X_i is arc-consistent with respect to another variable X_j if for every value in the current domain D_i there is some value in the domain D_j that satisfies the binary constraint on the arc (X_i, X_j).
- A network is arc-consistent if every variable is arc-consistent with every other variable.
- Arc consistency tightens down the domains (unary constraint) using the arcs (binary constraints).

AC-3 algorithm:

```
function AC-3(csp) returns false if an inconsistency is found and true otherwise
  inputs: csp, a binary CSP with components (X, D, C)
  local variables: queue, a queue of arcs, initially all the arcs in csp

  while queue is not empty do
    (X_i, X_j) ← REMOVE-FIRST(queue)
    if REVISE(csp, X_i, X_j) then
      if size of D_i = 0 then return false
      for each X_k in X_i.NEIGHBORS - {X_j} do
        add (X_k, X_i) to queue
  return true
```

```
function REVISE(csp, X_i, X_j) returns true iff we revise the domain of X_i
  revised ← false
  for each x in D_i do
    if no value y in D_j allows (x,y) to satisfy the constraint between X_i and X_j then
      delete x from D_i
      revised ← true
  return revised
```

Figure 6.3 The arc-consistency algorithm AC-3. After applying AC-3, either every arc is arc-consistent, or some variable has an empty domain, indicating that the CSP cannot be solved. The name "AC-3" was used by the algorithm's inventor (Mackworth, 1977) because it's the third version developed in the paper.

- AC-3 maintains a queue of arcs which initially contains all the arcs in the CSP.
- AC-3 then pops off an arbitrary arc (X_i, X_j) from the queue and makes X_i arc-consistent with respect to X_j.
- If this leaves D_i unchanged, just moves on to the next arc;
- But if this revises D_i, then add to the queue all arcs (X_k, X_i) where X_k is a neighbor of X_i.
- If D_i is revised down to nothing, then the whole CSP has no consistent solution, return failure;
- Otherwise, keep checking, trying to remove values from the domains of variables until no more arcs are in the queue.
- The result is an arc-consistent CSP that have the same solutions as the original one but have smaller domains.

The complexity of AC-3:

Assume a CSP with *n* variables, each with domain size at most *d*, and with *c* binary constraints (arcs). Checking consistency of an arc can be done in $O(d^2)$ time, total worst-case time is $O(cd^3)$.

Node Consistency:

Ensures that each variable's value satisfies the unary constraints on that variable.

Path Consistency:

Extends the concept of arc consistency to consider triples of variables, ensuring that constraints are satisfied over paths in the constraint graph.

Generalized Arc Consistency (GAC):

Extends arc consistency to handle constraints involving more than two variables.

Example: Solving Sudoku with Constraint Propagation

In Sudoku, each cell is a variable, and the domain is the numbers 1 to 9. The constraints are:

Each row must contain unique numbers.

Each column must contain unique numbers.

Each 3x3 subgrid must contain unique numbers.

Constraint propagation helps by:

Initially removing numbers from domains based on the initial clues.

Iteratively refining the domains of each cell by applying the constraints of uniqueness.

Benefits of Constraint Propagation

Efficiency: Reduces the search space significantly, leading to faster solutions.

Pruning: Eliminates many impossible solutions early in the process.

Simplification: Simplifies the problem, making it easier to apply other solving techniques like backtracking.

Conclusion

Constraint propagation is a powerful technique in AI for efficiently solving CSPs. By iteratively applying constraints to reduce the possible values of variables, it narrows down the search space, making it easier to find solutions to complex problems. This technique is widely used in various applications, from scheduling and planning to solving puzzles and optimizing resources.

Backtracking search for CSPs

- Backtracking search, a form of depth-first search, is commonly used for solving CSPs. Inference can be interwoven with search.
- **Commutativity:** CSPs are all commutative. A problem is commutative if the order of application of any given set of actions has no effect on the outcome.
- **Backtracking search**: A depth-first search that chooses values for one variable at a time and backtracks when a variable has no legal values left to assign.
- Backtracking algorithm repeatedly chooses an unassigned variable, and then tries all values in the domain of that variable in turn, trying to find a solution. If an inconsistency is detected, then BACKTRACK returns failure, causing the previous call to try another value.
- There is no need to supply BACKTRACKING-SEARCH with a domain-specific initial state, action function, transition model, or goal test.
- BACKTRACKING-SARCH keeps only a single representation of a state and alters that representation rather than creating a new one.

```
function BACKTRACKING-SEARCH(csp) returns a solution, or failure
  return BACKTRACK({ }, csp)

function BACKTRACK(assignment, csp) returns a solution, or failure
  if assignment is complete then return assignment
  var ← SELECT-UNASSIGNED-VARIABLE(csp)
  for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
    if value is consistent with assignment then
      add {var = value} to assignment
      inferences ← INFERENCE(csp, var, value)
      if inferences ≠ failure then
        add inferences to assignment
        result ← BACKTRACK(assignment, csp)
        if result ≠ failure then
          return result
    remove {var = value} and inferences from assignment
  return failure
```

Figure 6.5 A simple backtracking algorithm for constraint satisfaction problems. The algorithm is modeled on the recursive depth-first search of Chapter 3. By varying the functions SELECT-UNASSIGNED-VARIABLE and ORDER-DOMAIN-VALUES, we can implement the general-purpose heuristics discussed in the text. The function INFERENCE can optionally be used to impose arc-, path-, or k-consistency, as desired. If a value choice leads to failure (noticed either by INFERENCE or by BACKTRACK), then value assignments (including those made by INFERENCE) are removed from the current assignment and a new value is tried.

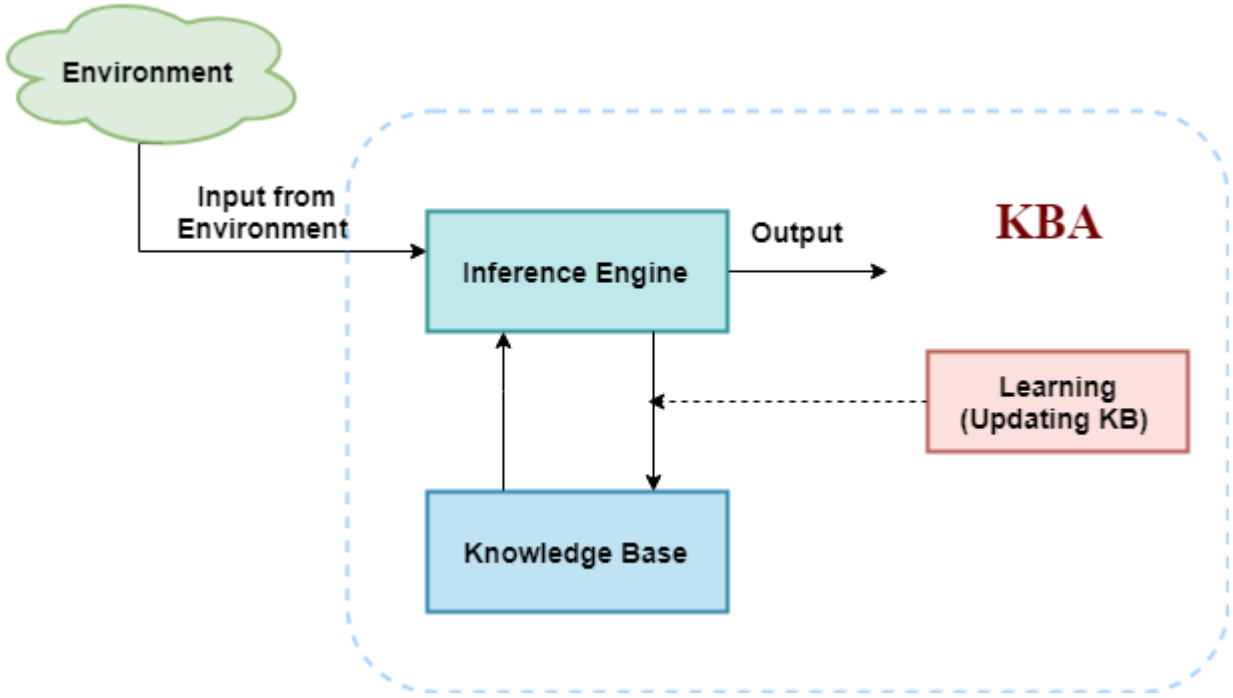
Knowledge-Based Agents

- An intelligent agent needs **knowledge** about the real world for taking decisions and **reasoning** to act efficiently.
- Knowledge-based agents are those agents who have the capability of **maintaining an internal state of knowledge, reason over that knowledge, update their knowledge after observations and take actions. These agents can represent the world with some formal representation and act intelligently**.
- Knowledge-based agents are composed of two main parts:
    - **Knowledge-base and**
    - **Inference system**.

A knowledge-based agent must able to do the following:

- An agent should be able to represent states, actions, etc.
- An agent Should be able to incorporate new precepts
- An agent can update the internal representation of the world
- An agent can deduce the internal representation of the world
- An agent can deduce appropriate actions.

The architecture of knowledge-based agent:



The above diagram is representing a generalized architecture for a knowledge-based agent. The knowledge-based agent (KBA) take input from the environment by perceiving the environment. The input is taken by the inference engine of the agent and which also communicate with KB to decide as per the knowledge store in KB. The learning element of KBA regularly updates the KB by learning new knowledge.

Knowledge base: Knowledgebase is a central component of a knowledge-based agent, it is also known as KB. It is a collection of sentences (here 'sentence' is a technical term, and it is not identical to sentence in English). These sentences are expressed in a language which is called a knowledge representation language. The Knowledgebase of KBA stores fact about the world.

Why use a knowledge base?

Knowledgebase is required for updating knowledge for an agent to learn with experiences and act as per the knowledge.

Inference system

Inference means deriving new sentences from old. Inference system allows us to add a new sentence to the knowledge base. A sentence is a proposition about the world. Inference system applies logical rules to the KB to deduce new information.

Inference system generates new facts so that an agent can update the KB. An inference system works mainly in two rules which are given as:

- **Forward chaining**
- **Backward chaining**

Operations Performed by KBA

Following are three operations which are performed by KBA in order to show the intelligent behavior:

1. **TELL:** This operation tells the knowledge base what it perceives from the environment.
2. **ASK:** This operation asks the knowledge base what action it should perform.
3. **Perform:** It performs the selected action.

A generic knowledge-based agent:

Following is the structure outline of a generic knowledge-based agents program:

```
function KB-AGENT(percept):
persistent: KB, a knowledge base
        t, a counter, initially 0, indicating time
TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
Action = ASK(KB, MAKE-ACTION-QUERY(t))
TELL(KB, MAKE-ACTION-SENTENCE(action, t))
 t = t + 1
 return action
```

The knowledge-based agent takes percept as input and returns an action as output. The agent maintains the knowledge base, KB, and it initially has some background knowledge of the real world. It also has a counter to indicate the time for the whole process, and this counter is initialized with zero.

Each time when the function is called, it performs its three operations:

- Firstly it TELLs the KB what it perceives.
- Secondly, it asks KB what action it should take
- Third agent program TELLS the KB that which action was chosen.

The MAKE-PERCEPT-SENTENCE generates a sentence as setting that the agent perceived the given percept at the given time.

The MAKE-ACTION-QUERY generates a sentence to ask which action should be done at the current time.

MAKE-ACTION-SENTENCE generates a sentence which asserts that the chosen action was executed.

Various levels of knowledge-based agent:

A knowledge-based agent can be viewed at different levels which are given below:

1. Knowledge level

Knowledge level is the first level of knowledge-based agent, and in this level, we need to specify what the agent knows, and what the agent goals are. With these specifications, we can fix its behavior. For example, suppose an automated taxi agent needs to go from a station A to station B, and he knows the way from A to B, so this comes at the knowledge level.

2. Logical level:

At this level, we understand that how the knowledge representation of knowledge is stored. At this level, sentences are encoded into different logics. At the logical level, an encoding of knowledge into logical sentences occurs. At the logical level we can expect to the automated taxi agent to reach to the destination B.

3. Implementation level:

This is the physical representation of logic and knowledge. At the implementation level agent perform actions as per logical and knowledge level. At this level, an automated taxi agent actually implements his knowledge and logic so that he can reach to the destination.

Approaches to designing a knowledge-based agent:

There are mainly two approaches to build a knowledge-based agent:

1. Declarative approach: We can create a knowledge-based agent by initializing with an empty knowledge base and telling the agent all the sentences with which we want to start with. This approach is called Declarative approach.

2. Procedural approach: In the procedural approach, we directly encode desired behavior as a program code. Which means we just need to write a program that already encodes the desired behavior or agent.

However, in the real world, a successful agent can be built by combining both declarative and procedural approaches, and declarative knowledge can often be compiled into more efficient procedural code.

Propositional logic

Propositional logic (PL) is the simplest form of logic where all the statements are made by propositions. A proposition is a declarative statement which is either true or false. It is a technique of knowledge representation in logical and mathematical form.

Example:

1. a) It is Sunday.
2. b) The Sun rises from West (False proposition)
3. c) 3+3= 7(False proposition)
4. d) 5 is a prime number.

Following are some basic facts about propositional logic:

- Propositional logic is also called Boolean logic as it works on 0 and 1.
- In propositional logic, we use symbolic variables to represent the logic, and we can use any symbol for a representing a proposition, such A, B, C, P, Q, R, etc.
- Propositions can be either true or false, but it cannot be both.
- Propositional logic consists of an object, relations or function, and **logical connectives**.
- These connectives are also called logical operators.
- The propositions and connectives are the basic elements of the propositional logic.
- Connectives can be said as a logical operator which connects two sentences.
- A proposition formula which is always true is called **tautology**, and it is also called a valid sentence.
- A proposition formula which is always false is called **Contradiction**.
- A proposition formula which has both true and false values is called**contingency**.

- Statements which are questions, commands, or opinions are not propositions such as "**Where is Rohini**", "**How are you**", "**What is your name**", are not propositions.

Syntax of propositional logic:

The syntax of propositional logic defines the allowable sentences for the knowledge representation. There are two types of Propositions:

a. Atomic Propositions
b. Compound propositions

- **Atomic Proposition:** Atomic propositions are the simple propositions. It consists of a single proposition symbol. These are the sentences which must be either true or false.

Example:

1. 2+2 is 4, it is an atomic proposition as it is a **true** fact.
2. b) "The Sun is cold" is also a proposition as it is a **false** fact.

- **Compound proposition:** Compound propositions are constructed by combining simpler or atomic propositions, using parenthesis and logical connectives.

Example:

1. a) "It is raining today, and street is wet."
2. b) "Ankit is a doctor, and his clinic is in Mumbai."

Logical Connectives:

Logical connectives are used to connect two simpler propositions or representing a sentence logically. We can create compound propositions with the help of logical connectives. There are mainly five connectives, which are given as follows:

1. **Negation:** A sentence such as ¬ P is called negation of P. A literal can be either Positive literal or negative literal.
2. **Conjunction:** A sentence which has ∧ connective such as, **P ∧ Q** is called a conjunction.

   Example: Rohan is intelligent and hardworking. It can be written as,

   P= Rohan is intelligent,

   Q= Rohan is hardworking. → P∧ Q.
3. **Disjunction:** A sentence which has ∨ connective, such as **P ∨ Q**. is called disjunction, where P and Q are the propositions.

   Example: "Ritika is a doctor or Engineer",

   Here P= Ritika is Doctor. Q= Ritika is Doctor, so we can write it as **P ∨ Q**.
4. **Implication:** A sentence such as P → Q, is called an implication. Implications are also known as if-then rules. It can be represented as

   If it is raining, then the street is wet.

   Let P= It is raining, and Q= Street is wet, so it is represented as P → Q
5. **Biconditional:** A sentence such as **P⇔ Q is a Biconditional sentence, example**

   If I am breathing, then I am alive

   P= I am breathing, Q= I am alive, it can be represented as P ⇔ Q.

Following is the summarized table for Propositional Logic Connectives:

Connective symbols	Word	Technical term	Example
∧	AND	Conjunction	A ∧ B
∨	OR	Disjunction	A ∨ B
→	Implies	Implication	A → B
⇔	If and only if	Biconditional	A⇔ B
¬ or ~	Not	Negation	¬ A or ¬ B

Truth Table:

In propositional logic, we need to know the truth values of propositions in all possible scenarios. We can combine all the possible combination with logical connectives, and the representation of these combinations in a tabular format is called **Truth table**. Following are the truth table for all logical connectives:

For Negation:

P	¬ P
True	**False**
False	**True**

For Conjunction:

P	Q	P∧ Q
True	True	**True**
True	False	**False**
False	True	**False**
False	False	**False**

For disjunction:

P	Q	P ∨ Q.
True	True	**True**
False	True	**True**
True	False	**True**
False	False	**False**

For Implication:

P	Q	P→ Q
True	True	**True**
True	False	**False**
False	True	**True**
False	False	**True**

For Biconditional:

P	Q	P⇔ Q
True	True	**True**
True	False	**False**
False	True	**False**
False	False	**True**

Truth table with three propositions:

We can build a proposition composing three propositions P, Q, and R. This truth table is made-up of 8n Tuples as we have taken three proposition symbols.

P	Q	R	¬R	P∨ Q	P∨Q→¬R
True	True	True	False	True	False
True	True	False	True	True	True
True	False	True	False	True	False
True	False	False	True	True	True
False	True	True	False	True	False
False	True	False	True	True	True
False	False	True	False	False	True
False	False	False	True	False	True

Precedence of connectives:

Just like arithmetic operators, there is a precedence order for propositional connectors or logical operators. This order should be followed while evaluating a propositional problem. Following is the list of the precedence order for operators:

Precedence	Operators
First Precedence	Parenthesis
Second Precedence	Negation
Third Precedence	Conjunction(AND)
Fourth Precedence	Disjunction(OR)
Fifth Precedence	Implication
Six Precedence	Biconditional

Logical equivalence:

Logical equivalence is one of the features of propositional logic. Two propositions are said to be logically equivalent if and only if the columns in the truth table are identical to each other.

Let's take two propositions A and B, so for logical equivalence, we can write it as A⇔B. In below truth table we can see that column for ¬A∨ B and A→B, are identical hence A is Equivalent to B

A	B	¬A	¬A∨ B	A→B
T	T	F	**T**	**T**
T	F	F	**F**	**F**
F	T	T	**T**	**T**
F	F	T	**T**	**T**

Properties of Operators:

- **Commutativity:**
  - P∧ Q= Q ∧ P, or
  - P ∨ Q = Q ∨ P.
- **Associativity:**
  - (P ∧ Q) ∧ R= P ∧ (Q ∧ R),
  - (P ∨ Q) ∨ R= P ∨ (Q ∨ R)
- **Identity element:**
  - P ∧ True = P,
  - P ∨ True= True.
- **Distributive:**
  - P∧ (Q ∨ R) = (P ∧ Q) ∨ (P ∧ R).
  - P ∨ (Q ∧ R) = (P ∨ Q) ∧ (P ∨ R).
- **DE Morgan's Law:**
  - ¬ (P ∧ Q) = (¬P) ∨ (¬Q)
  - ¬ (P ∨ Q) = (¬ P) ∧ (¬Q).
- **Double-negation elimination:**
  - ¬ (¬P) = P.

Limitations of Propositional logic:

- We cannot represent relations like ALL, some, or none with propositional logic. Example:
  a. **All the girls are intelligent.**
  b. **Some apples are sweet.**
- Propositional logic has limited expressive power.
- In propositional logic, we cannot describe statements in terms of their properties or logical relationships.

Propositional Theorem Proving: Inference and Proofs

Inference:

In artificial intelligence, we need intelligent computers which can create new logic from old logic or by evidence, **so generating the conclusions from evidence and facts is termed as Inference**.

Inference rules:

Inference rules are the templates for generating valid arguments. Inference rules are applied to derive proofs in artificial intelligence, and the proof is a sequence of the conclusion that leads to the desired goal.

In inference rules, the implication among all the connectives plays an important role. Following are some terminologies related to inference rules:

- **Implication:** It is one of the logical connectives which can be represented as P → Q. It is a Boolean expression.
- **Converse:** The converse of implication, which means the right-hand side proposition goes to the left-hand side and vice-versa. It can be written as Q → P.
- **Contrapositive:** The negation of converse is termed as contrapositive, and it can be represented as ¬ Q → ¬ P.
- **Inverse:** The negation of implication is called inverse. It can be represented as ¬ P → ¬ Q.

From the above term some of the compound statements are equivalent to each other, which we can prove using truth table:

P	Q	P → Q	Q→ P	¬ Q → ¬ P	¬ P → ¬ Q.
T	T	T	T	T	T
T	F	F	T	F	T
F	T	T	F	T	F
F	F	T	T	T	T

Hence from the above truth table, we can prove that P → Q is equivalent to ¬ Q → ¬ P, and Q→ P is equivalent to ¬ P → ¬ Q.

Types of Inference rules:

1. Modus Ponens:

The Modus Ponens rule is one of the most important rules of inference, and it states that if P and P → Q is true, then we can infer that Q will be true. It can be represented as:

Notation for Modus ponens: $\frac{P \rightarrow Q, \quad P}{\therefore Q}$

Example:

Statement-1: "If I am sleepy then I go to bed" ==> P→ Q

Statement-2: "I am sleepy" ==> P

Conclusion: "I go to bed." ==> Q.

Hence, we can say that, if P→ Q is true, and P is true then Q will be true.

Proof by Truth table:

P	Q	$P \rightarrow Q$
0	**0**	**0**
0	**1**	**1**
1	**0**	**0**
1	**1**	**1**

2. Modus Tollens:

The Modus Tollens rule state that if P→ Q is true and **¬ Q is true, then ¬ P** will also true. It can be represented as:

Notation for Modus Tollens: $\frac{P \rightarrow Q, \ \sim Q}{\sim P}$

Statement-1: "If I am sleepy then I go to bed" ==> P→ Q

Statement-2: "I do not go to the bed."==> ~Q

Statement-3: Which infers that "**I am not sleepy**" => ~P

Proof by Truth table:

P	Q	$\sim P$	$\sim Q$	$P \rightarrow Q$
0	**0**	**1**	**1**	**1**
0	**1**	**1**	**0**	**1**
1	**0**	**0**	**1**	**0**
1	**1**	**0**	**0**	**1**

3. Hypothetical Syllogism:

The Hypothetical Syllogism rule state that if P→R is true whenever P→Q is true, and Q→R is true. It can be represented as the following notation:

Example:

Statement-1: If you have my home key then you can unlock my home. **P→Q**

Statement-2: If you can unlock my home then you can take my money. **Q→R**

Conclusion: If you have my home key then you can take my money. **P→R**

Proof by truth table:

P	Q	R	$P \rightarrow Q$	$Q \rightarrow R$	$P \rightarrow R$	
0	0	0	1	1	1	←
0	0	1	1	1	1	←
0	1	0	1	0	1	
0	1	1	1	1	1	←
1	0	0	0	1	1	
1	0	1	0	1	1	
1	1	0	1	0	0	
1	1	1	1	1	1	←

4. Disjunctive Syllogism:

The Disjunctive syllogism rule state that if P∨Q is true, and ¬P is true, then Q will be true. It can be represented as:

Notation of Disjunctive syllogism: $\frac{P \vee Q, \ \neg P}{Q}$

Example:

Statement-1: Today is Sunday or Monday. ==>PvQ

Statement-2: Today is not Sunday. ==> ¬P

Conclusion: Today is Monday. ==> Q

Proof by truth-table:

P	Q	$\neg P$	$P \vee Q$
0	**0**	**1**	**0**
0	**1**	**1**	**1**
1	**0**	**0**	**1**
1	**1**	**0**	**1**

5. Addition:

The Addition rule is one the common inference rule, and it states that If P is true, then PvQ will be true.

Notation of Addition: $\frac{P}{P \vee Q}$

Example:

Statement: I have a vanilla ice-cream. ==> P

Statement-2: I have Chocolate ice-cream.

Conclusion: I have vanilla or chocolate ice-cream. ==> (PvQ)

Proof by Truth-Table:

P	Q	$P \vee Q$
0	**0**	**0**
1	**0**	**1**
0	**1**	**1**
1	**1**	**1**

6. Simplification:

The simplification rule state that if **P∧ Q** is true, then **Q or P** will also be true. It can be represented as:

Notation of Simplification rule: $\frac{P \wedge Q}{Q}$ Or $\frac{P \wedge Q}{P}$

Proof by Truth-Table:

P	Q	$P \wedge Q$
0	**0**	**0**
1	**0**	**0**
0	**1**	**0**
1	**1**	**1** ←

7. Resolution:

The Resolution rule state that if P∨Q and ¬ P∧R is true, then Q∨R will also be true. **It can be represented as**

Notation of Resolution $\frac{P \vee Q,\ \neg P \wedge R}{Q \vee R}$

Proof by Truth-Table:

P	¬ P	Q	R	$P \vee Q$	¬ P∧R	$Q \vee R$
0	**1**	**0**	**0**	**0**	**0**	**0**
0	**1**	**0**	**1**	**0**	**0**	**1**
0	**1**	**1**	**0**	**1**	**1**	**1** ←
0	**1**	**1**	**1**	**1**	**1**	**1** ←
1	**0**	**0**	**0**	**1**	**0**	**0**
1	**0**	**0**	**1**	**1**	**0**	**1**
1	**0**	**1**	**0**	**1**	**0**	**1**
1	**0**	**1**	**1**	**1**	**0**	**1** ←

Proof by Resolution

Resolution

The idea of resolution is simple: if we know that

- p is true or q is true
- and we also know that p is false or r is true
- then it must be the case that q is true or r is true.

This line of reasoning is formalized in the Resolution Tautology:

(p OR q) AND (NOT p OR r) -> q OR r

In order to apply resolution in a proof:

1. we express our hypotheses and conclusion as a product of sums (conjunctive normal form), such as those that appear in the Resolution Tautology.
2. each maxterm in the CNF of the hypothesis becomes a *clause* in the proof.
3. we apply the resolution tautology to pairs of clauses, producing new clauses.
4. if we produce all the clauses of the conclusion, we have proven it.

Proof with Resolution

Given the following hypotheses:

1. If it rains, Joe brings his umbrella (r -> u)
2. If Joe has an umbrella, he doesn't get wet (u -> NOT w)
3. If it doesn't rain, Joe doesn't get wet (NOT r -> NOT w)

prove that Joes doesn't get wet (NOT w)

We first put each hypothesis in CNF:

1. r -> u == (NOT r OR u)
2. u -> NOT w == (NOT u OR NOT w)
3. NOT r -> NOT w == (r OR NOT w)

We then use resolution on the hypotheses to derive the conclusion (NOT w):

1. NOT r OR u Premise
2. NOT u OR NOT w Premise
3. r OR NOT w Premise
4. NOT r OR NOT w L1, L2, resolution
5. NOT w OR NOT w L3, L4, resolution
6. NOT w L5, idempotence
7. QED

Proofs by Contradiction using Resolution

We can combine resolution with proof by contradiction (where we assert the negation of what we wish to prove, and from that premise derive FALSE) to direct our search towards smaller and smaller clauses, with the goal of producing FALSE.

Proof by contradiction:

(NOT p -> 0) == p

We use proof by contradiction to drive our search for a proof; we are looking for the smallest possible goal clause (false), so any use of equivalences or resolution that brings us to simpler expressions is working towards that goal.

We can redo the previous proof (about Joe and his umbrella) using proof by contradiction with resolution:

1. NOT r OR u Premise

2. NOT u OR NOT w Premise

3. r OR NOT w Premise

4. w Negation of conclusion

5. NOT r OR NOT w L1, L2, resolution

6. NOT w OR NOT w L3, L5, resolution

7. NOT w L6, idempotence

8. FALSE L4, L7, resolution

Proof by Resolution: Example 1

If either C173 or C220 is required, then all students will take computer science. C173 and C240 are required. Prove that all students will take computer science.

We formalize the proof as follows:

P1. (C173 OR C220) -> ACS

P2. C173

P3. C240

Prove: ACS

We then rewrite our hypotheses in conjunctive normal form:

P1: (NOT C173 OR ACS) (NOT C220 OR ACS)

P2: C173

P3: C240

Then we use proof by contradiction, by asserting the clauses of the premises and the negation of the conclusion, and deriving false.

1. NOT C173 OR ACS Premise
2. NOT C220 OR ACS Premise
3. C173 Premise
4. C240 Premise
5. NOT ACS Negation of conclusion
6. NOT C173 L1, L5, resolution
7. FALSE L3, L6, resolution

UNIT-III
First-Order Logic

In the topic of Propositional logic, we have seen how to represent statements using propositional logic. But unfortunately, in propositional logic, we can only represent the facts, which are either true or false. PL is not sufficient to represent complex sentences or natural language statements. The propositional logic has very limited expressive power. Consider the following sentence, which we cannot represent using PL logic.

- **"Some humans are intelligent", or**
- **"Sachin likes cricket."**

To represent the above statements, PL logic is not sufficient, so we required some more powerful logic, such as first-order logic.

First-Order logic:

- First-order logic is another way of knowledge representation in artificial intelligence. It is an extension to propositional logic.
- FOL is sufficiently expressive to represent the natural language statements in a concise way.
- First-order logic is also known as Predicate logic or First-order predicate logic. First-order logic is a powerful language that develops information about the objects in an easier way and can also express the relationship between those objects.
- First-order logic (like natural language) does not only assume that the world contains facts like propositional logic but also assumes the following things in the world:
  - **Objects:** A, B, people, numbers, colors, wars, theories, squares, pits, wumpus, ...
  - **Relations**: It can be unary relation such as: red, round, is adjacent, or n-any relation such as: the sister of, brother of, has color, comes between
  - **Function:** Father of, best friend, third inning of, end of, ...

- As a natural language, first-order logic also has two main parts:
  - **Syntax:** The syntax of first-order logic specifies the rules for constructing valid expressions, including terms and formulas. Terms are expressions that refer to objects and include constants, variables, and functions. Formulas are logical statements formed by combining predicates, terms, quantifiers, and logical connectives.
  - **Semantics:** The semantics of FOL define the meaning of the expressions. An interpretation assigns a domain of discourse and provides meaning to the constants, functions, and predicates. A formula is considered true under an interpretation if it accurately describes the relationships and properties of the objects in the domain.

Syntax of First-Order logic:

- The syntax of FOL determines which collection of symbols is a logical expression in first-order logic. The basic syntactic elements of first-order logic are symbols. We write statements in short-hand notation in FOL.

Basic Elements of First-order logic:

Following are the basic elements of FOL syntax:

- **Constants:** Constants represent specific objects within the domain of discourse. For example, in each domain, Alice, 2, and New York could be constants.
- **Variables:** Variables stand for unspecified objects in the domain. Commonly used symbols for variables include x, y, and z.
- **Predicates:** Predicates are functions that return true or false, representing properties of objects or relationships between them.
- For example, Likes (Alice, Bob) indicates that Alice likes Bob, and Greater than (x, 2) means that x is greater than 2.
- **Functions:** Functions map objects to other objects. For instance, Mother of(x) might denote the mother of x.

- **Quantifiers**: Quantifiers specify the scope of variables. The two main quantifiers are:
  - **Universal Quantifier (∀):** Indicates that a predicate applies to all elements in the domain.
  - For example, ∀x (Person(x) → Mortal(x)) means "All persons are mortal."
  - **Existential Quantifier (∃):** Indicates that there is at least one element in the domain for which the predicate holds.
  - For example, ∃x (Person(x) ∧ Likes(x, IceCream)) means "There exists a person who likes ice cream."
- **Logical Connectives:** Logical connectives include conjunction (∧), disjunction (∨), implication (→), biconditional (↔), and negation (¬). These connectives are used to form complex logical statements.

Atomic sentences:

- Atomic sentences are the most basic sentences of first-order logic. These sentences are formed from a predicate symbol followed by a parenthesis with a sequence of terms.
- We can represent atomic sentences as Predicate (term1, term2, ..., term n).

Example: Ravi and Ajay are brothers: => Brothers(Ravi, Ajay).

Chinky is a cat: => cat (Chinky).

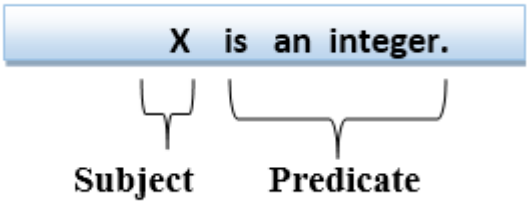
Complex Sentences:

- Complex sentences are made by combining atomic sentences using connectives.

First-order logic statements can be divided into two parts:

- **Subject:** Subject is the main part of the statement.
- **Predicate:** A predicate can be defined as a relation, which binds two atoms together in a statement.

Consider the statement: "x is an integer.", it consists of two parts, the first part x is the subject of the statement and second part "is an integer," is known as a predicate.



Quantifiers in First-order logic:

- A quantifier is a language element which generates quantification, and quantification specifies the quantity of specimen in the universe of discourse.
- These are the symbols that permit to determine or identify the range and scope of the variable in the logical expression. There are two types of quantifier:
    a. Universal Quantifier, (for all, everyone, everything)
    b. Existential quantifier, (for some, at least one).

Universal Quantifier:

Universal quantifier is a symbol of logical representation, which specifies that the statement within its range is true for everything or every instance of a particular thing.

The Universal quantifier is represented by a symbol ∀, which resembles an inverted A.

Note: In universal quantifier we use implication "→".

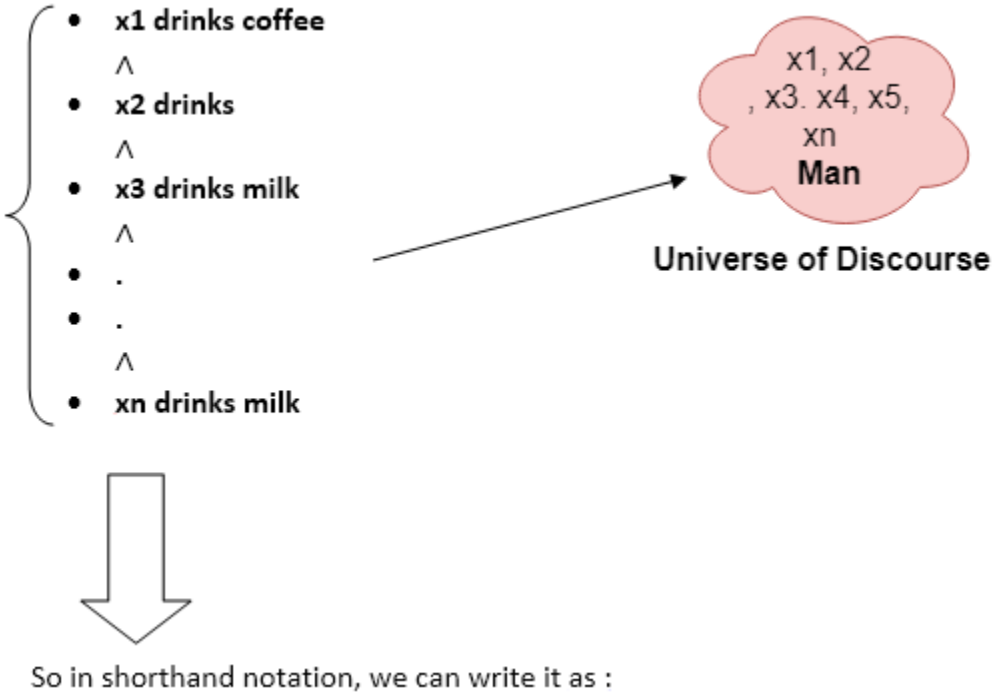
If x is a variable, then ∀x is read as:

- **For all x**
- **For each x**
- **For every x.**

Example:

All man drink coffee.

Let a variable x which refers to a cat so all x can be represented in UOD as below:



So in shorthand notation, we can write it as :

∀x man(x) → drink (x, coffee).

It will be read as: There are all x where x is a man who drink coffee.

Existential Quantifier:

Existential quantifiers are the type of quantifiers, which express that the statement within its scope is true for at least one instance of something.

It is denoted by the logical operator ∃, which resembles as inverted E. When it is used with a predicate variable then it is called as an existential quantifier.

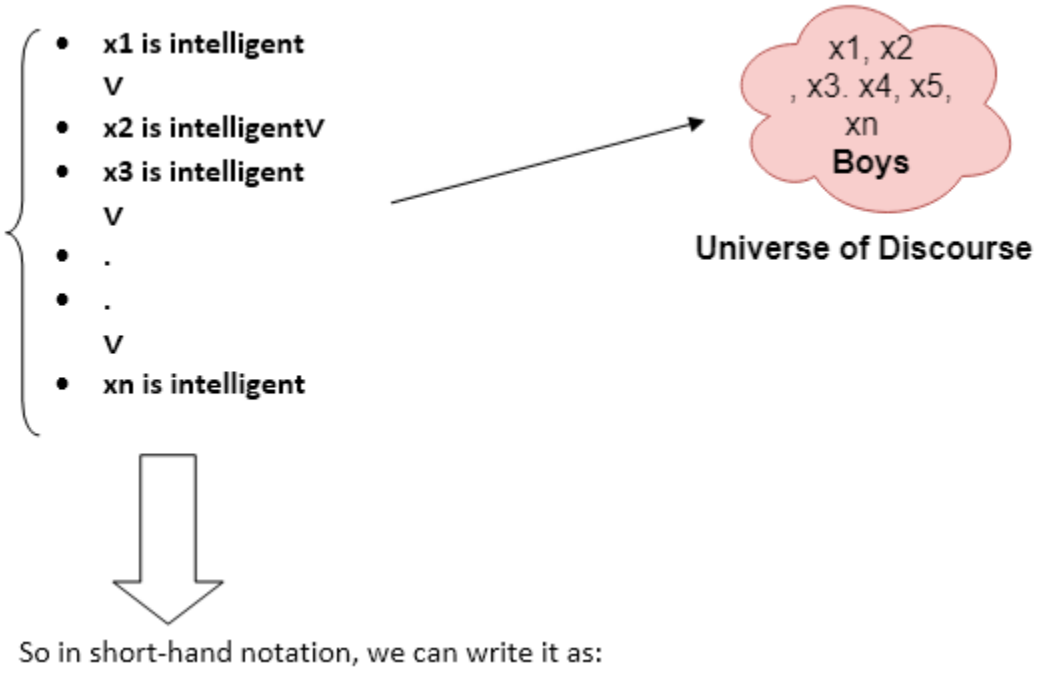
Note: In Existential quantifier we always use AND or Conjunction symbol (∧).

If x is a variable, then existential quantifier will be **∃x or ∃(x).** And it will be read as:

- **There exists a 'x.'**
- **For some 'x.'**
- **For at least one 'x.'**

Example:

Some boys are intelligent.



So in short-hand notation, we can write it as:

∃x: boys(x) ∧ intelligent(x)

Points to remember:

- The main connective for universal quantifier ∀ is implication →.
- The main connective for existential quantifier ∃ is and ∧.

Properties of Quantifiers:

- In universal quantifier, ∀x∀y is similar to ∀y∀x.
- In Existential quantifier, ∃x∃y is similar to ∃y∃x.
- ∃x∀y is not similar to ∀y∃x.

Some Examples of FOL using quantifier:

1. All birds fly.

In this question the predicate is "**fly(bird)**."

And since there are all birds who fly so it will be represented as follows.

∀x bird(x) →fly(x).

2.Every man respects his parent.

In this question, the predicate is "**respect(x, y)," where x=man, and y= parent**.

Since there is every man so will use ∀, and it will be represented as follows:

∀x man(x) → respects (x, parent).

3. Some boys play cricket.

In this question, the predicate is "**play(x, y)**," where x= boys, and y= game. Since there are some boys so we will use **∃, and it will be represented as**:

∃x boys(x) → play(x, cricket).

4. Not all students like both Mathematics and Science.

In this question, the predicate is "**like(x, y)," where x= student, and y= subject**.

Since there are not all students, so we will use **∀ with negation, so** following representation for this:

¬∀ (x) [student(x) → like(x, Mathematics) ∧ like(x, Science)].

5. Only one student failed in Mathematics.

In this question, the predicate is "**failed(x, y)," where x= student, and y= subject**.

Since there is only one student who failed in Mathematics, so we will use following representation for this:

∃(x) [student(x) → failed (x, Mathematics) ∧∀ (y) [¬(x==y) ∧ student(y) → ¬failed (x, Mathematics)].

Free and Bound Variables:

The quantifiers interact with variables which appear in a suitable way. There are two types of variables in First-order logic which are given below:

Free Variable: A variable is said to be a free variable in a formula if it occurs outside the scope of the quantifier.

Example: ∀x ∃(y)[P (x, y, z)], where z is a free variable.

Bound Variable: A variable is said to be a bound variable in a formula if it occurs within the scope of the quantifier.

Example: ∀x [A (x) B(y)], here x and y are the bound variables.

Applications of First-Order Logic in AI:

First-order logic is instrumental in various AI applications, enabling systems to reason about complex knowledge structures and make informed decisions. Some of the key applications of FOL in AI include:

1. **Knowledge Representation**: FOL provides a robust framework for representing complex relationships and properties of objects. For example, in a medical diagnosis system, predicates can represent symptoms, diseases, and their relationships, allowing the system to reason about potential diagnoses.
2. **Automated Theorem Proving**: Automated theorem proving involves using algorithms to prove mathematical theorems. FOL provides the foundational structure for many theorem proving systems, enabling them to verify software and hardware correctness, prove mathematical theorems, and more.
3. **Natural Language Processing (NLP)**: In NLP, FOL is used to parse and understand natural language by representing the meaning of sentences in a formal, logical manner. This allows AI systems to perform tasks such as question answering, machine translation, and text summarization.

4. **Expert Systems**: Expert systems encode expert knowledge using FOL and reason about it to make decisions. For example, a legal expert system might use FOL to represent legal rules and infer outcomes based on given facts, aiding in legal decision-making processes.
5. **Semantic Web**: The Semantic Web uses FOL to describe relationships between web resources, enabling more intelligent information retrieval and processing. This enhances the ability of AI systems to understand and interact with web content.

Advanced Concepts in First-Order Logic:

1. **Unification**: Unification is the process of finding a substitution that makes two logical expressions identical. It is a fundamental operation in automated reasoning and logic programming, enabling the matching of predicates with variables.
2. **Resolution**: Resolution is a rule of inference used for automated theorem proving. It involves refuting a set of clauses by deriving a contradiction, proving that the original statement is true. Resolution is a powerful method for proving logical theorems in FOL.
3. **Model Checking**: Model checking is a technique used to verify the correctness of systems with respect to a given specification. FOL is used to express the properties and behaviors of the system, enabling the verification of complex systems such as software, hardware, and protocols.
4. **Logic Programming**: Logic programming languages, such as Prolog, use FOL to express programs. These languages allow for declarative programming, where the programmer specifies what needs to be done rather than how to do it. Logic programming is used in various AI applications, including natural language processing, expert systems, and knowledge representation.

Knowledge Engineering in First-order logic

The process of constructing a knowledgebase in first-order logic is called knowledge-engineering. A knowledge engineer is someone who investigates a particular domain, learns what concepts are important in that domain, and creates a formal representation of the objects and relations in the domain.

In this topic, we will understand the Knowledge engineering process in an electronic circuit domain, which is already familiar. This approach is mainly suitable for creating **a special-purpose knowledge base**.

The knowledge-engineering process:

Knowledge engineering projects vary widely in content, scope, and difficulty, but all such projects include the following steps:

1. Identify the task. The knowledge engineer must set the range of questions that the knowledge base will support and the kinds of facts that will be available for each specific problem instance. For example, does the wumpus knowledge base need to be able to choose actions or is it required to answer questions only about the contents of the environment? Will the sensor facts include the current location? The task will determine what knowledge must be represented to connect problem instances to answers. This step is analogous to the PEAS process for designing agents in the previous chapter.

2. Assemble the relevant knowledge: The knowledge engineer might already be an expert in the domain or might need to work with real experts to extract what they know—a process called knowledge acquisition. At this stage, knowledge is not represented formally. The idea is to understand the scope of the knowledge base, as determined by the task, and to understand how the domain works. For the wumpus world, which is defined

by an artificial set of rules, the relevant knowledge is easy to identify. (Notice, however, that the definition of adjacency was not supplied explicitly in the wumpus-world rules.)

For real domains, the issue of relevance can be quite difficult—for example, a system for simulating VLSI designs might or might not need to consider stray capacitances and skin effects.

3. Decide on a vocabulary of predicates, functions, and constants: That is, translate the important domain-level concepts into logic-level names. This involves many questions of knowledge-engineering style. Like programming style, this can have a significant impact on the eventual success of the project. For example, should pits be represented by objects or by a unary predicate on squares? Should the agent's orientation be a function or a predicate? Should the wumpus's location depend on time? Once the ONTOLOGY choices have been made, the result is a vocabulary that is known as the ontology of the domain. The word ontology means a particular theory of the nature of being or existence. The ontology determines what kinds of things exist but does not determine their specific properties and interrelationships.

4. Encode general knowledge about the domain: The knowledge engineer writes down the axioms for all the vocabulary terms. This pins down (to the extent possible) the meaning of the terms, enabling the expert to check the content. Often, this step reveals misconceptions or gaps in the vocabulary that must be fixed by returning to step 3 and iterating through the process.

5. Encode a description of the specific problem instance: If the ontology is well thought out, this step will be easy. It will involve writing simple atomic sentences about instances of concepts that are already part of the ontology. For a logical agent, problem instances are supplied by the sensors, whereas a "disembodied" knowledge base is supplied with

additional sentences in the same way that traditional programs are supplied with input data.

6. Pose queries to the inference procedure and get answers: This is where the reward is: we can let the inference procedure operate on the axioms and problem-specific facts to derive the facts we are interested in knowing. Thus, we avoid the need for writing an application-specific solution algorithm.

7. Debug the knowledge base: Alas, the answers to queries will not often be correct on the first try. More precisely, the answers will be correct for the knowledge base as written, assuming that the inference procedure is sound, but they will not be the ones that the user is expecting.

For example, if an axiom is missing, some queries will not be answerable from the knowledge base. A considerable debugging process could ensue. Missing axioms or axioms that are too weak can be easily identified by noticing places where the chain of reasoning stops unexpectedly. For example, if the knowledge base includes a diagnostic rule for finding the wumpus,

$$\forall s \; Smelly(s) \Rightarrow Adjacent(Home(Wumpus), s) ,$$

instead of the biconditional, then the agent will never be able to prove the absence of wumpuses. Incorrect axioms can be identified because they are false statements about the world. For example, the sentence

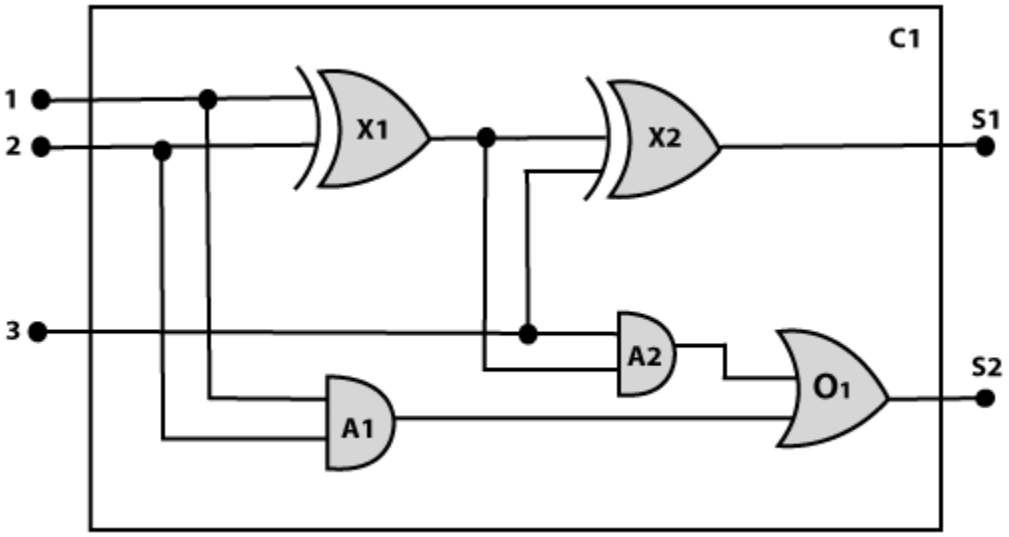
$$\forall x \; NumOfLegs(x, 4) \Rightarrow Mammal(x)$$

is false for reptiles, amphibians, and, more importantly, tables. The falsehood of this sentence can be determined independently of the rest of the knowledge base. In contrast,

a typical error in a program looks like this: offset = position + 1. It is impossible to tell whether this statement is correct without looking at the rest of the program to see whether, for example, offset is used to refer to the current position, or to one beyond the current position, or whether the value of position is changed by another statement and so offset should also be changed again. To understand this seven-step process better, we now apply it to an extended example—the domain of electronic circuits.

The electronic circuits domain:

We will develop an ontology and knowledge base that allows us to reason about digital circuits of the kind shown in the following figure. We follow the seven-step process for knowledge engineering.



1. Identify the task:

The first step of the process is to identify the task, and for the digital circuit, there are various reasoning tasks.

At the first level or highest level, we will examine the functionality of the circuit:

Does the circuit add properly?

What will be the output of gate A2, if all the inputs are high?

At the second level, we will examine the circuit structure details such as:

Which gate is connected to the first input terminal?

Does the circuit have feedback loops?

2. Assemble the relevant knowledge:

In the second step, we will assemble the relevant knowledge which is required for digital circuits. So for digital circuits, we have the following required knowledge:

- Logic circuits are made up of wires and gates.
- Signal flows through wires to the input terminal of the gate, and each gate produces the corresponding output which flows further.
- In this logic circuit, there are four types of gates used: **AND, OR, XOR, and NOT.**
- All these gates have one output terminal and two input terminals (except NOT gate, it has one input terminal).

3. Decide on vocabulary:

The next step of the process is to select functions, predicate, and constants to represent the circuits, terminals, signals, and gates. Firstly we will distinguish the gates from each other and from other objects. Each gate is represented as an object which is named by a constant, such as, **Gate(X1).** The functionality of each gate is determined by its type, which is taken as constants such as **AND, OR, XOR, or NOT**. Circuits will be identified by a predicate: Circuit (C1).

For the terminal, we will use predicate: Terminal(x).

For gate input, we will use the function In(1, X1) for denoting the first input terminal of the gate, and for output terminal we will use Out (1, X1).

The function Arity(c, i, j) is used to denote that circuit c has i input, j output.

The connectivity between gates can be represented by predicate Connect(Out(1, X1), In(1, X1)).

We use a unary predicate On (t), which is true if the signal at a terminal is on.

4. Encode general knowledge about the domain:

To encode the general knowledge about the logic circuit, we need some following rules:

- If two terminals are connected then they have the same input signal, it can be represented as:

∀ t1, t2 Terminal (t1) ∧ Terminal (t2) ∧ Connect (t1, t2) → Signal (t1) = Signal (2).

- Signal at every terminal will have either value 0 or 1, it will be represented as:

∀ t Terminal (t) →Signal (t) = 1 ∨Signal (t) = 0.

- Connect predicates are commutative:

∀ t1, t2 Connect(t1, t2) → Connect (t2, t1).

- Representation of types of gates:

∀ g Gate(g) ∧ r = Type(g) → r = OR ∨r = AND ∨r = XOR ∨r = NOT.

- Output of AND gate will be zero if and only if any of its input is zero.

∀ g Gate(g) ∧ Type(g) = AND →Signal (Out(1, g))= 0 ⇔ ∃n Signal (In(n, g))= 0.

- Output of OR gate is 1 if and only if any of its input is 1:

∀ g Gate(g) ∧ Type(g) = OR → Signal (Out(1, g))= 1 ⇔ ∃n Signal (In(n, g))= 1

- Output of XOR gate is 1 if and only if its inputs are different:

∀ g Gate(g) ∧ Type(g) = XOR → Signal (Out(1, g)) = 1 ⇔ Signal (In(1, g)) ≠ Signal (In(2, g)).

Output of NOT gate is invert of its input:

∀ g Gate(g) ∧ Type(g) = NOT → Signal (In(1, g)) ≠ Signal (Out(1, g)).

- All the gates in the above circuit have two inputs and one output (except NOT gate).

∀ g Gate(g) ∧ Type(g) = NOT → Arity(g, 1, 1)

∀ g Gate(g) ∧ r =Type(g) ∧ (r= AND ∨r= OR ∨r= XOR) → Arity (g, 2, 1).

- All gates are logic circuits:

∀ g Gate(g) → Circuit (g).

5. Encode a description of the problem instance:

Now we encode the problem of circuit C1, firstly we categorize the circuit and its gate components. This step is easy if ontology about the problem is already thought. This step

involves the writing simple atomics sentences of instances of concepts, which is known as ontology.

For the given circuit C1, we can encode the problem instance in atomic sentences as below:

Since in the circuit there are two XOR, two AND, and one OR gate so atomic sentences for these gates will be:

For XOR gate: Type(x1)= XOR, Type(X2) = XOR

For AND gate: Type(A1) = AND, Type(A2)= AND

For OR gate: Type (O1) = OR.

And then represent the connections between all the gates.

Note: Ontology defines a particular theory of the nature of existence.

6. Pose queries to the inference procedure and get answers:

In this step, we will find all the possible set of values of all the terminal for the adder circuit. The first query will be:

What should be the combination of input which would generate the first output of circuit C1, as 0 and a second output to be 1?

$\exists$ i1, i2, i3 Signal (In(1, C1))=i1 $\wedge$ Signal (In(2, C1))=i2 $\wedge$ Signal (In(3, C1))= i3

$\wedge$ Signal (Out(1, C1)) =0 $\wedge$ Signal (Out(2, C1))=1

7. Debug the knowledge base:

Now we will debug the knowledge base, and this is the last step of the complete process. In this step, we will try to debug the issues of knowledge base.

In the knowledge base, we may have omitted assertions like $1 \neq 0$.

Inference in First-Order Logic

Inference in First-Order Logic is used to deduce new facts or sentences from existing sentences. Before understanding the FOL inference rule, let's understand some basic terminologies used in FOL.

Substitution:

Substitution is a fundamental operation performed on terms and formulas. It occurs in all inference systems in first-order logic. The substitution is complex in the presence of quantifiers in FOL. If we write **F[a/x]**, so it refers to substitute a constant "**a**" in place of variable "**x**".

Note: First-order logic is capable of expressing facts about some or all objects in the universe.

Equality:

First-Order logic does not only use predicate and terms for making atomic sentences but also uses another way, which is equality in FOL. For this, we can use **equality symbols** which specify that the two terms refer to the same object.

Example: Brother (John) = Smith.

As in the above example, the object referred by the **Brother (John)** is similar to the object referred by **Smith**. The equality symbol can also be used with negation to represent that two terms are not the same objects.

Example: ¬(x=y) which is equivalent to x ≠y.

FOL inference rules for quantifier:

As propositional logic we also have inference rules in first-order logic, so following are some basic inference rules in FOL:

- **Universal Generalization**
- **Universal Instantiation**
- **Existential Instantiation**
- **Existential introduction**

1. Universal Generalization:

- Universal generalization is a valid inference rule which states that if premise P(c) is true for any arbitrary element c in the universe of discourse, then we can have a conclusion as ∀ x P(x).
- It can be represented as: $\frac{P(c)}{\forall x\, P(x)}$.
- This rule can be used if we want to show that every element has a similar property.
- In this rule, x must not appear as a free variable.

Example: Let's represent, P(c): "**A byte contains 8 bits**", so for **∀ x P(x)** "**All bytes contain 8 bits**.", it will also be true.

2. Universal Instantiation:

- Universal instantiation is also called as universal elimination or UI is a valid inference rule. It can be applied multiple times to add new sentences.
- The new KB is logically equivalent to the previous KB.
- As per UI, **we can infer any sentence obtained by substituting a ground term for the variable**.

- The UI rule state that we can infer any sentence P(c) by substituting a ground term c (a constant within domain x) from **∀ x P(x) for any object in the universe of discourse**.
- It can be represented as: $\frac{\forall x\, P(x)}{P(c)}$.

Example:1.

IF "Every person like ice-cream"=> ∀x P(x) so we can infer that

"John likes ice-cream" => P(c)

Example: 2. Let's take a famous example,

"All kings who are greedy are Evil." So let our knowledge base contains this detail as in the form of FOL:

∀x king(x) ∧ greedy (x) → Evil (x),

So from this information, we can infer any of the following statements using Universal Instantiation:

- **King(John) ∧ Greedy (John) → Evil (John),**
- **King(Richard) ∧ Greedy (Richard) → Evil (Richard),**
- **King(Father(John)) ∧ Greedy (Father(John)) → Evil (Father(John)),**

3. Existential Instantiation:

- Existential instantiation is also called as Existential Elimination, which is a valid inference rule in first-order logic.
- It can be applied only once to replace the existential sentence.
- The new KB is not logically equivalent to old KB, but it will be satisfiable if old KB was satisfiable.

- This rule states that one can infer P(c) from the formula given in the form of ∃x P(x) for a new constant symbol c.
- The restriction with this rule is that c used in the rule must be a new term for which P(c) is true.
- It can be represented as: $\frac{\exists x\, P(x)}{P(c)}$

Example:

From the given sentence: **∃x Crown(x) ∧ OnHead(x, John),**

So we can infer: **Crown(K) ∧ OnHead(K, John),** as long as K does not appear in the knowledge base.

- The above used K is a constant symbol, which is called **Skolem constant**.
- The Existential instantiation is a special case of **Skolemization process**.

4. Existential introduction

- An existential introduction is also known as an existential generalization, which is a valid inference rule in first-order logic.
- This rule states that if there is some element c in the universe of discourse which has a property P, then we can infer that there exists something in the universe which has the property P.
- It can be represented as: $\frac{P(c)}{\exists x P(x)}$
- **Example: Let's say that,**

  "Priyanka got good marks in English."

  "Therefore, someone got good marks in English."

Generalized Modus Ponens Rule:

For the inference process in FOL, we have a single inference rule which is called Generalized Modus Ponens. It is lifted version of Modus ponens.

Generalized Modus Ponens can be summarized as, " P implies Q and P is asserted to be true, therefore Q must be True."

According to Modus Ponens, for atomic sentences **pi, pi', q**. Where there is a substitution θ such that SUBST **(θ, pi',) = SUBST(θ, pi)**, it can be represented as:

$$\frac{p1', p2', \ldots, pn', (p1 \wedge p2 \wedge \ldots \wedge pn \Rightarrow q)}{SUBST(\theta, q)}$$

Example:

We will use this rule for Kings are evil, so we will find some x such that x is king, and x is greedy so we can infer that x is evil.

1. Here let say, p1' is king(John) p1 is king(x)
2. p2' is Greedy(y) p2 is Greedy(x)
3. θ is {x/John, y/John} q is evil(x)
4. SUBST(θ,q).

Unification

- Unification is a process of making two different logical atomic expressions identical by finding a substitution. Unification depends on the substitution process.
- It takes two literals as input and makes them identical using substitution.
- Let Ψ_1 and Ψ_2 be two atomic sentences and σ be a unifier such that, $\boldsymbol{\Psi_1\sigma = \Psi_2\sigma}$, then it can be expressed as **UNIFY(Ψ_1, Ψ_2)**.
- **Example: Find the MGU for Unify{King(x), King(John)}**

Let Ψ_1 = King(x), Ψ_2 = King(John),

Substitution θ = {John/x} is a unifier for these atoms and applying this substitution, and both expressions will be identical.

- The UNIFY algorithm is used for unification, which takes two atomic sentences and returns a unifier for those sentences (If any exist).
- Unification is a key component of all first-order inference algorithms.
- It returns fail if the expressions do not match with each other.
- The substitution variables are called Most General Unifier or MGU.

E.g. Let's say there are two different expressions, **P(x, y), and P(a, f(z))**.

In this example, we need to make both above statements identical to each other. For this, we will perform the substitution.

P(x, y)..... (i)

P(a, f(z))..... (ii)

- Substitute x with a, and y with f(z) in the first expression, and it will be represented as **a/x** and f(z)/y.
- With both the substitutions, the first expression will be identical to the second expression and the substitution set will be: **[a/x, f(z)/y]**.

Conditions for Unification:

Following are some basic conditions for unification:

- Predicate symbol must be same, atoms or expression with different predicate symbol can never be unified.
- Number of Arguments in both expressions must be identical.
- Unification will fail if there are two similar variables present in the same expression.

Unification Algorithm:

Algorithm: Unify(Ψ_1, Ψ_2)

```
Step. 1: If Ψ1 or Ψ2 is a variable or constant, then:
     a) If Ψ1 or Ψ2 are identical, then return NIL.
     b) Else if Ψ1is a variable,
           a. then if Ψ1 occurs in Ψ2, then return FAILURE
           b. Else return { (Ψ2/ Ψ1)}.
     c) Else if Ψ2 is a variable,
           a. If Ψ2 occurs in Ψ1 then return FAILURE,
           b. Else return {( Ψ1/ Ψ2)}.
     d) Else return FAILURE.
Step.2: If the initial Predicate symbol in Ψ1 and Ψ2 are not
same, then return FAILURE.
Step. 3: IF Ψ1 and Ψ2 have a different number of arguments, then
return FAILURE.
Step. 4: Set Substitution set(SUBST) to NIL.
```

Step. 5: For i=1 to the number of elements in Ψ_1.

a) Call Unify function with the ith element of Ψ_1 and ith element of Ψ_2, and put the result into S.

b) If S = failure then returns Failure

c) If S ≠ NIL then do,

a. Apply S to the remainder of both L1 and L2.

b. SUBST= APPEND(S, SUBST).

Step.6: Return SUBST.

Implementation of the Algorithm

Step.1: Initialize the substitution set to be empty.

Step.2: Recursively unify atomic sentences:

a. Check for Identical expression match.

b. If one expression is a variable v_i, and the other is a term t_i which does not contain variable v_i, then:

a. Substitute t_i / v_i in the existing substitutions

b. Add t_i /v_i to the substitution setlist.

c. If both the expressions are functions, then function name must be similar, and the number of arguments must be the same in both the expression.

For each pair of the following atomic sentences find the most general unifier (If exist).

1. Find the MGU of {p(f(a), g(Y)) and p(X, X)}

Sol: S_0 => Here, Ψ_1 = p(f(a), g(Y)), and Ψ_2 = p(X, X)

SUBST θ= {f(a) / X}

S1 => Ψ_1 = p(f(a), g(Y)), and Ψ_2 = p(f(a), f(a))

SUBST θ= {f(a) / g(y)}, **Unification failed**.

Unification is not possible for these expressions.

2. Find the MGU of {p(b, X, f(g(Z))) and p(Z, f(Y), f(Y))}

Here, Ψ_1 = p(b, X, f(g(Z))) , and Ψ_2 = p(Z, f(Y), f(Y))

S_0 => { p(b, X, f(g(Z))); p(Z, f(Y), f(Y))}

SUBST θ={b/Z}

S_1 => { p(b, X, f(g(b))); p(b, f(Y), f(Y))}

SUBST θ={f(Y) /X}

S_2 => { p(b, f(Y), f(g(b))); p(b, f(Y), f(Y))}

SUBST θ= {g(b) /Y}

S_2 => { p(b, f(g(b)), f(g(b)); p(b, f(g(b)), f(g(b))} **Unified Successfully.**

And Unifier = { b/Z, f(Y) /X , g(b) /Y}.

3. Find the MGU of {p (X, X), and p (Z, f(Z))}

Here, Ψ_1 = {p (X, X), and Ψ_2 = p (Z, f(Z))

S_0 => {p (X, X), p (Z, f(Z))}

SUBST θ= {X/Z}

S1 => {p (Z, Z), p (Z, f(Z))}

SUBST θ= {f(Z) / Z}, **Unification Failed**.

Hence, unification is not possible for these expressions.

4. Find the MGU of UNIFY(prime (11), prime(y))

Here, Ψ_1 = {prime(11) , and Ψ_2 = prime(y)}

S_0 => {prime(11) , prime(y)}

SUBST θ= {11/y}

S_1 => {prime(11) , prime(11)} , **Successfully unified.**

Unifier: {11/y}.

5. Find the MGU of Q(a, g(x, a), f(y)), Q(a, g(f(b), a), x)}

Here, Ψ_1 = Q(a, g(x, a), f(y)), and Ψ_2 = Q(a, g(f(b), a), x)

S_0 => {Q(a, g(x, a), f(y)); Q(a, g(f(b), a), x)}

SUBST θ= {f(b)/x}

S_1 => {Q(a, g(f(b), a), f(y)); Q(a, g(f(b), a), f(b))}

SUBST θ= {b/y}

S_1 => {Q(a, g(f(b), a), f(b)); Q(a, g(f(b), a), f(b))}, **Successfully Unified.**

Unifier: [a/a, f(b)/x, b/y].

6. UNIFY(knows(Richard, x), knows(Richard, John))

Here, Ψ_1 = knows(Richard, x), and Ψ_2 = knows(Richard, John)

S_0 => { knows(Richard, x); knows(Richard, John)}

SUBST θ= {John/x}

S_1 => { knows(Richard, John); knows(Richard, John)}, **Successfully Unified.**

Unifier: {John/x}.

Forward Chaining and backward chaining

In artificial intelligence, forward and backward chaining is one of the important topics, but before understanding forward and backward chaining lets first understand that from where these two terms came.

Inference engine:

The inference engine is the component of the intelligent system in artificial intelligence, which applies logical rules to the knowledge base to infer new information from known facts. The first inference engine was part of the expert system. Inference engine commonly proceeds in two modes, which are:

a. **Forward chaining**
b. **Backward chaining**

Horn Clause and Definite clause:

Horn clause and definite clause are the forms of sentences, which enables knowledge base to use a more restricted and efficient inference algorithm. Logical inference algorithms use forward and backward chaining approaches, which require KB in the form of the **first-order definite clause**.

Definite clause: A clause which is a disjunction of literals with **exactly one positive literal** is known as a definite clause or strict horn clause.

Horn clause: A clause which is a disjunction of literals with **at most one positive literal** is known as horn clause. Hence all the definite clauses are horn clauses.

Example: (¬ p V ¬ q V k). It has only one positive literal k.

It is equivalent to $p \wedge q \rightarrow k$.

Forward Chaining:

Forward chaining is also known as a forward deduction or forward reasoning method when using an inference engine. Forward chaining is a form of reasoning which start with atomic sentences in the knowledge base and applies inference rules (Modus Ponens) in the forward direction to extract more data until a goal is reached.

The Forward-chaining algorithm starts from known facts, triggers all rule whose premises are satisfied, and add their conclusion to the known facts. This process repeats until the problem is solved.

Properties of Forward-Chaining:

- It is a down-up approach, as it moves from bottom to top.
- It is a process of making a conclusion based on known facts or data, by starting from the initial state and reaches the goal state.
- Forward-chaining approach is also called as data-driven as we reach to the goal using available data.
- Forward -chaining approach is commonly used in the expert system, such as CLIPS, business, and production rule systems.

Consider the following famous example which we will use in both approaches:

Example:

"As per the law, it is a crime for an American to sell weapons to hostile nations. Country A, an enemy of America, has some missiles, and all the missiles were sold to it by Robert, who is an American citizen."

Prove that **"Robert is criminal."**

To solve the above problem, first, we will convert all the above facts into first-order definite clauses, and then we will use a forward-chaining algorithm to reach the goal.

Facts Conversion into FOL:

- It is a crime for an American to sell weapons to hostile nations. (Let's say p, q, and r are variables)

  American (p) ∧ weapon(q) ∧ sells (p, q, r) ∧ hostile(r) → Criminal(p) ...(1)

- Country A has some missiles. **?p Owns(A, p) ∧ Missile(p)**. It can be written in two definite clauses by using Existential Instantiation, introducing new Constant T1.

  Owns(A, T1) .....(2)

  Missile(T1) .....(3)

- All of the missiles were sold to country A by Robert.

  ?p Missiles(p) ∧ Owns (A, p) → Sells (Robert, p, A) .....(4)

- Missiles are weapons.

  Missile(p) → Weapons (p) .....(5)

- Enemy of America is known as hostile.

  Enemy(p, America) →Hostile(p) .....(6)

- Country A is an enemy of America.

  Enemy (A, America) .....(7)

- Robert is American

  American(Robert). .....(8)

Forward chaining proof:

Step-1:

In the first step we will start with the known facts and will choose the sentences which do not have implications, such as: **American (Robert), Enemy(A, America), Owns(A, T1), and Missile(T1)**. All these facts will be represented as below.



Step-2:

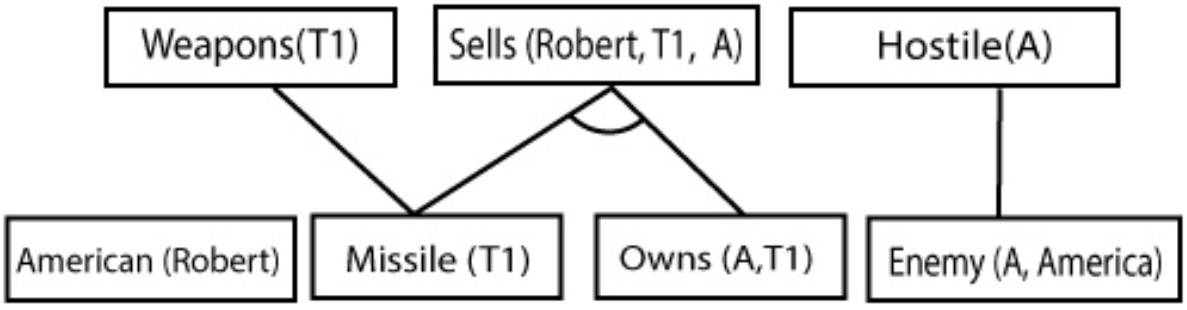
At the second step, we will see those facts which infer from available facts and with satisfied premises.

Rule-(1) does not satisfy premises, so it will not be added in the first iteration.

Rule-(2) and (3) are already added.

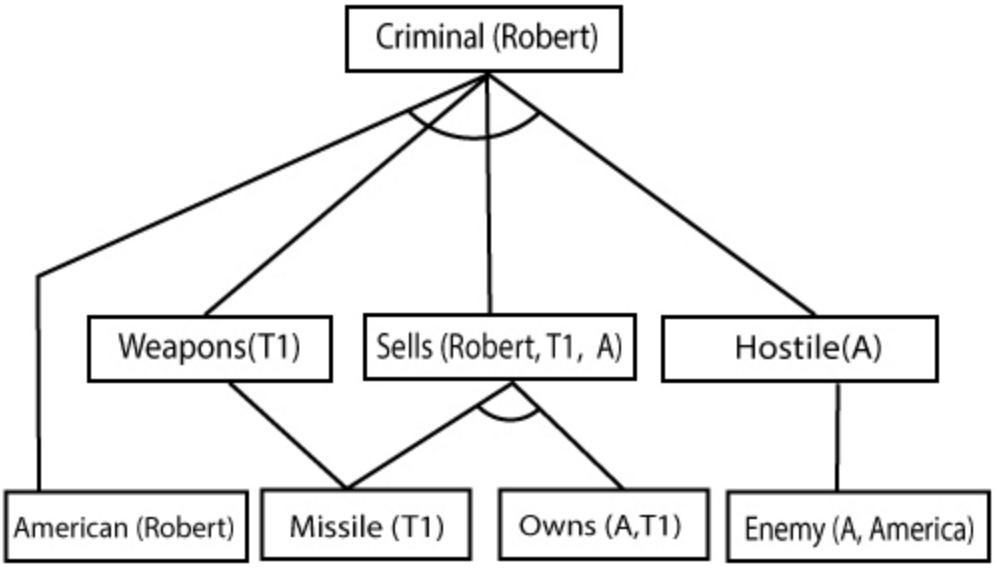
Rule-(4) satisfy with the substitution {p/T1}, **so Sells (Robert, T1, A)** is added, which infers from the conjunction of Rule (2) and (3).

Rule-(6) is satisfied with the substitution (p/A), so Hostile (A) is added and which infers from Rule-(7).



Step-3:

At step-3, as we can check Rule-(1) is satisfied with the substitution **{p/Robert, q/T1, r/A}, so we can add Criminal(Robert)** which infers all the available facts. And hence we reached our goal statement.



Hence it is proved that Robert is Criminal using forward chaining approach.

Backward Chaining:

Backward-chaining is also known as a backward deduction or backward reasoning method when using an inference engine. A backward chaining algorithm is a form of reasoning, which starts with the goal and works backward, chaining through rules to find known facts that support the goal.

Properties of backward chaining:

- It is known as a top-down approach.
- Backward-chaining is based on modus ponens inference rule.
- In backward chaining, the goal is broken into sub-goal or sub-goals to prove the facts true.

- It is called a goal-driven approach, as a list of goals decides which rules are selected and used.

Backward -chaining algorithm is used in game theory, automated theorem proving tools, inference engines, proof assistants, and various AI applications.

The backward-chaining method mostly used a **depth-first search** strategy for proof.

Example:

In backward-chaining, we will use the same above example, and will rewrite all the rules.

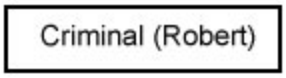
- **American (p) ∧ weapon(q) ∧ sells (p, q, r) ∧ hostile(r) → Criminal(p) ...(1)**
  Owns(A, T1) ……..(2)
- **Missile(T1)**
- **?p Missiles(p) ∧ Owns (A, p) → Sells (Robert, p, A) ……(4)**
- **Missile(p) → Weapons (p) …….(5)**
- **Enemy(p, America) →Hostile(p) ……..(6)**
- **Enemy (A, America) ……..(7)**
- **American (Robert). ……..(8)**

Backward-Chaining proof:

In Backward chaining, we will start with our goal predicate, which is **Criminal(Robert)**, and then infer further rules.

Step-1:

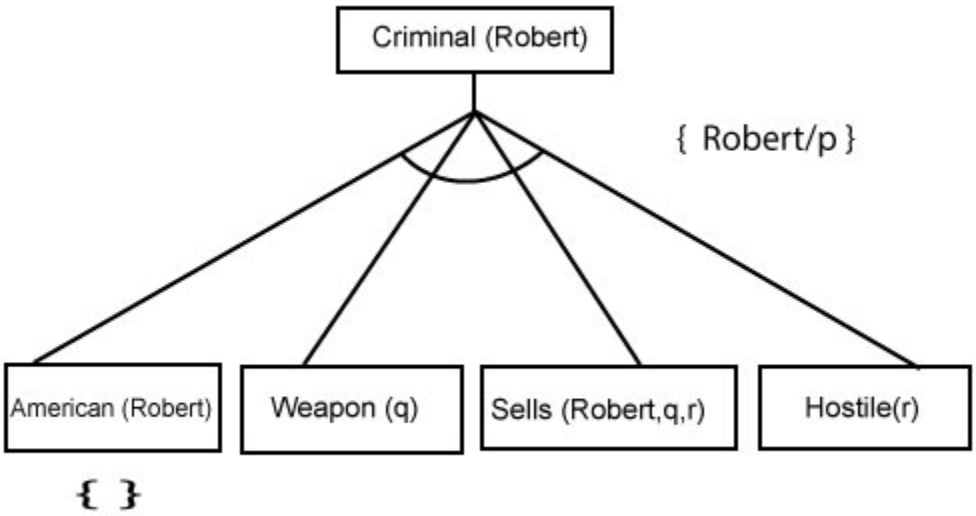
At the first step, we will take the goal fact. And from the goal fact, we will infer other facts, and at last, we will prove those facts true. So our goal fact is "Robert is Criminal," so following is the predicate of it.



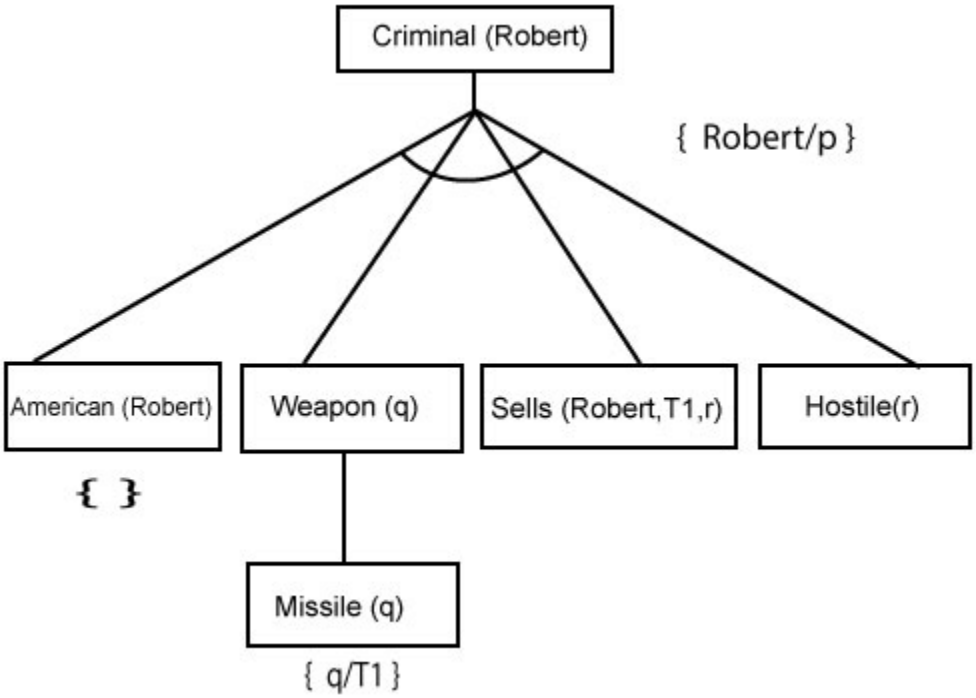
Step-2:

At the second step, we will infer other facts form goal fact which satisfies the rules. So as we can see in Rule-1, the goal predicate Criminal (Robert) is present with substitution {Robert/P}. So we will add all the conjunctive facts below the first level and will replace p with Robert.

Here we can see American (Robert) is a fact, so it is proved here.

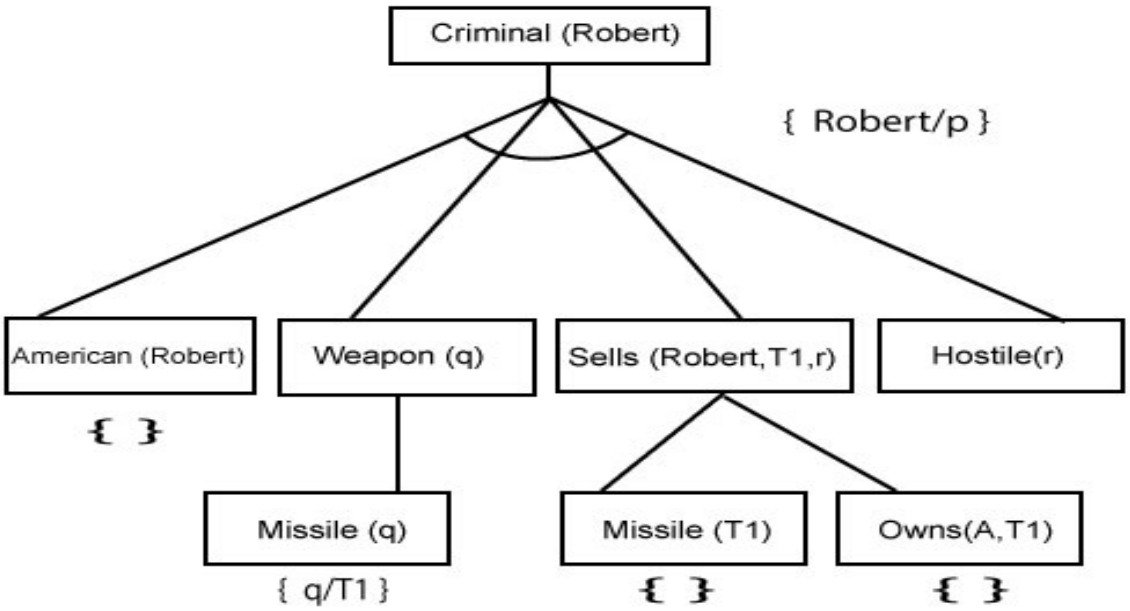


Step-3: At step-3, we will extract further fact Missile(q) which infer from Weapon(q), as it satisfies Rule-(5). Weapon (q) is also true with the substitution of a constant T1 at q.



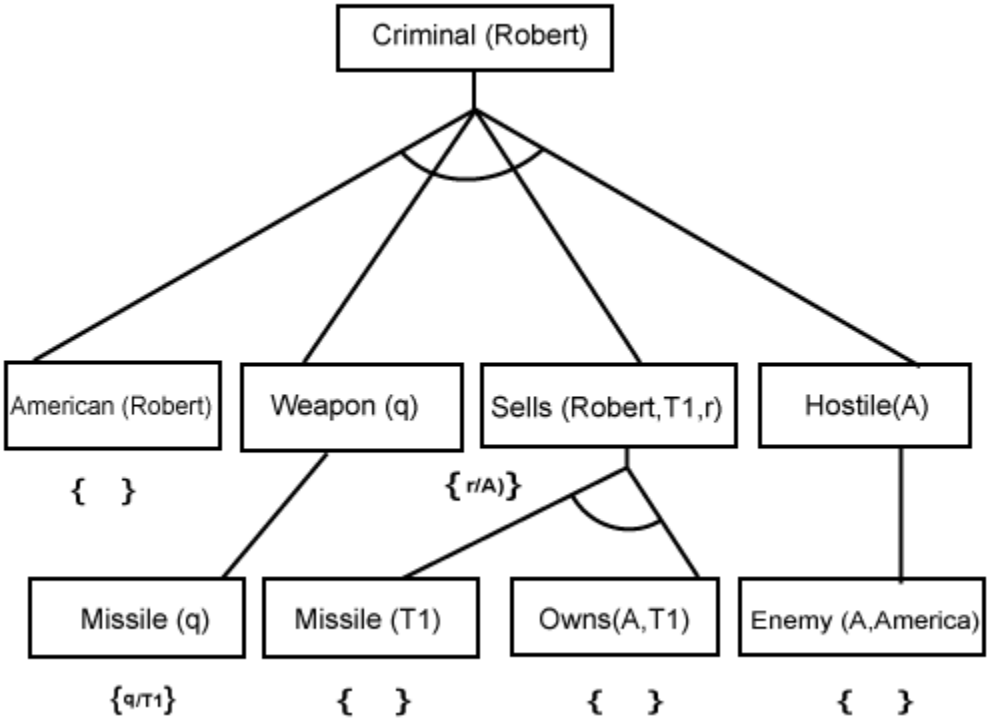
Step-4:

At step-4, we can infer facts Missile (T1) and Owns(A, T1) form Sells(Robert, T1, r) which satisfies the **Rule- 4**, with the substitution of A in place of r. So these two statements are proved here.



Step-5:

At step-5, we can infer the fact **Enemy(A, America)** from **Hostile(A)** which satisfies Rule-6. And hence all the statements are proved true using backward chaining.



Resolution

Resolution is a theorem proving technique that proceeds by building refutation proofs, i.e., proofs by contradictions. It was invented by a Mathematician John Alan Robinson in the year 1965.

Resolution is used, if there are various statements are given, and we need to prove a conclusion of those statements. Unification is a key concept in proofs by resolutions. Resolution is a single inference rule which can efficiently operate on the **conjunctive normal form or clausal form**.

Clause: Disjunction of literals (an atomic sentence) is called a **clause**. It is also known as a unit clause.

Conjunctive Normal Form: A sentence represented as a conjunction of clauses is said to be **conjunctive normal form** or **CNF**.

The resolution inference rule:

The resolution rule for first-order logic is simply a lifted version of the propositional rule. Resolution can resolve two clauses if they contain complementary literals, which are assumed to be standardized apart so that they share no variables.

$$\frac{l_1 \vee \ldots\ldots \vee l_{k,} \qquad m_1 \vee \ldots\ldots \vee m_n}{\text{SUBST}(\theta, l_1 \vee \ldots\ldots \vee l_{i-1} \vee l_{i+1} \vee \ldots \vee l_k \vee m_1 \vee \ldots\ldots \vee m_{j-1} \vee m_{j+1} \vee \ldots\ldots \vee m_n)}$$

Where $\mathbf{l_i}$ and $\mathbf{m_j}$ are complementary literals.

This rule is also called the **binary resolution rule** because it only resolves exactly two literals.

Example:

We can resolve two clauses which are given below:

[Animal (g(x) V Loves (f(x), x)] **and** **[¬ Loves(a, b) V ¬Kills(a, b)]**

Where two complimentary literals are: **Loves (f(x), x) and ¬ Loves (a, b)**

These literals can be unified with unifier **θ= [a/f(x), and b/x]** , and it will generate a resolvent clause:

[Animal (g(x) V ¬ Kills(f(x), x)].

Steps for Resolution:

1. Conversion of facts into first-order logic.
2. Convert FOL statements into CNF
3. Negate the statement which needs to prove (proof by contradiction)
4. Draw resolution graph (unification).

To better understand all the above steps, we will take an example in which we will apply resolution.

Example:

a. **John likes all kind of food.**

b. **Apple and vegetable are food**

c. **Anything anyone eats and not killed is food.**

d. **Anil eats peanuts and still alive**

e. **Harry eats everything that Anil eats.**

Prove by resolution that:

f. **John likes peanuts.**

Step-1: Conversion of Facts into FOL

In the first step we will convert all the given statements into its first order logic.

a. ∀x: food(x) → likes(John, x)
b. food(Apple) Λ food(vegetables)
c. ∀x ∀y: eats(x, y) Λ ¬ killed(x) → food(y)
d. eats (Anil, Peanuts) Λ alive(Anil).
e. ∀x : eats(Anil, x) → eats(Harry, x)
f. ∀x: ¬ killed(x) → alive(x) } **added predicates.**
g. ∀x: alive(x) →¬ killed(x) }
h. likes(John, Peanuts)

Step-2: Conversion of FOL into CNF

In First order logic resolution, it is required to convert the FOL into CNF as CNF form makes easier for resolution proofs.

- **Eliminate all implication (→) and rewrite**
  a. ∀x ¬ food(x) V likes(John, x)
  b. food(Apple) Λ food(vegetables)
  c. ∀x ∀y ¬ [eats(x, y) Λ ¬ killed(x)] V food(y)
  d. eats (Anil, Peanuts) Λ alive(Anil)
  e. ∀x ¬ eats(Anil, x) V eats(Harry, x)
  f. ∀x¬ [¬ killed(x)] V alive(x)
  g. ∀x ¬ alive(x) V ¬ killed(x)
  h. likes(John, Peanuts).

- **Move negation (¬) inwards and rewrite**
    a. ∀x ¬ food(x) V likes (John, x)
    b. food (Apple) Λ food(vegetables)
    c. ∀x ∀y ¬ eats (x, y) V killed(x) V food(y)
    d. eats (Anil, Peanuts) Λ alive (Anil)
    e. ∀x ¬ eats (Anil, x) V eats (Harry, x)
    f. ∀x ¬killed(x)] V alive(x)
    g. ∀x ¬ alive(x) V ¬ killed(x)
    h. likes (John, Peanuts).

- **Rename variables or standardize variables**
    a. ∀x ¬ food(x) V likes(John, x)
    b. food(Apple) Λ food(vegetables)
    c. ∀y ∀z ¬ eats(y, z) V killed(y) V food(z)
    d. eats (Anil, Peanuts) Λ alive(Anil)
    e. ∀w¬ eats(Anil, w) V eats(Harry, w)
    f. ∀g ¬killed(g)] V alive(g)
    g. ∀k ¬ alive(k) V ¬ killed(k)
    h. likes(John, Peanuts).

- **Eliminate existential instantiation quantifier by elimination.**

    In this step, we will eliminate existential quantifier ∃, and this process is known as **Skolemization**. But in this example problem since there is no existential quantifier so all the statements will remain same in this step.

- **Drop Universal quantifiers.**

    In this step we will drop all universal quantifier since all the statements are not implicitly quantified, so we don't need it.

a. ¬ food(x) V likes(John, x)
b. food(Apple)
c. food(vegetables)
d. ¬ eats(y, z) V killed(y) V food(z)
e. eats (Anil, Peanuts)
f. alive(Anil)
g. ¬ eats(Anil, w) V eats(Harry, w)
h. killed(g) V alive(g)
i. ¬ alive(k) V ¬ killed(k)
j. likes(John, Peanuts).

Note: Statements "food(Apple) Λ food(vegetables)" and "eats (Anil, Peanuts) Λ alive(Anil)" can be written in two separate statements.

- **Distribute conjunction Λ over disjunction ¬.**

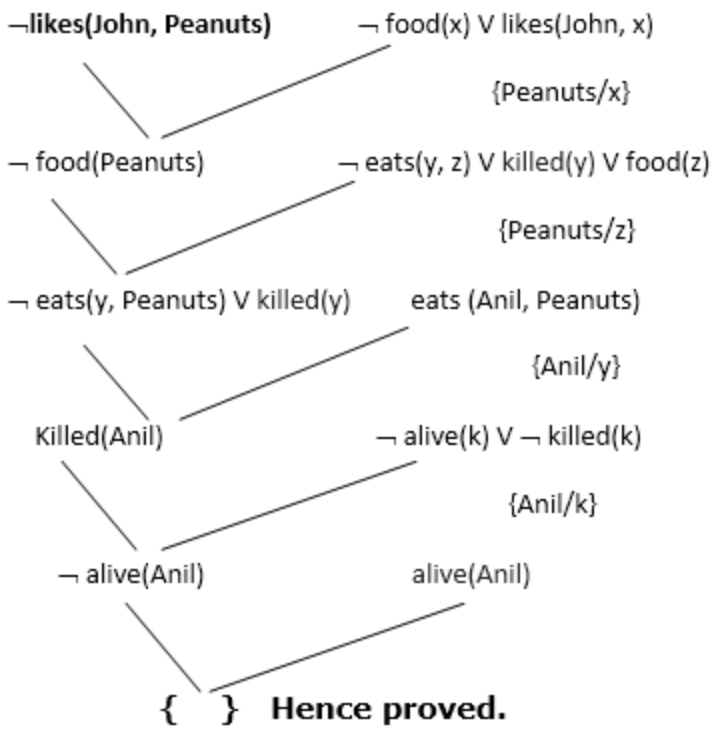
  This step will not make any change in this problem.

Step-3: Negate the statement to be proved

In this statement, we will apply negation to the conclusion statements, which will be written as ¬likes(John, Peanuts)

Step-4: Draw Resolution graph:

Now in this step, we will solve the problem by resolution tree using substitution. For the above problem, it will be given as follows:



Hence the negation of the conclusion has been proved as a complete contradiction with the given set of statements.

Explanation of Resolution graph:

- In the first step of resolution graph, **¬likes(John, Peanuts)** , and **likes(John, x)** get resolved(canceled) by substitution of **{Peanuts/x}**, and we are left with **¬ food(Peanuts)**
- In the second step of the resolution graph, **¬ food(Peanuts)** , and **food(z)** get resolved (canceled) by substitution of **{ Peanuts/z}**, and we are left with **¬ eats(y, Peanuts) V killed(y)** .
- In the third step of the resolution graph, **¬ eats(y, Peanuts)** and **eats (Anil, Peanuts)** get resolved by substitution **{Anil/y}**, and we are left with **Killed(Anil)** .
- In the fourth step of the resolution graph, **Killed(Anil)** and **¬ killed(k)** get resolve by substitution **{Anil/k}**, and we are left with **¬ alive(Anil)** .

- In the last step of the resolution graph **¬ alive(Anil)** and **alive(Anil)** get resolved.

UNIT- IV

Planning

We have defined AI as the study of rational action, which means that planning—devising a plan of action to achieve one's goals—is a critical part of AI. We have seen two examples of planning agents so far:

1. the search-based problem-solving agent and
2. the hybrid logical agent.

In this chapter we introduce a representation for planning problems that scales up to problems that could not be handled by those earlier approaches.

DEFINITION OF CLASSICAL PLANNING:

Classical planning refers to the process of building complex plan of actions by the agent while utilizing the structure of a problem efficiently.

The problem-solving agent can find sequences of actions that result in a goal state. But it deals with atomic representations of states and thus needs good domain-specific heuristics to perform well.

The hybrid propositional logical agent can find plans without domain-specific heuristics because it uses domain-independent heuristics based on the logical structure of the problem. But it depends on ground (variable-free) propositional inference, which means that it may be filled when there are many actions and states.

For example, in the wumpus world, the simple action of moving a step forward had to be repeated for all four agent orientations, T time steps, and n^2 current locations.

In response to this, planning researchers have settled on a factored representation— one in which a state of the world is represented by a

collection of variables. We use a language called PDDL, the Planning Domain Definition Language, that allows us to express all PDDL actions with one action schema. There have been several versions of PDDL; we selected a simple version. To define a search problem, PDDL describes the four things:

- the initial state,
- the actions that are available in a state,
- the result of applying an action, and
- the goal test.

States are represented using a set of changing aspects of the world(fluents). They are integrated using conjunction i.e., 'Λ'.

Actions are described by a set of action schemas that implicitly define the ACTIONS(s) and RESULT (s, a) functions needed to do a problem-solving search.

Classical planning usually focuses on the problems which do not modify most of the things with its actions. The set of actions should be represented in a concise form like 'Δ' instead of specifying the overall objects. Another way of concise representation used by PDDL is the specification of only the things which are changed. The remaining things are simply ignored.

Another way of concise representation for actions which are variable-free is to use a single action schema to represent a set of actions. A typical schema carries action name, variables involved, a precondition and the effect. Here, the precondition represents the states of executing the action and effect represents the output of action. Both these fields are conjunctions of literals. If a state satisfies the precondition, then the action associated with it is called 'applicable'.

Example: The spare tire problem

The problem of spare tire occurs when a tire is punctured and that tire must be replaced with the spare tire (i.e., new tire) to make the vehicle free from puncture.

Initial State of the Vehicle: (Flat → Axle) ^ (Spare → Trunk)

according to the initial state, a flat tire (i.e., puncture tire) is on the axle of the vehicle and spare tire is on the trunk.

Goal State of the Vehicle: (Spare → Axle) ^ (Flat → Trunk)

In the goal state, the spare tire must be mounted on the axle of the vehicle and flat tire must be mounted on the trunk of the vehicle.

The following actions must be performed to replace a flat tire with a spare tire on the axle.

1) First, separate the flat tire from the axle.
2) Now, separate the spare tire from the trunk.
3) Then place spare tire on the axle
4) Finally, place a flat tire on the trunk.

The following are the sequence of events which must be followed with the above actions to achieve the goal state.

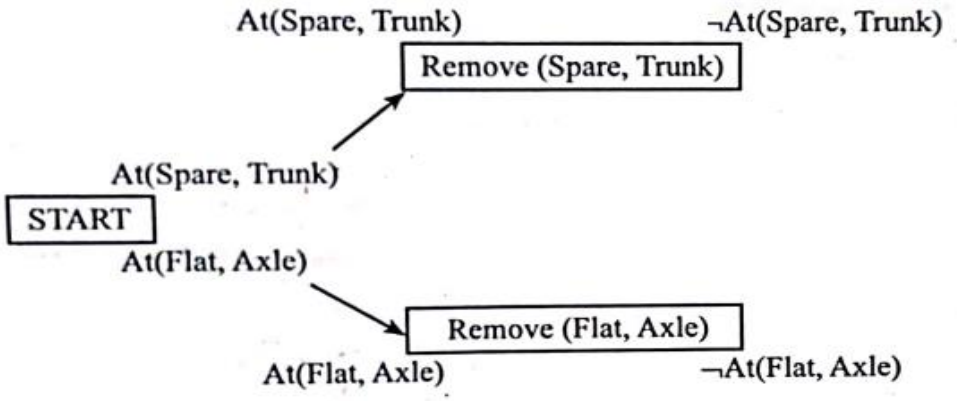
1. Action (Remove (Spare, Trunk))
   Precond: At (Spare, Trunk)
   Effect: ¬At (Spare, Trunk)

This action is performed for removing a spare tire from the trunk with a precondition that the spare tire must be available on the trunk so that it can be removed from the trunk. However, this action gets effected when there is no spare tire on the trunk.

2. Action (Remove (Flat, Axle))
   Precond: At(Flat, Axle)
   Effect: ¬At(Flat, Axle)

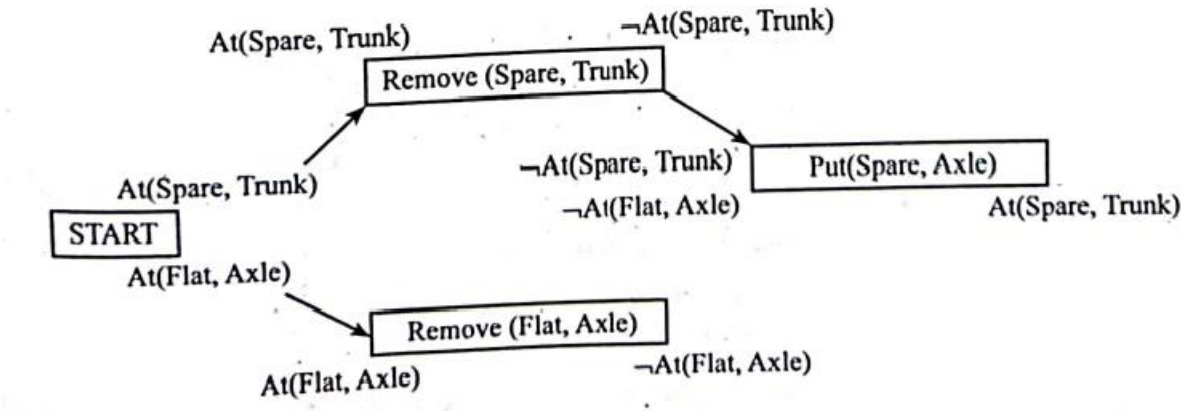
This action is performed for removing flat tire from the axle with a precondition that the flat tire must be available on the axle so that it can be removed from the axle. However, this action gets effected when there is no flat tire on the axle.

in the below figure, actions are represented by the boxes. The left and right sides of the boxes represent preconditions and their effects respectively.



3. Action (Put (Spare, Axle))
   Precond: ¬At (Spare, Trunk) ^ ¬ (Flat, Axle)
   Effect: At (Spare, Trunk)

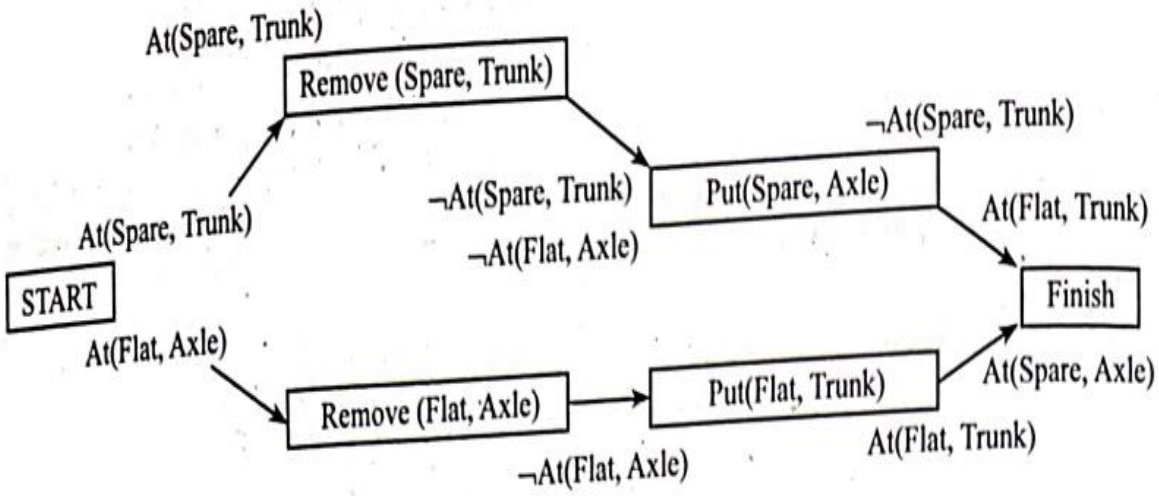
This action is performed by placing spare tire on the axle of the vehicle with a precondition that the spare tire which has to be placed on axle must not be on the trunk (i.e., ¬At (Spare, Trunk)) and flat tire must not be on the axle. However, this action gets effected if the spare tire is still on the trunk.



4. Action(Put(Flat, Trunk)
   Precond: ¬At(Flat,Axle)^ ¬At(Spare, Trunk)
   Effect:At(Flat, Trunk)

This action is performed by placing the flat tire on the trunk with a precondition that the flat tire must not be present on the axle and spare tire must not be on the trunk so that it can mount a new spare tire on the axle of the vehicle. However, this action gets effected when the flat tire is still on the trunk.

The below figure illustrates the goal state of the spare tire problem by following all the actions and their sequence of events.



A solution to the problem is [Remove (Flat, Axle), Remove (Spare, Trunk), PutOn(Spare, Axle) PutOn(Flat, Trunk)].

$$
\begin{array}{l}
Init(Tire(Flat) \wedge Tire(Spare) \wedge At(Flat, Axle) \wedge At(Spare, Trunk)) \\
Goal(At(Spare, Axle)) \\
Action(Remove(obj, loc), \\
\quad \text{PRECOND: } At(obj, loc) \\
\quad \text{EFFECT: } \neg At(obj, loc) \wedge At(obj, Ground)) \\
Action(PutOn(t, Axle), \\
\quad \text{PRECOND: } Tire(t) \wedge At(t, Ground) \wedge \neg At(Flat, Axle) \\
\quad \text{EFFECT: } \neg At(t, Ground) \wedge At(t, Axle)) \\
Action(LeaveOvernight, \\
\quad \text{PRECOND:} \\
\quad \text{EFFECT: } \neg At(Spare, Ground) \wedge \neg At(Spare, Axle) \wedge \neg At(Spare, Trunk) \\
\qquad \wedge \neg At(Flat, Ground) \wedge \neg At(Flat, Axle) \wedge \neg At(Flat, Trunk))
\end{array}
$$

Figure 10.2 The simple spare tire problem.

Example: The blocks world

- The blocks world One of the most famous planning domains is known as the blocks world.
- This domain consists of a set of cube-shaped blocks sitting on a table.
- The blocks can be stacked, but only one block can fit directly on top of another.
- A robot arm can pick up a block and move it to another position, either on the table or on top of another block.
- The arm can pick up only one block at a time, so it cannot pick up a block that has another one on it.
- The **goal** will always be to build one or more stacks of blocks, specified in terms of what blocks are on top of what other blocks.
- For example, a goal might be to get block A on B and block B on C.

$Init(On(A, Table) \land On(B, Table) \land On(C, A)$
$\quad \land Block(A) \land Block(B) \land Block(C) \land Clear(B) \land Clear(C))$
$Goal(On(A, B) \land On(B, C))$
$Action(Move(b, x, y),$
$\quad$ PRECOND: $On(b, x) \land Clear(b) \land Clear(y) \land Block(b) \land Block(y) \land$
$\quad\quad (b \neq x) \land (b \neq y) \land (x \neq y),$
$\quad$ EFFECT: $On(b, y) \land Clear(x) \land \neg On(b, x) \land \neg Clear(y))$
$Action(MoveToTable(b, x),$
$\quad$ PRECOND: $On(b, x) \land Clear(b) \land Block(b) \land (b \neq x),$
$\quad$ EFFECT: $On(b, Table) \land Clear(x) \land \neg On(b, x))$

Figure 10.3 A planning problem in the blocks world: building a three-block tower. One solution is the sequence $[MoveToTable(C, A), Move(B, Table, C), Move(A, Table, B)]$.

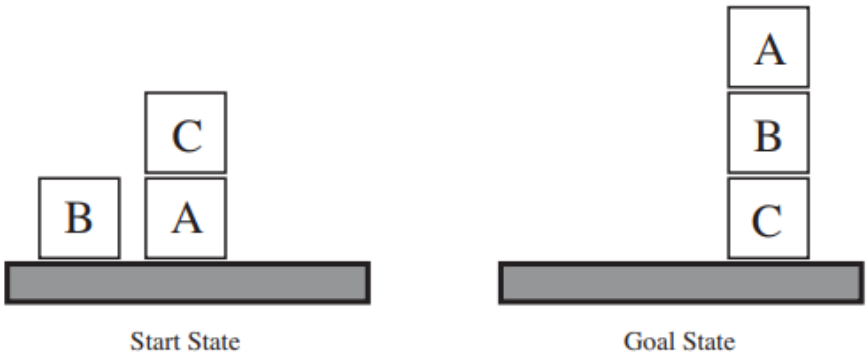


Figure 10.4 Diagram of the blocks-world problem in Figure 10.3.

- We use On (b, x) to indicate that block b is on x, where x is either another block or the table.
- The action for moving block b from the top of x to the top of y will be Move (b, x, y).
- Now, one of the preconditions on moving b is that no other block be on it.
- In first-order logic, this would be ¬∃ x On (x, b) or, alternatively, ∀ x ¬On (x, b).
- Basic PDDL does not allow quantifiers, so instead we introduce a predicate Clear (x) that is true when nothing is on x.
- The action Move moves a block b from x to y if both b and y are clear.

- After the move is made, b is still clear but y is not.
- A first attempt at the Move schema is

  Action(Move(b, x, y),

  PRECOND:On(b, x) ∧ Clear (b) ∧ Clear (y),

  EFFECT:On(b, y) ∧ Clear (x) ∧ ¬On(b, x) ∧ ¬Clear (y)) .
- Unfortunately, this does not maintain Clear properly when x or y is the table.
- When x is the Table, this action has the effect Clear (Table), but the table should not become clear; and when y = Table, it has the precondition Clear (Table), but the table does not have to be clear for us to move a block onto it. To fix this, we do two things.
- First, we introduce another action to move a block b from x to the table:

  Action(MoveToTable(b, x),

  PRECOND:On(b, x) ∧ Clear (b),

  EFFECT:On(b, Table) ∧ Clear (x) ∧ ¬On(b, x)) .
- Second, we take the interpretation of Clear (x) to be "there is a clear space on x to hold a block." Under this interpretation, Clear (Table) will always be true.
- The only problem is that nothing prevents the planner from using Move (b, x, Table) instead of MoveToTable(b, x).
- We could live with this problem—it will lead to a larger-than-necessary search space but will not lead to incorrect answers—or we could introduce the predicate Block and add Block(b)∧ Block(y) to the precondition of Move.

Example: Air cargo transport

$Init(At(C_1, SFO) \land At(C_2, JFK) \land At(P_1, SFO) \land At(P_2, JFK)$
$\land Cargo(C_1) \land Cargo(C_2) \land Plane(P_1) \land Plane(P_2)$
$\land Airport(JFK) \land Airport(SFO))$
$Goal(At(C_1, JFK) \land At(C_2, SFO))$
$Action(Load(c, p, a),$
PRECOND: $At(c, a) \land At(p, a) \land Cargo(c) \land Plane(p) \land Airport(a)$
EFFECT: $\neg At(c, a) \land In(c, p))$
$Action(Unload(c, p, a),$
PRECOND: $In(c, p) \land At(p, a) \land Cargo(c) \land Plane(p) \land Airport(a)$
EFFECT: $At(c, a) \land \neg In(c, p))$
$Action(Fly(p, from, to),$
PRECOND: $At(p, from) \land Plane(p) \land Airport(from) \land Airport(to)$
EFFECT: $\neg At(p, from) \land At(p, to))$

Figure 10.1 A PDDL description of an air cargo transportation planning problem.

- Figure 10.1 shows an air cargo transport problem involving loading and unloading cargo and flying it from place to place.
- The problem can be defined with three actions: Load, Unload, and Fly.
- The actions affect two predicates: In(c, p) means that cargo c is inside plane p, and At(x, a) means that object x (either plane or cargo) is at airport a. Note that some care must be taken to make sure the At predicates are maintained properly.
- When a plane flies from one airport to another, all the cargo inside the plane goes with it.
- In first-order logic it would be easy to quantify all objects that are inside the plane. But basic PDDL does not have a universal quantifier, so we need a different solution.
- The approach we use is to say that a piece of cargo ceases to be At anywhere when it is In a plane; the cargo only becomes At the new airport when it is unloaded.
- So At really means "available for use at a given location."

- The following plan is a solution to the problem:
- [Load (C1, P1, SFO),Fly(P1, SFO, JFK), Unload(C1, P1, JFK), Load(C2, P2, JFK),Fly(P2, JFK , SFO), Unload(C2, P2, SFO)] .
- Finally, there is the problem of spurious actions such as Fly(P1, JFK, JFK), which should be a no-op, but which has contradictory effects (according to the definition, the effect would include At(P1, JFK) ∧ ¬At(P1, JFK)).
- It is common to ignore such problems, because they seldom cause incorrect plans to be produced. The correct approach is to add inequality preconditions saying that the from and to airports must be different.

Algorithms for Planning with State-Space Search

The most straightforward approach of planning algorithm, is state- space search

- **Forward state-space search (Progression)**
- **Backward state-space search (Regression)**

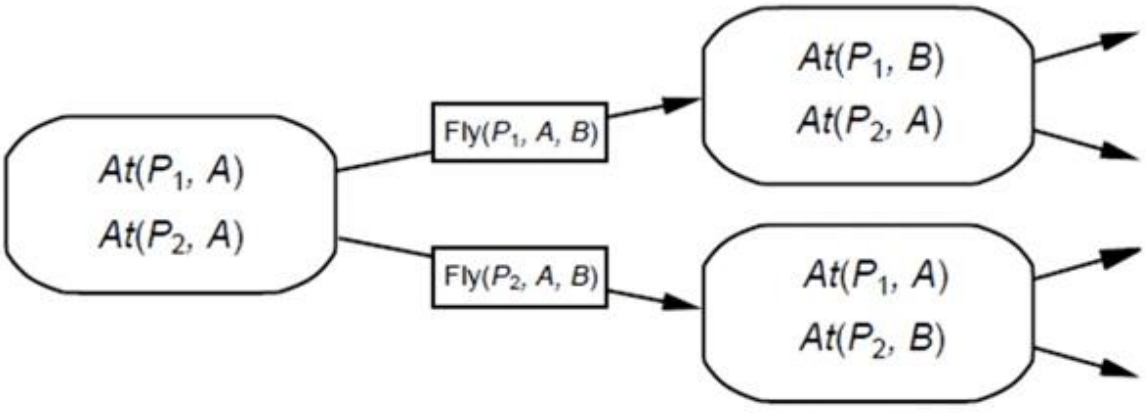
✓ The descriptions of actions in a planning problem and specify both preconditions and effects.

✓ It is possible to search in both directions: either forward from the initial state or backward from the goal.

✓ We can also use explicit action and goal representations, to derive effective heuristics automatically.

Forward State Space Search (Progression):

- From initial state, search forward by selecting operators whose preconditions can be unified with literals in the state
- New state includes positive literals of effect; the negated literals of effect are deleted
- Search forward until goal unifies with resulting state
- This is forward state-space search using STRIPS operators



Problem Formulation for Forward State Space Search (Progression):

Initial state:

- Initial state of the planning problem

Actions:

- Applicable to the current state.
- First actions' preconditions are satisfied, Successor states are generated
- Add positive literals to add list and negative literals to delete list.

Goal test:

- Whether the state satisfies the goal of the planning

Step cost:

- Fach action is 1 (assumed)

FSSS Algorithm

(1) compute whether or not a state is a goal state,

(2) find the set of all actions that are applicable to a state, and

(3) compute a successor state, that is the result of applying an action to a state

- Algorithm takes as input the statement P = (0, s0, g) of a planning problem P. (O contains a list of actions)
- If P is solvable, then Forward-search (O, s0, g) returns a solution plan; otherwise, it returns failure.

0r

1. Forward-search (O, so, g)
2. s←s0
3. л←the empty plan
4. loop
    1. if s satisfies g then return л
    2. applicable {a | a is a ground instance of an operator in O, and precond(a) is true in s}

3. if applicable = Ø then return failure

4. Non-deterministically choose an action a E applicable

5. s←y(s, a)

6. л←л.a

Example: Transportation of air cargo between airports.

Init (At(C1, SFO) ^ At(C2, JFK) ^ At(P1, SFO) ^ At(P2, JFK) ^ Cargo (C1) ^Cargo (C2)

Plane (P1) ^Plane (P2)^ Airport (JFK) ^ Airport (SFO))

Goal (At(C1, JFK) ^ At(C2, SFO))

Action (Load(c, p, a),

PRECOND: At(c, a) ^ At(p, a) ^ Cargo (c) ^ Plane (p) ^ Airport (a)

EFFECT: ¬ At(c, a) ^ In(c, p))

Action (Unload (c, p, a),

PRECOND: In(c, p) ^ At(p, a) ^ Cargo (c) Plane (p) ^ Airport (a)

EFFECT: At(c, a) ^ ¬ In(c, p))

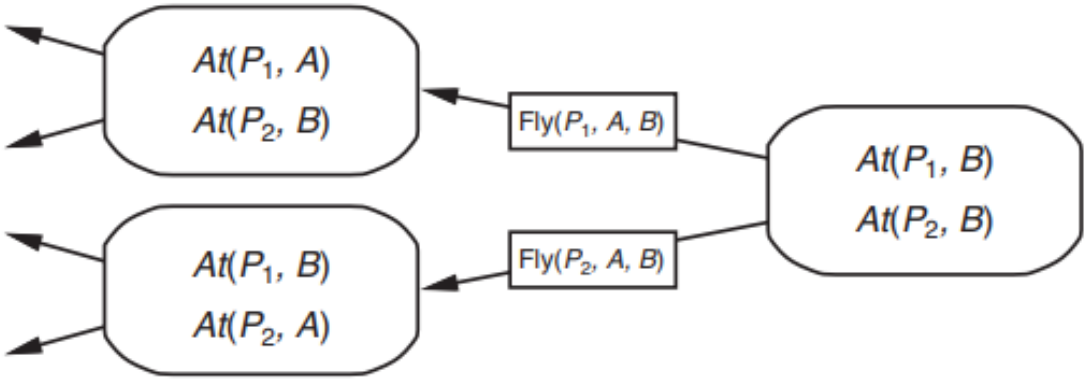
Action (Fly(p, from, to),

PRECOND: At(p, from) ^ Plane (p) ^ Airport (from) ^ Airport (to)

EFFECT: At(p, from) i At(p, to))

Backward state-space search (Regression):

- The goal state must unify with at least one of the positive literals in the operator's effect.
- Its preconditions must hold in the previous situation, and these become subgoals which might be satisfied by the initial conditions.
- Perform backward chaining from goal.
- Again, this is just state-space search using STRIPS operators.



- Backward state search starting from the goal state(s).
- Using the inverse of the actions to search backward for the initial state.

Problem Formulation for Regression:

Initial state:

- Initial state of the planning problem

Actions:

- Applicable to the current state.
- First actions' preconditions are satisfied, Successor states are generated
- Add positive literals to add list and negative literals to delete list.

Goal test:

- Whether the state satisfies the goal of the planning

Step cost:

- Each action is 1 (assumed)

BSSS Algorithm:

- Start at the goal,
- Test the goal is initial state, otherwise
- Apply inverses of the planning operators to produce subgoals,
- The algorithm will stop if we produce a set of subgoals that satisfies the initial state.

Or

1. Backward-search (O, so, g)
2. π ←the empty plan
3. loop
   1. if so satisfies g then return π
   2. applicable {a | a is a ground instance of an operator in O that is relevant for g}
   3. if applicable = Ø then return failure
   4. Non-deterministically choose an action a € applicable
   5. π←α. π
   6. g←y-1(g, a)

PLANNING GRAPHS

All the heuristics we have suggested can suffer from inaccuracies. Here we use a special data structure called a planning graph can be used to give better heuristic estimates.

These heuristics can be applied to any of the search techniques we have seen so far. Alternatively, we can search for a solution over the space formed by the planning graph, using an algorithm called **GRAPHPLAN**.

A planning problem asks if we can reach a goal state from the initial state. Suppose we are given a tree of all possible actions from the initial state to successor states, and their successors, and so on.

If we indexed this tree appropriately, we could answer the planning question "can we reach state G from state S0" immediately, just by looking it up. Of course, the tree is of exponential size, so this approach is impractical.

A planning graph is polynomial size approximation to this tree that can be constructed quickly. The planning graph can't answer definitively whether G is reachable from S0, but it can estimate how many steps it takes to reach G. The estimate is always correct when it reports the goal is not reachable, and it never overestimates the number of steps, so it is an **admissible heuristic.**

A planning graph is a directed graph organized into levels:

first a level S0 for the initial state, consisting of nodes representing each fluent that holds in S0;

then a level A0 consisting of nodes for each ground action that might be applicable in S0; then alternating levels Si followed by Ai; until we reach a termination condition.

Roughly speaking, Si contains all the literals that could hold at time i, depending on the actions executed at preceding time steps.

If it is possible that either P or ¬P could hold, then both will be represented in Si. Also roughly speaking, Ai contains all the actions that could have their preconditions satisfied at time i.

We say "roughly speaking" because the planning graph records only a restricted subset of the possible negative interactions among actions; therefore, a literal might show up at level **Sj** when it could not be true until a later level, if at all. Despite the possible error, the level **j** at which a literal first appears is a good estimate of how difficult it is to achieve the literal from the initial state.

Planning graphs work only for propositional planning problems—ones with no variables. it is straightforward to propositionalize a set of action schemas. Despite the resulting increase in the size of the problem description, planning graphs have proved to be effective tools for solving hard planning problems.

$Init(Have(Cake))$
$Goal(Have(Cake) \wedge Eaten(Cake))$
$Action(Eat(Cake)$
PRECOND: $Have(Cake)$
EFFECT: $\neg Have(Cake) \wedge Eaten(Cake))$
$Action(Bake(Cake)$
PRECOND: $\neg Have(Cake)$
EFFECT: $Have(Cake))$

Figure 10.7 The "have cake and eat cake too" problem.

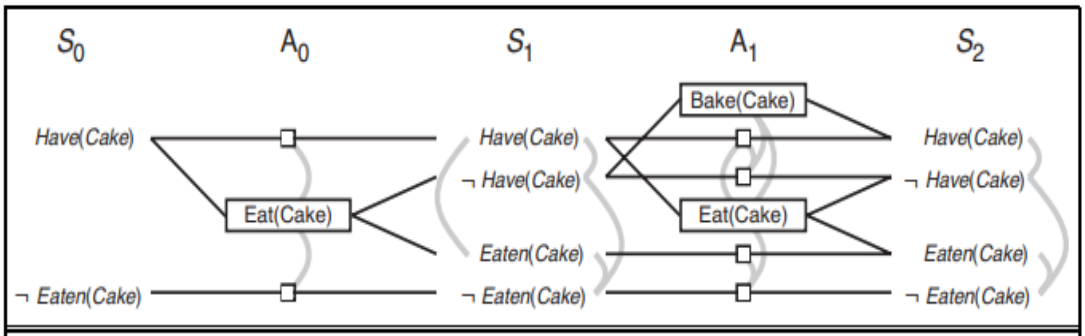


Figure 10.8 The planning graph for the "have cake and eat cake too" problem up to level S_2. Rectangles indicate actions (small squares indicate persistence actions), and straight lines indicate preconditions and effects. Mutex links are shown as curved gray lines. Not all mutex links are shown, because the graph would be too cluttered. In general, if two literals are mutex at S_i, then the persistence actions for those literals will be mutex at A_i and we need not draw that mutex link.

Figure 10.7 shows a simple planning problem, and Figure 10.8 shows its planning graph.

Each action at level **Ai** is connected to its preconditions at **Si** and its effects at **Si+1**. So a literal appears because an action caused it, but we also want to say that a literal can persist if no action negates it.

This is represented by a persistence action (sometimes called PERSISTENCE ACTION a no-op). For every literal **C**, we add to the problem a persistence action with precondition **C** and effect **C**.

Level A0 in Figure 10.8 shows one "real" action, **Eat(Cake)**, along with two persistence actions drawn as small square boxes.

Level A0 contains all the actions that could occur in state S0, but just as important it records conflicts between actions that would prevent them from occurring together.

The gray lines in Figure 10.8 indicate mutual exclusion (or mutex) links. For example, **Eat(Cake)** is mutually exclusive with the persistence of either **Have(Cake)** or **¬Eaten(Cake)**. We shall see shortly how mutex links are computed.

Level **S1** contains all the literals that could result from picking any subset of the actions in **A0**, as well as mutex links (gray lines) indicating literals that could not appear together, regardless of the choice of actions.

For example, **Have(Cake)** and **Eaten(Cake)** are mutex depending on the choice of actions in **A0**, either, but not both, could be the result.

In other words, **S1** represents a belief state: a set of possible states. The members of this set are all subsets of the literals such that there is no mutex link between any members of the subset.

We continue in this way, alternating between state level **Si** and action level **Ai** until we reach a point where two consecutive levels are identical. At this point, we say that the graph has leveled off.

The graph in Figure 10.8 levels off at **S2**. What we end up with is a structure where every **Ai** level contains all the actions that are applicable in **Si**, along with constraints saying that two actions cannot both be executed at the same level.

Every **Si** level contains all the literals that could result from any possible choice of actions in **Ai−1**, along with constraints saying which pairs of literals are not possible.

It is important to note that the process of constructing the planning graph does not require choosing among actions, which would entail combinatorial search. Instead, it just records the impossibility of certain choices using mutex links. We now define mutex links for both actions and literals.

A mutex relation holds between two actions at a given level if any of the following three conditions holds:

• **Inconsistent effects:** one action negates an effect of the other. For example, **Eat(Cake)** and the persistence of **Have(Cake)** have inconsistent effects because they disagree on the effect **Have(Cake).**

• **Interference:** one of the effects of one action is the negation of a precondition of the other. For example **Eat(Cake)** interferes with the persistence of **Have(Cake)** by negating its precondition.

• **Competing needs:** one of the preconditions of one action is mutually exclusive with a precondition of the other. For example, **Bake(Cake)** and **Eat(Cake)** are mutex because they compete on the value of the **Have(Cake)** precondition.

A mutex relation holds between two literals at the same level if one is the negation of the other or if each possible pair of actions that could achieve the two literals is mutually exclusive. This condition is called inconsistent support.

For example, **Have(Cake)** and **Eaten(Cake)** are mutex in **S1** because the only way of achieving **Have(Cake)**, the persistence action, is mutex with the only way of achieving **Eaten(Cake),** namely **Eat(Cake).**

In S2 the two literals are not mutex, because there are new ways of achieving them, such as Bake(Cake) and the persistence of Eaten(Cake), that are not mutex.

A planning graph is polynomial in the size of the planning problem. For a planning problem with **l literals** and **a actions**, each **Si** has no more than **l nodes** and **l^2 mutex links**, and each **Ai** has no more than **a + l nodes** (including the no-ops), **$(a + l)^2$ mutex links**, and **2(al + l) precondition** and **effect links**.

Thus, an entire graph with **n levels** has a size of **$O(n(a + l)^2)$.** The time to build the graph has the same complexity.

Graph plan Algorithm

Graph plan algorithm is used to describe a process of extracting a plan directly from a planning graph instead of graph that produces a heuristic. this algorithm consists of two steps which are one after the other in a loop. Initially, it verifies that all the goal literals exist in the current level and mutex links does not exists in any of the pairs. If this case occurs, then a solution can be obtained within a current graph.

A graph is expanded with the addition of actions and state literals for the current level and next level respectively. This process is continued till a solution is obtained or it is known that a solution cannot be obtained.

```
function GRAPHPLAN(problem) returns solution or failure
  graph ← INITIAL-PLANNING-GRAPH(problem)
  goals ← CONJUNCTS(problem.GOAL)
  nogoods ← an empty hash table
  for tl = 0 to ∞ do
    if goals all non-mutex in S_t of graph then
      solution ← EXTRACT-SOLUTION(graph, goals, NUMLEVELS(graph), nogoods)
      if solution ≠ failure then return solution
    if graph and nogoods have both leveled off then return failure
    graph ← EXPAND-GRAPH(graph, problem)
```

Figure 10.9 The GRAPHPLAN algorithm. GRAPHPLAN calls EXPAND-GRAPH to add a level until either a solution is found by EXTRACT-SOLUTION, or no solution is possible.

Example:

The operation or working of the graph plan is explained using a spare tire problem. The planning graph for a spare tire problem is shown in figure 10.10 as follows,

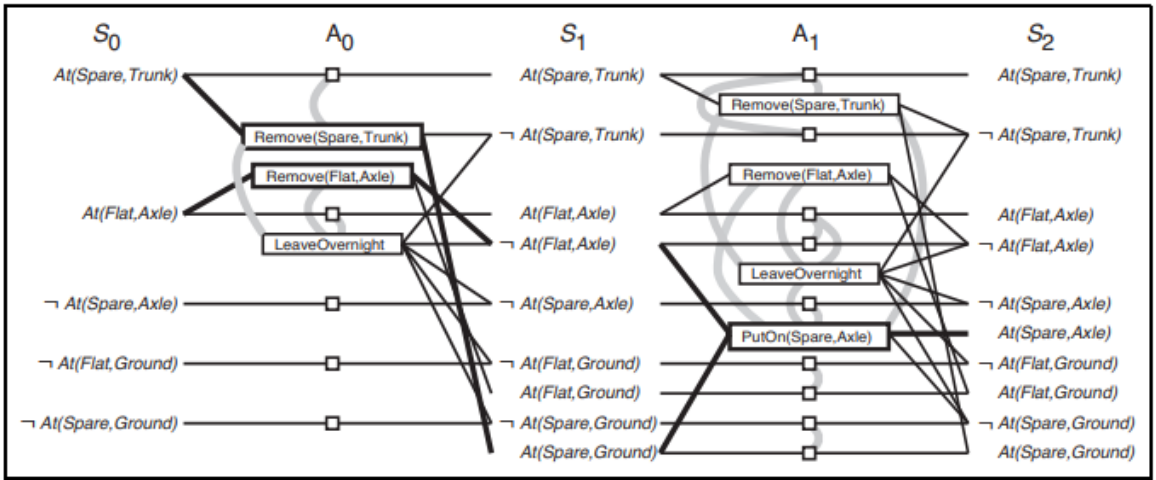


Figure 10.10 The planning graph for the spare tire problem after expansion to level S_2. Mutex links are shown as gray lines. Not all links are shown, because the graph would be too cluttered if we showed them all. The solution is indicated by bold lines and outlines.

The graph is shown in Figure 10.10. The first line of GRAPHPLAN initializes the planning graph to a one-level (S0) graph representing the initial state. The positive fluents from the problem description's initial state are shown, as are the relevant negative fluents. Not shown are the unchanging positive literals (such as Tire(Spare)) and the irrelevant negative literals.

The goal At(Spare, Axle) is not present in S0, so we need not call EXTRACT-SOLUTION— we are certain that there is no solution yet. Instead, EXPAND-GRAPH adds into A0 the three actions whose preconditions exist at level S0 (i.e., all the actions except PutOn(Spare, Axle)), along with persistence actions for all the literals in S0. The effects of the actions are added at level S1. EXPAND-GRAPH then looks for mutex relations and adds them to the graph.

At(Spare, Axle)is still not present in S1, so again we do not call EXTRACT-SOLUTION. We call EXPAND-GRAPH again, adding A1 and S1 and giving us the planning graph shown in Figure 10.10. Now that we have the full complement of actions, it is worthwhile to look at some of the examples of mutex relations and their causes:

- **Inconsistent effects:** Remove(Spare, Trunk) is mutex with LeaveOvernight because one has the effect At(Spare, Ground) and the other has its negation.
- **Interference:** Remove(Flat, Axle) is mutex with LeaveOvernight because one has the precondition At(Flat, Axle) and the other has its negation as an effect.
- **Competing needs:** PutOn(Spare, Axle) is mutex with Remove(Flat, Axle) because one has At(Flat, Axle) as a precondition and the other has its negation.
- **Inconsistent support:** At(Spare, Axle) is mutex with At(Flat, Axle) in S2 because the only way of achieving At(Spare, Axle) is by PutOn(Spare, Axle), and that is mutex with the persistence action that is the only way of achieving At(Flat, Axle). Thus, the mutex relations detect the immediate conflict that arises from trying to put two objects in the same place at the same time.

This time, when we go back to the start of the loop, all the literals from the goal are present in S2, and none of them is mutex with any other. That means that a solution might exist, and EXTRACT-SOLUTION will try to find it.

We can formulate EXTRACT-SOLUTION as a Boolean constraint satisfaction problem (CSP) where the variables are the actions at each level, the values for each variable are in or out of the plan, and the constraints are the mutexes and the need to satisfy each goal and precondition.

Alternatively, we can define EXTRACT-SOLUTION as a backward search problem, where each state in the search contains a pointer to a level in the planning graph and a set of unsatisfied goals. We define this search problem as follows:

- The initial state is the last level of the planning graph, Sn, along with the set of goals from the planning problem.

- The actions available in a state at level Si are to select any conflict-free subset of the actions in Ai−1 whose effects cover the goals in the state. The resulting state has level Si−1 and has as its set of goals the preconditions for the selected set of actions. By "conflict free," we mean a set of actions such that no two of them are mutex and no two of their preconditions are mutex.
- The goal is to reach a state at level S0 such that all the goals are satisfied.
- The cost of each action is 1.

For this particular problem, we start at S2 with the goal At(Spare, Axle).

The only choice we have for achieving the goal set is PutOn(Spare, Axle). That brings us to a search state at S1 with goals At(Spare, Ground) and ¬At(Flat, Axle).

The former can be achieved only by Remove(Spare, Trunk), and the latter by either Remove(Flat, Axle) or LeaveOvernight. But LeaveOvernight is mutex with Remove(Spare, Trunk), so the only solution is to choose Remove(Spare, Trunk) and Remove(Flat, Axle).

That brings us to a search state at S0 with the goals At(Spare, Trunk) and At(Flat, Axle). Both of these are present in the state, so we have a solution: the actions Remove(Spare, Trunk) and Remove(Flat, Axle) in level A0, followed by PutOn(Spare, Axle) in A1.

Probabilistic Reasoning

Uncertainty:

Till now, we have learned knowledge representation using first-order logic and propositional logic with certainty, which means we were sure about the predicates. With this knowledge representation, we might write A→B, which means if A is true then B is true but consider a situation where we are not sure about whether A is true or not then we cannot express this statement, this situation is called uncertainty.

So, to represent uncertain knowledge, where we are not sure about the predicates, we need uncertain reasoning or probabilistic reasoning.

Causes of uncertainty:

Following are some leading causes of uncertainty to occur in the real world.

1. Information occurred from unreliable sources.
2. Experimental Errors
3. Equipment fault
4. Temperature variation
5. Climate change.

Probabilistic reasoning:

Probabilistic reasoning is a way of knowledge representation where we apply the concept of probability to indicate the uncertainty in knowledge. In probabilistic reasoning, we combine probability theory with logic to handle the uncertainty.

We use probability in probabilistic reasoning because it provides a way to handle the uncertainty that is the result of someone's laziness and ignorance.

In the real world, there are lots of scenarios, where the certainty of something is not confirmed, such as "It will rain today," "behavior of someone for some situations," "A match between two teams or two players." These are probable sentences for which we can assume that it will happen but not sure about it, so here we use probabilistic reasoning.

Need of probabilistic reasoning in AI:

- When there are unpredictable outcomes.
- When specifications or possibilities of predicates becomes too large to handle.
- When an unknown error occurs during an experiment.

In probabilistic reasoning, there are two ways to solve problems with uncertain knowledge:

- **Bayes' rule**
- **Bayesian Statistics**

As probabilistic reasoning uses probability and related terms, so before understanding probabilistic reasoning, let's understand some common terms:

Probability: Probability can be defined as a chance that an uncertain event will occur. It is the numerical measure of the likelihood that an event will occur. The value of probability always remains between 0 and 1 that represent ideal uncertainties.

$0 \leq P(A) \leq 1$, where P(A) is the probability of an event A.

P(A) = 0, indicates total uncertainty in an event A.

P(A) =1, indicates total certainty in an event A.

We can find the probability of an uncertain event by using the below formula.

$$\textbf{Probability of occurrence} = \frac{\text{Number of desired outcomes}}{\text{Total number of outcomes}}$$

- P(¬A) = probability of a not happening event.
- P(¬A) + P(A) = 1.

Event: Each possible outcome of a variable is called an event.

Sample space: The collection of all possible events is called sample space.

Random variables: Random variables are used to represent the events and objects in the real world.

Prior probability: The prior probability of an event is probability computed before observing new information.

Posterior Probability: The probability that is calculated after all evidence or information has considered. It is a combination of prior probability and new information.

Conditional probability:

Conditional probability is a probability of occurring an event when another event has already happened.

Let's suppose, we want to calculate the event A when event B has already occurred, "the probability of A under the conditions of B", it can be written as:

$$P(A|B) = \frac{P(A \wedge B)}{P(B)}$$

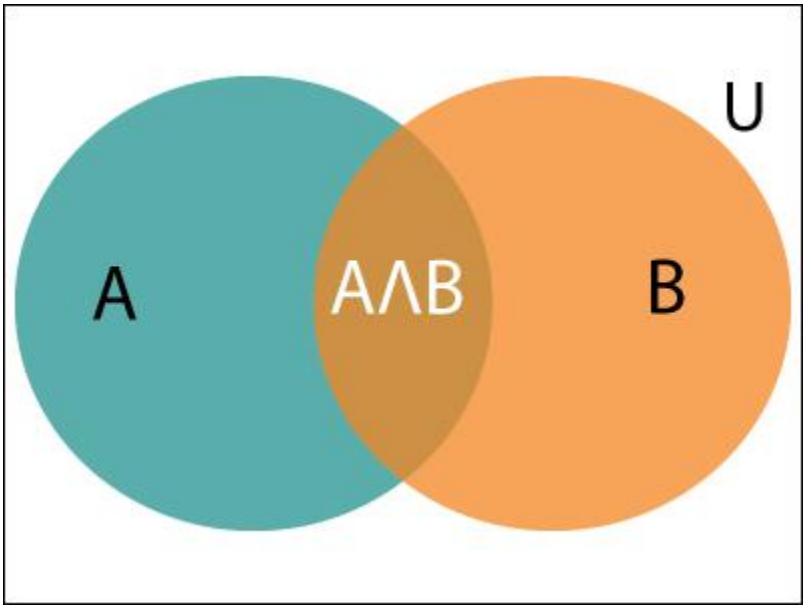
Where $P(A \wedge B)$= Joint probability of a and B

P(B)= Marginal probability of B.

If the probability of A is given and we need to find the probability of B, then it will be given as:

$$P(B|A) = \frac{P(A \wedge B)}{P(A)}$$

It can be explained by using the below Venn diagram, where B is occurred event, so sample space will be reduced to set B, and now we can only calculate event A when event B is already occurred by dividing the probability of **P(A∧*B*) by P (B)**.



Example:

In a class, there are 70% of the students who like English and 40% of the students who likes English and mathematics, and then what is the percent of students those who like English also like mathematics?

Solution:

Let, A is an event that a student likes Mathematics

B is an event that a student likes English.

$$P(A|B) = \frac{P(A \wedge B)}{P(B)} = \frac{0.4}{0.7} = 57\%$$

Hence, 57% are the students who like English also like Mathematics.

Bayes' theorem

Bayes' theorem is also known as **Bayes' rule, Bayes' law**, or **Bayesian reasoning**, which determines the probability of an event with uncertain knowledge.

In probability theory, it relates the conditional probability and marginal probabilities of two random events.

Bayes' theorem was named after the British mathematician **Thomas Bayes**. The **Bayesian inference** is an application of Bayes' theorem, which is fundamental to Bayesian statistics.

It is a way to calculate the value of P(B|A) with the knowledge of P(A|B).

Bayes' theorem allows updating the probability prediction of an event by observing new information of the real world.

Example: If cancer corresponds to one's age, then by using Bayes' theorem, we can determine the probability of cancer more accurately with the help of age.

Bayes' theorem can be derived using product rule and conditional probability of event A with known event B:

As from product rule we can write:

P(A ∧ B)= P(A|B) P(B) or

Similarly, the probability of event B with known event A:

P(A ∧ B)= P(B|A) P(A)

Equating right hand side of both the equations, we will get:

$$P(A|B) = \frac{P(B|A)\,P(A)}{P(B)} \quad \text{....(a)}$$

The above equation (a) is called as **Bayes' rule** or **Bayes' theorem**. This equation is basic of most modern AI systems for **probabilistic inference**.

It shows the simple relationship between joint and conditional probabilities. Here,

P(A|B) is known as **posterior**, which we need to calculate, and it will be read as Probability of hypothesis A when we have occurred an evidence B.

P(B|A) is called the likelihood, in which we consider that hypothesis is true, then we calculate the probability of evidence.

P(A) is called the **prior probability**, probability of hypothesis before considering the evidence

P(B) is called **marginal probability**, pure probability of an evidence.

In the equation (a), in general, we can write P (B) = P(A)*P(B|Ai), hence the Bayes' rule can be written as:

$$P(A_i|B) = \frac{P(A_i)*P(B|A_i)}{\sum_{i=1}^{k} P(A_i)*P(B|A_i)}$$

Where A1, A2, A3,....., An is a set of mutually exclusive and exhaustive events.

The more general case of Bayes' rule for multivalued variables can be written in the P notation as follows:

$$\mathbf{P}(Y \mid X) = \frac{\mathbf{P}(X \mid Y)\mathbf{P}(Y)}{\mathbf{P}(X)} ,$$

As before, this is to be taken as representing a set of equations, each dealing with specific values of the variables. We will also have occasion to use a more general version conditionalized on some background evidence e:

$$\mathbf{P}(Y \mid X, \mathbf{e}) = \frac{\mathbf{P}(X \mid Y, \mathbf{e})\mathbf{P}(Y \mid \mathbf{e})}{\mathbf{P}(X \mid \mathbf{e})} .$$

Applying Bayes' rule:

Bayes' rule allows us to compute the single term P(B|A) in terms of P(A|B), P(*B*), and P(A). This is very useful in cases where we have a good probability of these three terms and want to determine the fourth one. Suppose we want to perceive the effect of some unknown cause, and want to compute that cause, then the Bayes' rule becomes:

$$\mathbf{P(cause|effect)} = \frac{P(effect|cause)\,P(cause)}{P(effect)}$$

Example-1:

Question: what is the probability that a patient has diseases meningitis with a stiff neck?

Given Data:

A doctor is aware that the disease meningitis causes a patient to have a stiff neck, and it occurs 80% of the time. He is also aware of some more facts, which are given as follows:

- The Known probability that a patient has meningitis disease is 1/30,000.
- The Known probability that a patient has a stiff neck is 2%.

Let a be the proposition that patient has stiff neck and b be the proposition that patient has meningitis., so we can calculate the following as:

P(a|b) = 0.8

P(b) = 1/30000

P(a)= .02

$$\mathbf{P(b|a)} = \frac{P(a|b)P(b)}{P(a)} = \frac{0.8*(\frac{1}{30000})}{0.02} = 0.001333333.$$

Hence, we can assume that 1 patient out of 750 patients has meningitis disease with a stiff neck.

Example-2:

Question: From a standard deck of playing cards, a single card is drawn. The probability that the card is king is 4/52, then calculate posterior probability P(King|Face), which means the drawn face card is a king card.

Solution:

$$\mathbf{P(king|face)} = \frac{P(\text{Face}|\text{king}) * P(\text{King})}{P(\text{Face})} \quad \text{.....(i)}$$

P(king): probability that the card is King= 4/52= 1/13

P(face): probability that a card is a face card= 3/13

P(Face|King): probability of face card when we assume it is a king = 1

Putting all values in equation (i) we will get:

$$P(\text{king}|\text{face}) = \frac{1 * (\frac{1}{13})}{(\frac{3}{13})} = \mathbf{1/3,}$$ **it is a probability that a face card is a king card.**

Application of Bayes' theorem:

Following are some applications of Bayes' theorem:

- It is used to calculate the next step of the robot when the already executed step is given.
- Bayes' theorem is helpful in weather forecasting.
- It can solve the Monty Hall problem.